

Le Cours Python

Un manuel en 24 chapitres construit à partir des transcriptions de la série « Python Tutorials » de Corey Schafer — réorganisé, traduit et modernisé pour Python 3.12+, enrichi de 116 exercices corrigés à coloration mathématique et financière.

Partie I — Démarrer

Partie II — Les fondamentaux (ch. 2-9)

Partie III — Python au quotidien (ch. 10-18)

Partie IV — Programmation orientée objet (ch. 19-24)

Sommaire

Partie I — Démarrer

1	Installer Python et lancer son premier programme	5
---	--	---

Partie II — Les fondamentaux

2	Variables et chaînes de caractères (strings).....	11
3	Nombres : entiers et flottants	19
4	Listes, tuples et ensembles (sets).....	25
5	Dictionnaires : clé-valeur	34
6	Conditions et booléens	40
7	Boucles et itérations	47
8	Les fonctions	54
9	Modules et bibliothèque standard	61

Partie III — Python au quotidien

10	pip : installer des paquets tiers.....	69
11	Environnements virtuels : venv	74
12	Lire et écrire des fichiers	79
13	Gérer les erreurs : try / except.....	86
14	Les compréhensions	94
15	CSV : données tabulaires	101
16	JSON : données structurées et APIs	109
17	Dates et heures : datetime	117
18	Naviguer dans le système : os.....	125

Partie IV — Programmation orientée objet

19	POO 1 : classes et instances	134
20	POO 2 : variables de classe	141
21	POO 3 : classmethods et staticmethods	148
22	POO 4 : l'héritage.....	154
23	POO 5 : méthodes spéciales (dunder)	162
24	POO 6 : propriétés	169

Avant-propos

Comment utiliser ce cours

Chaque chapitre suit la vidéo correspondante (liens en tête de chapitre), dans l'ordre pensé par Corey Schafer — mais tout est relu d'un seul tenant : les rappels pointent vers les bons chapitres, les exemples se répondent, et le fil rouge de la partie IV (`CompteBancaire` , `Obligation` ...) grandit d'un chapitre à l'autre. Trois conseils d'usage : **tape le code toi-même** (le copier-coller n'apprend rien aux doigts), fais les **exercices avant** de lire les solutions, et reviens aux **fiches récap** en fin de chapitre pour réviser vite.

Les encadrés

Piège — les erreurs classiques, à lire deux fois : ce sont elles qui coûtent des heures de débogage.

Depuis la vidéo — ce qui a changé entre la vidéo (2015-2019) et Python 3.12+ en 2026 : f-strings, `zoneinfo`, VS Code...

Pour aller plus loin — apartés facultatifs, souvent orientés maths et finance : complexité, flottants, actualisation...

Sources

Ce cours est une réécriture pédagogique : le contenu suit fidèlement les leçons de la série « Python Tutorials » de Corey Schafer (concepts, progression, démonstrations), avec une rédaction, des exemples et des exercices originaux. Les URL complètes figurent en tête de chaque chapitre — et la version web de ce cours rend chaque lien cliquable. Les vidéos restent le meilleur complément : va les voir.

PARTIE I

Démarrer

CHAPITRE 1

Installer Python et lancer son premier programme

Vidéo source : [Python Tutorial for Beginners 1: Install and Setup for Mac and Windows](#) — Corey Schafer

Objectifs : installer Python 3 proprement sous Windows (et Mac), comprendre la différence entre le mode interactif et les scripts, écrire et exécuter ton premier fichier `.py`, choisir un éditeur de code.

1.1 Python 2 ou Python 3 ?

La question ne se pose plus : **Python 3, toujours**. Python 2 est officiellement abandonné depuis le 1^{er} janvier 2020 — plus aucune mise à jour, même de sécurité. Dans la vidéo (tournée à l'époque de Python 3.6), Corey Schafer le disait déjà : « il y a un mouvement quasi universel vers Python 3 ». C'est aujourd'hui un fait accompli.

Depuis la vidéo — La vidéo installe Python 3.6. En 2026, tu installeras Python **3.12 ou 3.13**. La démarche d'installation est rigoureusement identique ; tout ce que montre la série fonctionne pareil, et je signalerai au fil du cours les nouveautés apparues depuis (f-strings généralisées, `zoneinfo`, etc.).

1.2 Installation sous Windows

C'est ton cas, donc on détaille.

Étape 1 — Vérifier si Python est déjà installé. Ouvre l'invite de commandes : touche Windows, tape `cmd`, Entrée.

Puis :

```
C:\Users\mathias> python --version
```

Deux issues possibles : soit une version s'affiche (`Python 3.12.4` par exemple), soit Windows répond que « python n'est pas reconnu en tant que commande interne... ». Dans ce second cas, il faut l'installer.

Étape 2 — Télécharger. Va sur python.org → onglet **Downloads**. Le site détecte ton système et te propose directement le bon installateur. Clique sur le bouton de téléchargement de la dernière version 3.x, puis lance le fichier `.exe`.

Étape 3 — LA case à cocher. Sur le premier écran de l'installateur, **coche impérativement** « Add python.exe to PATH » avant de cliquer sur *Install Now*.

Piège — Cette case est le point critique de toute l'installation, et Corey insiste lourdement dessus. Le *PATH* est la liste des dossiers où Windows cherche les programmes quand tu tapes une commande. Sans cette case, taper `python` dans un terminal ne fonctionnera pas, et il faudra configurer les variables d'environnement à la main — pénible. Si tu as déjà installé Python sans la cocher : désinstalle et réinstalle en la cochant, c'est le plus simple.

Étape 4 — Vérifier. Ferme puis rouvre l'invite de commandes (important : un terminal déjà ouvert ne voit pas le nouveau PATH), et retape :

```
C:\Users\mathias> python --version
Python 3.12.4
```

Depuis la vidéo — Windows installe aussi le lanceur `py` : taper `py` équivaut à `python`, et `py -3.12` permet de choisir une version précise si plusieurs coexistent. Si jamais `python` ouvre le Microsoft Store au lieu de lancer Python (comportement agaçant de Windows), utilise `py`, ou désactive les « alias d'exécution d'application » dans les paramètres Windows.

1.3 Installation sous Mac (pour référence)

La vidéo couvre aussi Mac ; garde ces points en tête si tu changes de machine un jour. On télécharge le `.pkg` sur python.org et on l'exécute. Python s'invoque ensuite avec la commande `python3` (et non `python`). La vidéo montre comment créer un alias dans le fichier de configuration du shell pour que `python` pointe vers `python3` : une ligne `alias python=python3` (sans espaces autour du `=`) à la fin du fichier, puis rouvrir le terminal. Sur les Mac récents le shell par défaut est `zsh` : le fichier est `~/.zprofile` plutôt que le `~/.bash_profile` montré dans la vidéo. L'alias est optionnel — on peut simplement taper `python3` à chaque fois.

1.4 Le mode interactif : l'interpréteur

Python est un langage **interprété** : un programme (l'interpréteur) lit ton code et l'exécute ligne par ligne, sans étape de compilation visible. On peut dialoguer directement avec lui. Dans un terminal :

```
C:\Users\mathias> python
Python 3.12.4 ... on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Les trois chevrons `>>>` forment l'**invite** (*prompt*) : Python attend une instruction, l'exécute, affiche le résultat, et attend la suivante. Essaie :

```
>>> print("hello world")
hello world
>>> x = 10
>>> print(x)
```

```
10
>>> exit()
```

`exit()` (ou Ctrl+Z puis Entrée sous Windows) rend la main au terminal.

Le mode interactif est parfait pour **tester une idée en trois secondes** — on s'en servira sans arrêt, notamment avec `help()` et `dir()` (chapitre 2). Mais il a une limite évidente : tout disparaît quand on quitte, et écrire un programme de quarante lignes ligne à ligne n'a aucun sens. Pour un vrai programme, on écrit un **fichier**.

1.5 Ton premier script : `intro.py`

Un programme Python est un simple fichier texte dont le nom se termine par `.py`. Crée un dossier de travail (par exemple `C:\Users\mathias\Documents\python-cours`), et dedans un fichier `intro.py` contenant :

```
# Affiche un message de bienvenue
print("hello world")
```

Deux choses à voir ici.

Le commentaire. Tout ce qui suit un `#` sur une ligne est un **commentaire** : Python l'ignore complètement à l'exécution. Les commentaires sont destinés aux humains — toi dans six mois, ou quiconque lira ton code. On les utilise pour expliquer le *pourquoi* du code, pas pour paraphraser l'évident.

L'appel de fonction `print()`. `print` affiche à l'écran ce qu'on lui donne entre parenthèses. C'est la fonction que tu utiliseras le plus dans ce cours pour inspecter ce que fait ton code.

Exécution. Dans le terminal, place-toi dans le dossier et lance le script en donnant son nom à `python` :

```
C:\Users\mathias> cd Documents\python-cours
C:\Users\mathias\Documents\python-cours> python intro.py
hello world
```

C'est le cycle de travail fondamental : *éditer le fichier* → *l'exécuter* → *lire la sortie* → *recommencer*. Tout le reste du cours vit dans ce cycle.

Piège — L'extension `.py` est indispensable. Attention aussi à Windows qui masque par défaut les extensions : un fichier « `intro.py` » créé dans le Bloc-notes peut en réalité s'appeler `intro.py.txt`. Active l'affichage des extensions de fichiers dans l'Explorateur (onglet Affichage → Extensions de noms de fichiers).

1.6 IDLE : l'éditeur fourni avec Python

L'installation de Python inclut IDLE, un petit environnement de développement. Tu le trouves dans le menu Démarrer, dossier Python 3.x. À l'ouverture, IDLE présente un mode interactif (le même `>>>` que dans le terminal). Le menu **File** → **New File** ouvre un éditeur dans lequel écrire un script ; **File** → **Save** pour l'enregistrer en `.py` ; **Run** → **Run Module** (F5) pour l'exécuter.

IDLE suffit largement pour suivre ce cours, et c'est ce que Corey utilise dans les premières vidéos pour rester neutre. Mais dès que tu programmes sérieusement, un vrai éditeur change la vie.

1.7 Choisir un éditeur de code

La vidéo mentionne trois options : **Sublime Text** (éditeur léger et rapide, exécution via Tools → Build), **Atom**, et **PyCharm** (IDE complet de JetBrains, avec débogueur intégré et inspection du programme en cours d'exécution). Le message important de Corey n'a pas vieilli : *n'importe quel éditeur de texte fait l'affaire* — certains codent dans Vim ou Emacs — l'essentiel est d'en choisir un et d'apprendre à s'en servir.

Depuis la vidéo — Le paysage a changé : **Atom est abandonné depuis 2022**, et l'éditeur ultra-dominant aujourd'hui est **Visual Studio Code** (gratuit, extension « Python » officielle : coloration, autocomplétion, exécution par ▶, débogueur). C'est mon conseil pour toi : VS Code + extension Python. PyCharm reste une excellente alternative « tout inclus ». Pour ce cours, peu importe : les programmes sont identiques partout.

1.8 Ce qu'il faut retenir avant la suite

À partir du chapitre 2, tout ce que montre la série est **identique sous Windows, Mac et Linux** — l'installation était la seule étape dépendante du système. Le plan de la série que suit ce cours : types de données (chaînes, nombres, listes, dictionnaires), conditions, boucles, fonctions, modules — puis les outils du quotidien et la programmation orientée objet.

Exercices

Exercice 1.1 ● — Vérifie ton installation : ouvre un terminal, affiche la version de Python, entre dans le mode interactif, fais calculer `2 + 2`, affiche ton prénom avec `print`, puis quitte proprement.

Exercice 1.2 ● — Crée un fichier `presentation.py` qui affiche trois lignes : ton prénom, ta ville, et ton objectif avec Python. Ajoute un commentaire en tête de fichier expliquant ce que fait le script. Exécute-le depuis le terminal.

Exercice 1.3 ●● — Dans le mode interactif, tape `x = 10` puis `print(x)`. Quitte avec `exit()`, relance `python`, et tape `print(x)`. Que se passe-t-il et pourquoi ? Qu'est-ce que cela t'apprend sur la différence entre mode interactif et scripts ?

Solutions détaillées

Solution 1.1 —

```
C:\> python --version
Python 3.12.4
```



```
C:\> python
>>> 2 + 2
4
>>> print("Mathias")
Mathias
>>> exit()
```

Note au passage : en mode interactif, taper `2 + 2` seul suffit (l'interpréteur affiche la valeur de chaque expression) ; dans un script, il faudrait `print(2 + 2)` pour voir quelque chose.

Solution 1.2 —

```
# presentation.py – petit script de présentation (chapitre 1)
print("Mathias")
print("France")
print("Objectif : Python pour la finance quantitative")
```

```
C:\...\python-cours> python presentation.py
Mathias
France
Objectif : Python pour la finance quantitative
```

Solution 1.3 — Au second lancement, `print(x)` provoque `NameError: name 'x' is not defined`. Chaque session de l'interpréteur repart de zéro : les variables vivent en mémoire le temps de la session, puis disparaissent. C'est exactement pour cela qu'on écrit des fichiers `.py` : le code est conservé et rejouable à volonté, alors que l'état du mode interactif est éphémère.

Fiche récap — Chapitre 1

À retenir	Détail
Version	Python 3 uniquement (3.12+ en 2026) ; Python 2 est mort
Installation Windows	python.org → Downloads → cocher « Add python.exe to PATH »
Vérifier	<code>python --version</code> (ou <code>py --version</code>) dans un terminal ouvert
Mode interactif	<code>python</code> → invite <code>>>></code> ; quitter avec <code>exit()</code>
Script	fichier texte en <code>.py</code> ; exécuter avec <code>python fichier.py</code>
Commentaire	<code># tout ce qui suit est ignoré par Python</code>
Afficher	<code>print("...")</code>
Éditeur	VS Code recommandé (IDLE fourni ; PyCharm en IDE complet)

PARTIE II

Les fondamentaux

CHAPITRE 2

Variables et chaînes de caractères (strings)

Vidéos sources : [Python Tutorial for Beginners 2: Strings - Working with Textual Data](#) et [Python Quick Tip: F-Strings](#) — Corey Schafer

Objectifs : créer des variables, manipuler du texte (indexation, découpage, méthodes), formater des chaînes proprement avec les f-strings, et apprendre à explorer un objet avec `dir()` et `help()`.

2.1 Les variables

Une **variable** est un nom qui désigne une valeur en mémoire. On la crée par une simple **assignment** avec `=` :

```
message = "hello world"
print(message)
# → hello world
```

Partout où la variable apparaît ensuite, Python utilise la valeur qu'elle désigne. Remarque ce qui *n'est pas* là : pas de point-virgule en fin de ligne, pas de déclaration de type (`String message = ...` comme en Java). Python détermine le type tout seul et structure le code par les retours à la ligne et l'indentation.

Conventions de nommage (importantes dès maintenant, elles rendent le code lisible) :

- tout en minuscules, mots séparés par des underscores : `taux_interet`, `my_message` — c'est le style dit *snake_case*, standard en Python (par opposition au *camelCase* `myMessage` d'autres langages) ;
- des noms **descriptifs** : `message` dit ce que contient la variable ; `m` ne dit rien. Corey insiste : le code est lu bien plus souvent qu'il n'est écrit.

2.2 Créer des chaînes : guillemets simples, doubles, triples

Une valeur textuelle est une **chaîne de caractères** (*string*, type `str`). On la délimite par des guillemets simples `'...'` ou doubles `"..."` — les deux sont strictement équivalents. Le choix devient important quand le texte contient lui-même un guillemet :

```
# message = 'Bobby's World'    # SyntaxError : Python croit la chaîne finie après Bobby
```

Deux solutions :

```
message = 'Bobby\'s World'    # 1) échapper l'apostrophe avec un antislash
message = "Bobby's World"    # 2) plus lisible : délimiter avec l'autre type de guillemets
```

L'antislash `\` est le **caractère d'échappement** : il retire (ou donne) un sens spécial au caractère suivant. Tu croiseras surtout `\'`, `\"`, `\n` (retour à la ligne) et `\t` (tabulation).

Pour un texte sur **plusieurs lignes**, on utilise les triples guillemets `'''...'''` ou `"""..."""`, qui conservent les retours à la ligne tels quels :

```
message = """Ceci est un bon
dessin animé des années 90"""
print(message)
# → Ceci est un bon
# → dessin animé des années 90
```

2.3 Longueur et indexation

`len()` renvoie le nombre de caractères, **espaces compris** :

```
message = "hello world"
print(len(message))
# → 11
```

Une chaîne est une **séquence** : chaque caractère occupe une position numérotée, son **index**. On y accède avec des crochets — et attention, **la numérotation commence à 0** :

```
print(message[0])    # → h          (premier caractère)
print(message[10])   # → d          (dernier : longueur 11 → indices 0 à 10)
# print(message[11]) # IndexError: string index out of range
```

Piège — Le décalage d'indice est LA source d'erreurs du débutant : pour une chaîne de longueur n , les indices valides vont de 0 à $n - 1$. Demander l'indice n lève une `IndexError`. (Les indices négatifs, très pratiques, arrivent au chapitre 4 : `message[-1]` désigne le dernier caractère.)

2.4 Le slicing : extraire une tranche

La syntaxe `[début:fin]` extrait une sous-chaîne. Règle d'or : **l'indice de début est inclus, l'indice de fin est exclu**.

```
message = "hello world"
print(message[0:5])    # → hello      (indices 0,1,2,3,4 – le 5 est exclu)
print(message[:5])     # → hello      (début omis = depuis le début)
print(message[6:11])   # → world      (fin omise = jusqu'au bout)
print(message[6:])     # → world
```

Cette convention « fin exclue » peut sembler arbitraire, mais elle a de jolies propriétés : `message[:5]` contient exactement 5 caractères, et `message[:i] + message[i:]` reconstitue toujours la chaîne entière. En notation mathématique, `[a:b]` correspond à l'intervalle semi-ouvert $[a, b[$.

Pour aller plus loin — Le slicing accepte un troisième paramètre, le **pas** : `message[::2]` prend un caractère sur deux, et `message[::-1]` renverse la chaîne. Corey y consacre une vidéo dédiée ; retiens au moins `[::-1]`, classique des entretiens.

2.5 Les méthodes de chaînes

Chaque type de données embarque des **méthodes** : des fonctions attachées à l'objet, qu'on appelle avec la notation pointée `objet.methode()`. (À ce stade, retiens simplement : une méthode est une fonction qui appartient à un objet — la nuance précise s'éclaircira en POO, chapitre 19.)

```
message = "hello world"

print(message.lower())    # → hello world      (tout en minuscules)
print(message.upper())    # → HELLO WORLD      (tout en majuscules)
print(message.count("l")) # → 3                (nombre d'occurrences)
print(message.count("hello")) # → 1
print(message.find("world")) # → 6              (index où commence la sous-chaîne)
print(message.find("universe")) # → -1          (introuvable → -1, pas d'erreur)
```

`replace()` remplace une sous-chaîne par une autre :

```
message = "hello world"
new_message = message.replace("world", "universe")
print(new_message)    # → hello universe
```

Piège capital — Les méthodes de chaînes **ne modifient jamais la chaîne d'origine** : elles renvoient une nouvelle chaîne. `message.replace(...)` seul ne change rien à `message` :

```
message = "hello world"
message.replace("world", "universe")    # résultat calculé... puis jeté !
print(message)                          # → hello world (inchangé)
```

Il faut **récupérer la valeur renvoyée**, typiquement en réassignant : `message = message.replace("world", "universe")`. Cette réassignation d'une variable « sur elle-même » est parfaitement idiomatique et très courante. La raison profonde : les chaînes Python sont **immuables** (*immutable*) — on ne peut pas les modifier en place, seulement en créer de nouvelles. On retrouvera cette notion au chapitre 4 avec les tuples.

2.6 Concaténation : l'opérateur +

+ colle des chaînes bout à bout :

```
greeting = "Hello"
name = "Michael"
```

```
message = greeting + name
print(message)          # → HelloMichael    (aïe, pas d'espace)

message = greeting + ", " + name + ". Welcome!"
print(message)          # → Hello, Michael. Welcome!
```

Ça fonctionne, mais dès qu'il y a plus de deux morceaux, la ligne devient un puzzle de guillemets et de `+` difficile à relire — Corey le dit lui-même : « ça commence à devenir compliqué ». D'où les outils de formatage.

2.7 Formater : `.format()` puis les f-strings

La méthode `.format()` (l'outil principal à l'époque de la vidéo) : on écrit la phrase avec des accolades `{}` en guise de trous, et `.format()` les remplit dans l'ordre :

```
message = "{}, {}. Welcome!".format(greeting, name)
print(message)    # → Hello, Michael. Welcome!
```

Les f-strings (Python 3.6+) : on préfixe la chaîne d'un `f`, et on met les variables **directement dans les accolades** :

```
message = f"{greeting}, {name}. Welcome!"
print(message)    # → Hello, Michael. Welcome!
```

Finis les allers-retours entre les trous et la liste d'arguments : on lit la phrase telle qu'elle s'affichera.

Depuis la vidéo — Corey disait en 2017 « peu de gens utilisent encore les f-strings ». En 2026, ce sont **LE standard absolu** : utilise-les par défaut, tu ne verras `.format()` que dans du code ancien. Tout ce cours les emploie systématiquement.

Les accolades acceptent en réalité **n'importe quelle expression Python**, pas seulement une variable :

```
name = "michael"
print(f"Bienvenue, {name.upper()} !")    # → Bienvenue, MICHAEL !    (appel de méthode)
print(f"4 fois 11 font {4 * 11}")        # → 4 fois 11 font 44      (calcul)

person = {"name": "Jen", "age": 23}
print(f"{person['name']} a {person['age']} ans")    # → Jen a 23 ans    (accès dictionnaire)
```

Piège — Guillemets dans les guillemets : si la f-string est délimitée par `'...'` et qu'on accède à `person['name']`, les apostrophes entrent en collision. Solution simple : délimiter la f-string avec `"` et garder `'` à l'intérieur (Corey : « je préfère changer les guillemets que d'empiler des échappements »).

2.8 Les spécifications de format

Après un `:` dans l'accolade, on contrôle *la présentation* de la valeur. Les quatre cas de la vidéo f-strings, tous très utiles :

```
for n in [1, 9, 10]:
    print(f"La valeur est {n:02}")
# → La valeur est 01
# → La valeur est 09
# → La valeur est 10
```

`:02` = « au moins 2 caractères, complétés par des zéros » — parfait pour des numéros de fichiers ou des dates. `:04` donnerait 0001, 0009, 0010.

```
pi = 3.14159265
print(f"pi vaut environ {pi:.4f}") # → pi vaut environ 3.1416 (4 décimales, arrondi)
```

```
from datetime import datetime
birthday = datetime(1990, 1, 1)
print(f"Anniversaire : {birthday:%B %d, %Y}") # → Anniversaire : January 01, 1990
```

Les codes `%B`, `%d`, `%Y`... sont ceux de `strftime`, qu'on verra en détail au chapitre 17 — même Corey avoue devoir les rechercher à chaque fois dans la documentation.

Pour aller plus loin (finance) — Deux spécifications que tu utiliseras sans arrêt avec des chiffres : le séparateur de milliers et le pourcentage.

```
montant = 1234567.891
print(f"{montant:,.2f} €") # → 1,234,567.89 €
taux = 0.0525
print(f"taux : {taux:.2%}") # → taux : 5.25%
```

2.9 Explorer un objet : `dir()` et `help()`

Comment connaître toutes les méthodes disponibles sur une chaîne ? Sans quitter Python :

```
message = "hello world"
print(dir(message))
# → ['__add__', '__class__', ..., 'capitalize', 'casefold', 'center', 'count',
#     'find', 'format', ..., 'lower', ..., 'replace', ..., 'upper', 'zfill']
```

`dir()` liste tous les attributs et méthodes de l'objet. Ignore pour l'instant les noms encadrés de doubles underscores (les méthodes `__dunder__` — chapitre 23) ; le reste, ce sont les méthodes utilisables directement.

`dir` donne les noms, `help` donne le mode d'emploi :

```
help(str)           # documentation complète du type str, méthode par méthode
help(str.lower)     # documentation d'une seule méthode
```

```
lower(self, /)
    Return a copy of the string converted to lowercase.
```

Réflexe à prendre : avant de chercher sur Google, `help()` répond souvent plus vite — et ça marche pour *tous* les objets Python, pas seulement les chaînes.

Exercices

Exercice 2.1 ● — Crée une variable `citation` contenant exactement le texte : `L'analyse, c'est "voir" l'infini.` (apostrophe ET guillemets doubles inclus). Affiche-la, puis affiche sa longueur.

Exercice 2.2 ● — Soit `phrase = "python pour la finance"`. En une expression à chaque fois : affiche le premier caractère, le dernier caractère (sans compter à la main : utilise `len`), le mot `"python"` extrait par slicing, et la phrase entière en majuscules.

Exercice 2.3 ●● — Explique la sortie de ce programme *avant* de l'exécuter, puis vérifie :

```
ticker = "AAPL"
ticker.lower()
print(ticker)
```

Corrige-le pour qu'il affiche `aapl`.

Exercice 2.4 ●● — Avec `nom = "Mathias"` et `capital = 10000`, produis exactement cette ligne à l'aide d'une seule f-string : `Mathias investit 10,000.00 € au taux de 4.50%` (le taux étant stocké dans `taux = 0.045`).

Exercice 2.5 ●●● — Un identifiant client est stocké sous la forme `"martin.dupont.1987"`. À l'aide de `find` et du slicing (on n'a pas encore vu `split`), extrais le prénom dans une variable `prenom` et affiche-le avec une majuscule initiale (cherche la méthode adéquate avec `dir(str)` / `help`).

Solutions détaillées

Solution 2.1 —

```
citation = 'L\'analyse, c\'est "voir" l\'infini.'
# ou, plus simple à lire, avec des triples quotes :
citation = """L'analyse, c'est "voir" l'infini."""
```



```
print(citation)      # → L'analyse, c'est "voir" l'infini.
print(len(citation)) # → 33
```

La chaîne contenant à la fois des apostrophes et des guillemets doubles, la version triple-quotes évite tout échappement.

Solution 2.2 —

```
phrase = "python pour la finance"
print(phrase[0])      # → p
print(phrase[len(phrase) - 1]) # → e (indice n-1 ; dès le ch. 4 on écrira phrase[-1])
print(phrase[0:6])    # → python
print(phrase.upper()) # → PYTHON POUR LA FINANCE
```

Solution 2.3 — Le programme affiche `AAPL`, inchangé : `ticker.lower()` calcule une nouvelle chaîne `"aapl"` ... qui n'est stockée nulle part, car les chaînes sont immuables et la méthode ne modifie pas l'original. Correction :

```
ticker = "AAPL"
ticker = ticker.lower()
print(ticker) # → aapl
```

Solution 2.4 —

```
nom = "Mathias"
capital = 10000
taux = 0.045
print(f"{nom} investit {capital:,.2f} € au taux de {taux:.2%}")
# → Mathias investit 10,000.00 € au taux de 4.50%
```

Solution 2.5 —

```
identifiant = "martin.dupont.1987"
fin_prenom = identifiant.find(".") # index du premier point → 6
prenom = identifiant[:fin_prenom] # slicing jusqu'au point (exclu)
print(prenom.capitalize())        # → Martin
```

`capitalize()` (trouvée via `dir(str)`) met la première lettre en majuscule et le reste en minuscules. Au chapitre 4, `identifiant.split(".")` fera le découpage en une seule opération.

Fiche récap — Chapitre 2

Syntaxe	Effet
<code>x = valeur</code>	assignation ; noms en <code>snake_case</code> descriptif
<code>'...' / "..." / """..."""</code>	chaîne ; triple quotes = multi-lignes ; <code>\</code> échappement

Syntaxe	Effet
<code>len(s)</code>	longueur (espaces compris)
<code>s[0]</code> , <code>s[i]</code>	caractère à l'index i — commence à 0 , max $n-1$
<code>s[a:b]</code>	tranche : a inclus, b exclu ; <code>s[:b]</code> , <code>s[a:]</code>
<code>s.lower()</code> / <code>s.upper()</code>	casse — renvoie une nouvelle chaîne (immutabilité !)
<code>s.count(sub)</code>	nombre d'occurrences
<code>s.find(sub)</code>	index de la 1 ^{ère} occurrence, <code>-1</code> si absente
<code>s.replace(a, b)</code>	remplace — penser à réassigner : <code>s = s.replace(a, b)</code>
<code>a + b</code>	concaténation (vite illisible)
<code>f"{x}, {y}"</code>	f-string : le standard ; expressions permises dans <code>{}</code>
<code>f"{n:02}"</code> , <code>f"{x:.4f}"</code>	formats : zéros de tête, décimales ; <code>{v:,.2f}</code> , <code>{t:.2%}</code>
<code>dir(obj)</code> , <code>help(str.lower)</code>	lister les méthodes / lire la doc sans quitter Python

CHAPITRE 3

Nombres : entiers et flottants

Vidéo source : [Python Tutorial for Beginners 3: Integers and Floats - Working with Numeric Data](#) — Corey Schafer

Objectifs : manier les deux types numériques de base, tous les opérateurs arithmétiques et de comparaison, les fonctions `abs` / `round`, et les conversions de type (casting).

3.1 Deux types : `int` et `float`

Les nombres se représentent le plus souvent par des **entiers** (`int`, *integer* : nombre sans partie décimale) et des **flottants** (`float`, nombre à virgule). La fonction intégrée `type()` révèle le type de n'importe quel objet :

```
num = 3
print(type(num))    # → <class 'int'>

num = 3.14
print(type(num))    # → <class 'float'>
```

Remarque : la même variable `num` a pu désigner successivement un `int` puis un `float`. En Python, le type est porté par la *valeur*, pas par la variable — c'est le **typage dynamique**.

Pour aller plus loin — Les `int` de Python sont à précision arbitraire : `2**1000` se calcule exactement, sans débordement (pas de limite à 2^{31} comme en C). Les `float`, eux, sont des flottants IEEE 754 double précision — avec les limites qu'on verra en 3.7. Pour la lisibilité des grands nombres, Python autorise les underscores : `capital = 1_000_000`.

3.2 Les opérateurs arithmétiques

```
print(3 + 2)    # → 5        addition
print(3 - 2)    # → 1        soustraction
print(3 * 2)    # → 6        multiplication
print(3 / 2)    # → 1.5      division
print(3 // 2)   # → 1        division entière (floor division)
print(3 ** 2)   # → 9        puissance (exposant)
print(3 % 2)    # → 1        modulo (reste de la division)
```

Trois d'entre eux méritent qu'on s'arrête.

La division `/` renvoie *toujours* un `float` en Python 3, même quand elle tombe juste (`4 / 2` → `2.0`). Corey signale le piège historique : en Python 2, `3 / 2` donnait `1` (la décimale était jetée). C'est l'une des grandes ruptures entre les deux versions — si tu lis du vieux code, méfiance.

La **division entière** `//` garde le quotient entier : `3 // 2 → 1`. Précisément, elle arrondit **vers le bas** (*floor*) : `-3 // 2 → -2` (et non `-1`) — c'est $\lfloor -1,5 \rfloor = -2$.

La **puissance** `**` : `2 ** 10 → 1024`. (Pas de `^` comme en maths ou en R : en Python, `^` est un opérateur bit à bit — piège classique en venant de R !)

3.3 Le modulo, et le test pair/impair

`%` donne le **reste** de la division euclidienne. Corey s'y attarde car c'est un test omniprésent :

```
print(2 % 2)    # → 0
print(3 % 2)    # → 1
print(4 % 2)    # → 0
print(5 % 2)    # → 1
```

Le reste de la division par 2 ne peut valoir que 0 ou 1 : **reste 0 = nombre pair, reste 1 = impair**. Plus généralement, `n % k == 0` teste « n est divisible par k » — on s'en servira dans les conditions (chapitre 6) et les compréhensions (chapitre 14). En notation mathématique : `a % b = a mod b`.

3.4 Priorité des opérations

Python respecte la priorité usuelle (puissances, puis `*`/`/`, puis `+`/`-`), et les parenthèses forcent l'ordre :

```
print(3 * 2 + 1)    # → 7    (multiplication d'abord)
print(3 * (2 + 1))  # → 9    (parenthèses d'abord)
```

Conseil pratique : dans une formule un tant soit peu longue, mets les parenthèses *même quand elles sont facultatives* — le lecteur n'a pas à réciter les règles de priorité pour te comprendre.

3.5 Incrémenter : les assignments augmentées

Faire évoluer une variable à partir de sa propre valeur est tellement courant qu'il existe un raccourci :

```
num = 1
num = num + 1    # version explicite
print(num)       # → 2

num = 1
num += 1         # version idiomatique : assignment augmentée
print(num)       # → 2

num *= 10
print(num)       # → 20
```

Le raccourci existe pour tous les opérateurs : `+=`, `-=`, `*=`, `/=`, `//=`, `**=`, `%=`. (Python n'a en revanche **pas** de `num++` à la C/Java.)

3.6 Fonctions intégrées : `abs()` et `round()`

```
print(abs(-3))      # → 3      valeur absolue
print(round(3.75))  # → 4      arrondi à l'entier le plus proche
print(round(3.75, 1)) # → 3.8  2e argument : nombre de décimales
```

Piège — `round()` pratique l'**arrondi bancaire** (*banker's rounding*) : à exacte mi-distance, il arrondit vers le chiffre pair. Donc `round(2.5)` → 2 et `round(3.5)` → 4 ! Ce n'est pas l'arrondi « scolaire », mais il évite un biais statistique systématique vers le haut quand on arrondit en masse — un choix qui parlera au matheux. À savoir absolument avant de manipuler des montants.

3.7 Les comparaisons

Les opérateurs de comparaison confrontent deux valeurs et renvoient un **booléen** — `True` ou `False` (le type `bool`, au cœur du chapitre 6) :

```
num_1 = 3
num_2 = 2

print(num_1 == num_2)  # → False   égalité
print(num_1 != num_2)  # → True    différence
print(num_1 > num_2)    # → True    strictement supérieur
print(num_1 < num_2)    # → False   strictement inférieur
print(num_1 >= num_2)   # → True    supérieur ou égal
print(num_1 <= num_2)   # → False   inférieur ou égal
```

Piège fondamental — `=` assigne, `==` compare. `x = 3` range 3 dans `x` ; `x == 3` demande « x vaut-il 3 ? ». Les confondre est l'erreur n°1 en arrivant de maths, où le signe `=` joue les deux rôles.

Pour aller plus loin — L'imprécision des flottants. Essaie :

```
print(0.1 + 0.2)      # → 0.30000000000000004
print(0.1 + 0.2 == 0.3) # → False
print(round(2.675, 2)) # → 2.67 (et non 2.68 !)
```

Rien d'anormal : 0.1, 0.2 et 2.675 n'ont pas de représentation binaire finie (comme 1/3 en décimal), donc la machine stocke des approximations — et `round(2.675, 2)` arrondit le nombre *réellement stocké*, qui vaut 2.67499999... Conséquences pratiques : ne jamais tester l'égalité stricte entre flottants (utiliser `math.isclose(a, b)` ou comparer `abs(a-b)` à une tolérance), et pour la comptabilité au centime près, connaître le module `decimal` de la bibliothèque standard. Pour le calcul scientifique courant, la double précision (~15-16 chiffres significatifs) suffit très largement.

3.8 Conversions de type : le casting

Cas très concret soulevé par Corey : des nombres qui arrivent **sous forme de texte** (lecture d'un fichier, d'un formulaire web, d'un CSV...) :

```
num_1 = "100"
num_2 = "200"
print(num_1 + num_2)    # → 100200    (!!)
```

Pas d'erreur, mais un résultat piégeux : entre chaînes, **+** **concatène** (chapitre 2). Pour additionner, il faut d'abord **convertir** (*cast*) :

```
num_1 = int(num_1)
num_2 = int(num_2)
print(num_1 + num_2)    # → 300
```

Les fonctions de conversion portent le nom du type cible : `int("100")` → `100`, `float("3.14")` → `3.14`, `str(42)` → `"42"`. Si la conversion est impossible (`int("abc")`), Python lève une `ValueError` — on apprendra à la gérer proprement au chapitre 13.

Exercices

Exercice 3.1 ● — Sans exécuter, prédis le résultat et le *type* de chaque expression, puis vérifie : `7 / 2`, `7 // 2`, `7 % 2`, `7.0 // 2`, `2 ** 3 ** 2` (indice : `**` est associatif à droite), `"7" + "2"`.

Exercice 3.2 ● — Un capital de 1 500 € est placé à 3 % par an (intérêts composés). Calcule sa valeur au bout de 10 ans avec l'opérateur `**`, et affiche-la arrondie à 2 décimales via une f-string. (Formule : $C_n = C_0(1 + r)^n$.)

Exercice 3.3 ●● — Sur une année de 365 jours, on veut savoir combien de semaines complètes elle contient et combien de jours restent. Calcule les deux valeurs avec `//` et `%`, et affiche « 365 jours = 52 semaines et 1 jour(s) ».

Exercice 3.4 ●● — `montant = "1500"` et `taux = "0.03"` arrivent d'un fichier sous forme de chaînes. Écris le code qui calcule les intérêts d'une année (`montant * taux`) — attention aux types — et explique ce qui se passerait sans conversion.

Exercice 3.5 ●●● — Explique précisément pourquoi `round(2.5)` vaut `2` alors que `round(2.51)` vaut `3`, puis pourquoi `0.1 + 0.2 == 0.3` est `False`. Propose deux façons de tester correctement l'égalité `0.1 + 0.2 ≈ 0.3`.

Solutions détaillées

Solution 3.1 —

```
print(7 / 2)    # → 3.5    float — la division « vraie » renvoie toujours un float
print(7 // 2)   # → 3      int   — quotient entier
```

```
print(7 % 2)      # → 1      int    – reste
print(7.0 // 2)   # → 3.0    float  – un opérande float suffit à rendre le résultat float
print(2 ** 3 ** 2) # → 512    int    – associatif à droite : 2 ** (3 ** 2) = 29
print("7" + "2")  # → 72     str    – concaténation de chaînes, pas d'addition !
```

Solution 3.2 —

```
capital = 1500
taux = 0.03
valeur = capital * (1 + taux) ** 10
print(f"Valeur finale : {valeur:.2f} €")  # → Valeur finale : 2015.87 €
```

La priorité fait bien ce qu'on veut (`**` avant `*`), mais les parenthèses autour de `(1 + taux)` sont, elles, indispensables.

Solution 3.3 —

```
jours = 365
semaines = jours // 7      # → 52
reste = jours % 7          # → 1
print(f"{jours} jours = {semaines} semaines et {reste} jour(s)")
# → 365 jours = 52 semaines et 1 jour(s)
```

`//` et `%` forment une paire : `jours == semaines * 7 + reste`. (La fonction `divmod(365, 7)` renvoie les deux d'un coup.)

Solution 3.4 —

```
montant = "1500"
taux = "0.03"
interets = int(montant) * float(taux)
print(f"Intérêts : {interets:.2f} €")  # → Intérêts : 45.00 €
```

Sans conversion, `montant * taux` lèverait `TypeError: can't multiply sequence by non-int of type 'str'` — Python refuse de multiplier deux chaînes. (Piège dans le piège : `"1500" * 3` est, lui, légal et donne `"150015001500"` — répétition de chaîne !) Noter aussi : `int(taux)` échouerait (`ValueError`), car `"0.03"` n'est pas un entier — d'où `float` pour le taux.

Solution 3.5 — `round(2.5)` applique l'arrondi bancaire : à mi-distance exacte entre 2 et 3, il choisit l'entier **pair**, donc 2. `2.51` n'est pas à mi-distance : plus proche de 3, il donne 3. Quant à `0.1 + 0.2`, chacun des deux termes est stocké en binaire par une approximation ; leur somme vaut exactement `0.30000000000000004`, qui n'est pas le flottant le plus proche de 0.3 → l'égalité stricte échoue. Tests corrects :

```
import math
a = 0.1 + 0.2
print(math.isclose(a, 0.3))      # → True    (tolérance relative par défaut)
print(abs(a - 0.3) < 1e-9)      # → True    (tolérance absolue choisie à la main)
```

Fiche récap — Chapitre 3

Syntaxe	Effet
<code>int</code> , <code>float</code>	entier (précision arbitraire) / flottant (IEEE 754) ; <code>type(x)</code>
<code>+</code> <code>-</code> <code>*</code> <code>/</code>	arithmétique ; <code>/</code> renvoie toujours un float (Python 3)
<code>//</code>	division entière (arrondi vers le bas : <code>-3 // 2</code> → <code>-2</code>)
<code>%</code>	modulo (reste) ; <code>n % 2 == 0</code> ⇔ pair ; <code>n % k == 0</code> ⇔ divisible
<code>**</code>	puissance (pas <code>^</code>) ; associatif à droite
<code>x += 1</code> , <code>x *= 2</code> ...	assignations augmentées (pas de <code>x++</code>)
<code>abs(x)</code>	valeur absolue
<code>round(x, n)</code>	arrondi (n décimales) — arrondi bancaire : <code>round(2.5)</code> → 2
<code>==</code> <code>!=</code> <code>></code> <code><</code> <code>>=</code> <code><=</code>	comparaisons → <code>True</code> / <code>False</code> ; <code>=</code> assigne , <code>==</code> compare
<code>int("100")</code> , <code>float("3.14")</code> , <code>str(42)</code>	conversions (casting) ; échec → <code>ValueError</code>
Flottants	<code>0.1 + 0.2 != 0.3</code> → jamais d'égalité stricte ; <code>math.isclose</code> , module <code>decimal</code>

CHAPITRE 4

Listes, tuples et ensembles (sets)

Vidéo source : [Python Tutorial for Beginners 4: Lists, Tuples, and Sets](#) — Corey Schafer

Objectifs : maîtriser la liste (le conteneur à tout faire de Python) et ses méthodes, comprendre la mutabilité et ses conséquences, savoir quand préférer un tuple ou un set.

4.1 Les listes : créer, mesurer, accéder

Une **liste** (*list*) stocke des valeurs **ordonnées** entre crochets. Fil rouge de la vidéo : une liste de cours universitaires.

```
courses = ['history', 'math', 'physics', 'compsci']
print(courses)      # → ['history', 'math', 'physics', 'compsci']
print(len(courses)) # → 4
```

L'accès par index fonctionne exactement comme pour les chaînes (chapitre 2) — **à partir de 0** :

```
print(courses[0])    # → history
print(courses[3])    # → compsci      (dernier : longueur 4 → index 3)
# print(courses[4])  # IndexError: list index out of range
```

Les **indices négatifs** comptent depuis la fin — et `-1` désigne *toujours* le dernier élément :

```
print(courses[-1])   # → compsci
print(courses[-2])   # → physics
```

Corey le souligne : `[-1]` est souvent préférable à `[len(courses) - 1]`, car on n'a pas besoin de connaître la longueur. C'est un idiome Python omniprésent.

Le **slicing** aussi se transpose tel quel (début inclus, fin exclue) :

```
print(courses[0:2])  # → ['history', 'math']
print(courses[:2])   # → ['history', 'math']
print(courses[2:])   # → ['physics', 'compsci']
```

4.2 Ajouter des éléments : `append`, `insert`, `extend`

```
courses = ['history', 'math', 'physics', 'compsci']

courses.append('art')
print(courses)  # → ['history', 'math', 'physics', 'compsci', 'art']
```

`append` ajoute à la fin. Pour insérer à une position précise, `insert(index, valeur)` — rien n'est écrasé, tout se décale :

```
courses = ['history', 'math', 'physics', 'compsci']
courses.insert(0, 'art')
print(courses)    # → ['art', 'history', 'math', 'physics', 'compsci']
```

Piège (le grand classique de cette vidéo) — Que se passe-t-il si on `insert` / `append` une liste ?

```
courses = ['history', 'math', 'physics', 'compsci']
courses_2 = ['art', 'education']
courses.insert(0, courses_2)
print(courses)    # → [['art', 'education'], 'history', 'math', 'physics',
                    'compsci']
print(courses[0])  # → ['art', 'education']
```

La liste entière est insérée **comme un seul élément** : on obtient une liste imbriquée, presque jamais ce qu'on voulait. `append(courses_2)` ferait pareil en fin de liste.

La bonne méthode pour fusionner : `extend`, qui ajoute les éléments *individuellement* :

```
courses = ['history', 'math', 'physics', 'compsci']
courses_2 = ['art', 'education']
courses.extend(courses_2)
print(courses)    # → ['history', 'math', 'physics', 'compsci', 'art', 'education']
```

Règle mnémotechnique : `append` = « ajoute cet objet » ; `extend` = « ajoute le contenu de cet objet ».

4.3 Retirer des éléments : `remove`, `pop`

```
courses = ['history', 'math', 'physics', 'compsci']

courses.remove('math')    # retire par VALEUR
print(courses)            # → ['history', 'physics', 'compsci']

popped = courses.pop()    # retire le DERNIER élément... et le RENVOIE
print(popped)             # → compsci
print(courses)            # → ['history', 'physics']
```

`pop` a cette double casquette — retirer et renvoyer la valeur retirée — qui le rend idéal pour traiter une liste comme une **pile** (*stack* : dernier arrivé, premier sorti). `pop(0)` retire le premier élément (file d'attente), et `remove` sur une valeur absente lève une `ValueError`.

4.4 Réordonner : `reverse`, `sort`, et la fonction `sorted`

```
courses = ['history', 'math', 'physics', 'compsci']
courses.reverse()
print(courses)    # → ['compsci', 'physics', 'math', 'history']

courses.sort()
print(courses)    # → ['compsci', 'history', 'math', 'physics']    (ordre alphabétique)

nums = [1, 5, 4, 3]
nums.sort()
print(nums)       # → [1, 3, 4, 5]

nums.sort(reverse=True)
print(nums)       # → [5, 4, 3, 1]    (décroissant)
```

Note bien : `courses.sort()` ne se réassigne pas — ces méthodes **modifient la liste en place** et renvoient `None`. C'est le contraire des méthodes de chaînes du chapitre 2 ! D'où le piège symétrique :

Piège — `courses = courses.sort()` détruit ta liste : `sort()` renvoie `None`, donc `courses` vaut ensuite `None`. On écrit soit `courses.sort()` (tri en place), soit la ligne suivante.

Quand on veut une version triée **sans toucher à l'original**, on utilise la fonction intégrée `sorted()` :

```
courses = ['history', 'math', 'physics', 'compsci']
sorted_courses = sorted(courses)
print(sorted_courses)    # → ['compsci', 'history', 'math', 'physics']
print(courses)           # → ['history', 'math', 'physics', 'compsci']    (intact)
```

Pour aller plus loin — `sort` et `sorted` acceptent un paramètre `key` : une fonction appliquée à chaque élément pour décider de l'ordre. `sorted(mots, key=len)` trie par longueur ; `sorted(mots, key=str.lower)` trie sans tenir compte de la casse. Très puissant une fois les fonctions vues (chapitre 8).

4.5 Agrégats et recherche : `min`, `max`, `sum`, `index`, `in`

```
nums = [1, 5, 4, 3]
print(min(nums))    # → 1
print(max(nums))    # → 5
print(sum(nums))    # → 13
```

Pour localiser une valeur :

```
courses = ['history', 'math', 'physics', 'compsci']
print(courses.index('compsci'))    # → 3
# print(courses.index('art'))      # ValueError: 'art' is not in list
```

Et pour simplement savoir si une valeur est présente, l'opérateur `in` renvoie un booléen :

```
print('math' in courses)    # → True
print('art' in courses)     # → False
```

Corey annonce la couleur : `in` deviendra crucial avec les conditions (chapitre 6) — « si le cours est dans la liste, alors... ».

4.6 Parcourir une liste : la boucle `for` et `enumerate`

Avant-goût du chapitre 7 — le parcours d'une liste est si fondamental que la vidéo l'introduit ici :

```
for course in courses:
    print(course)
# → history
# → math
# → physics
# → compsci
```

Lecture : « pour chaque `course` dans `courses` : ... ». À chaque tour de boucle, la variable `course` prend la valeur de l'élément suivant. Le nom `course` est arbitraire (`item`, `x`... fonctionnent pareil — mais un nom parlant aide le lecteur), et la ligne indentée sous le `:` constitue le **corps** de la boucle.

Pour obtenir l'*index* et la *valeur* en même temps, la fonction `enumerate` :

```
for index, course in enumerate(courses):
    print(index, course)
# → 0 history
# → 1 math
# → 2 physics
# → 3 compsci

for index, course in enumerate(courses, start=1):
    print(index, course)
# → 1 history
# → 2 math
# → 3 physics
# → 4 compsci
```

`start=1` est parfait pour un affichage numéroté « humain ». Réflexe Python : quand tu te surprends à écrire `for i in range(len(liste))`, c'est presque toujours `enumerate` qu'il te faut.

4.7 join et split : entre liste et chaîne

Transformer une liste de chaînes en une seule chaîne — pour l'afficher, l'écrire dans un fichier CSV... — c'est `join`, une méthode du **séparateur** :

```
course_str = ', '.join(courses)
print(course_str)    # → history, math, physics, compsci

course_str = ' - '.join(courses)
print(course_str)    # → history - math - physics - compsci
```

Piège — Le sens surprend au début : c'est `separateur.join(liste)`, et non `liste.join(separateur)`. Lis-le comme : « ce séparateur *joint* les éléments de la liste ».

L'opération inverse, découper une chaîne en liste, est `split` :

```
new_list = course_str.split(' - ')
print(new_list)    # → ['history', 'math', 'physics', 'compsci']
```

(`split()` sans argument découpe sur les espaces/blancs — pratique pour les mots d'une phrase.)

4.8 La mutabilité — et pourquoi elle peut te piéger

Les listes sont **muables** (*mutable*) : on peut les modifier en place, y compris élément par élément (`courses[0] = 'art'`). Conséquence subtile et capitale, montrée par Corey :

```
list_1 = ['history', 'math', 'physics', 'compsci']
list_2 = list_1    # PAS une copie : un second nom pour LE MÊME objet

list_1[0] = 'art'

print(list_1)    # → ['art', 'math', 'physics', 'compsci']
print(list_2)    # → ['art', 'math', 'physics', 'compsci']    (modifiée « aussi » !)
```

`list_2 = list_1` ne copie rien : les deux variables sont deux étiquettes collées sur **le même objet** en mémoire. Modifier l'objet via l'une se voit via l'autre. (Le chapitre 6 précisera cette histoire d'identité avec `is` et `id()` ; pour copier réellement : `list_2 = list_1.copy()` ou `list_1[:]`.)

4.9 Les tuples : des listes immuables

Un **tuple** ressemble à une liste, mais entre **parenthèses** — et il est **immuable** (*immutable*) : impossible de le modifier après création.

```
tuple_1 = ('history', 'math', 'physics', 'compsci')
print(tuple_1)      # → ('history', 'math', 'physics', 'compsci')
print(tuple_1[0])   # → history

# tuple_1[0] = 'art'   #   TypeError: 'tuple' object does not support item assignment
```

Tout ce qui *lit* (indexation, slicing, `len`, `min`/`max`/`sum`, boucles, `in`, `index`, `count`) fonctionne comme pour une liste ; tout ce qui *modifie* (`append`, `remove`, `sort`...) n'existe simplement pas — un `dir(tuple)` le confirme.

Quand choisir quoi ? La règle de Corey : liste si tu as besoin de modifier ; tuple si tu veux juste parcourir/accéder — avec en prime la garantie que personne (y compris le toi de la section 4.8) ne le modifiera par accident. En pratique, le tuple sert aussi pour les « enregistrements » à structure fixe : un point `(x, y)`, une paire `(date, prix)` — et on le retrouvera partout en déballage (`for index, course in enumerate(...)`) : c'est un tuple qu'on déballe !).

4.10 Les sets : unicité et opérations ensemblistes

Un **set** (ensemble) se note entre **accolades** ; il est **non ordonné** et **sans doublons** :

```
cs_courses = {'history', 'math', 'physics', 'compsci', 'math'}
print(cs_courses)  # → {'physics', 'compsci', 'history', 'math'}  (ordre imprévisible !)
```

Deux observations : le doublon `'math'` a été éliminé automatiquement, et l'ordre d'affichage n'a rien à voir avec l'ordre d'écriture — il peut même changer d'une exécution à l'autre. Un set ne s'indexe donc pas (`cs_courses[0]` n'existe pas).

Ses deux points forts :

1. Le test d'appartenance ultra-rapide. `'math' in cs_courses` fait le même travail que sur une liste, mais beaucoup plus efficacement — le set est fait pour ça.

Pour aller plus loin (le pourquoi mathématique) — Un set est une **table de hachage** : l'appartenance se teste en temps constant $O(1)$ en moyenne, contre un parcours linéaire $O(n)$ pour une liste. Sur une liste d'un million d'éléments, la différence se chiffre en milliers de fois. C'est aussi pourquoi un set n'a pas d'ordre : les éléments sont rangés selon leur *hash*, pas selon leur insertion.

2. Les opérations ensemblistes — exactement $\cap$, $\setminus$, $\cup$:

```
cs_courses = {'history', 'math', 'physics', 'compsci'}
art_courses = {'history', 'math', 'art', 'design'}

print(cs_courses.intersection(art_courses))  # → {'history', 'math'}      (∩ communs)
print(cs_courses.difference(art_courses))     # → {'physics', 'compsci'}  (∖ dans l'un
pas l'autre)
print(cs_courses.union(art_courses))          # → les 6 cours, sans doublons (∪)
```

Usage typique n°1 : **dédoubler une liste** en une ligne — `list(set(ma_liste))` (en perdant l'ordre).

4.11 Créer des conteneurs vides — et le piège du set

```
empty_list = []          # ou list()
empty_tuple = ()         # ou tuple()
empty_set = set()        # obligatoirement set()
```

Piège — `empty_set = {}` ne crée **pas** un set vide mais... un **dictionnaire** vide (chapitre 5) ! Les accolades sont partagées entre les deux types, et le dict a la priorité historique. Pour un set vide, c'est `set()`, sans exception.

Exercices

Exercice 4.1 ● — Pars de `portefeuille = ['AAPL', 'MSFT', 'GOOG']`. Enchaîne : ajoute `'NVDA'` à la fin ; insère `'AMZN'` en 2 position ; retire `'GOOG'` ; affiche le dernier élément avec un index négatif ; affiche la liste triée alphabétiquement **sans** modifier l'original, puis l'original pour prouver qu'il est intact.

Exercice 4.2 ● — Avec `rendements = [0.02, -0.01, 0.035, 0.007, -0.015]` : affiche le meilleur, le pire, et la somme des rendements, puis le rendement moyen arrondi à 4 décimales (pas de fonction `mean` : `sum` et `len` suffisent).

Exercice 4.3 ●● — Prédis la sortie complète avant d'exécuter :

```
a = [3, 1, 2]
b = a
c = a.copy()
b.sort()
print(a, b, c)
```

Explique chaque affichage en une phrase.

Exercice 4.4 ●● — On te donne `tickers = 'AAPL;MSFT;GOOG;AAPL;NVDA;MSFT'`. Produis : 1) la liste des tickers (dans l'ordre) ; 2) le nombre de tickers *distincts* ; 3) une chaîne des tickers distincts triés, séparés par `' , '`. (Enchaîne `split`, `set`, `sorted`, `join`.)

Exercice 4.5 ●●● — `clients_2025` et `clients_2026` sont deux listes de noms avec doublons possibles. Écris les expressions donnant : les clients fidèles (présents les deux années), les clients perdus (2025 mais pas 2026), les nouveaux (2026 seulement), et le nombre total de clients distincts sur les deux années. Teste avec `clients_2025 = ['Ana', 'Bob', 'Chloe', 'Ana']` et `clients_2026 = ['Bob', 'Dan', 'Chloe', 'Dan']`.

Solutions détaillées

Solution 4.1 —

```

portefeuille = ['AAPL', 'MSFT', 'GOOG']
portefeuille.append('NVDA')          # ['AAPL', 'MSFT', 'GOOG', 'NVDA']
portefeuille.insert(1, 'AMZN')       # ['AAPL', 'AMZN', 'MSFT', 'GOOG', 'NVDA']
portefeuille.remove('GOOG')          # ['AAPL', 'AMZN', 'MSFT', 'NVDA']
print(portefeuille[-1])               # → NVDA
print(sorted(portefeuille))           # → ['AAPL', 'AMZN', 'MSFT', 'NVDA']
print(portefeuille)                  # → ['AAPL', 'AMZN', 'MSFT', 'NVDA'] (ordre
                                     d'origine conservé)

```

(Ici l'original était déjà trié — l'important est le choix de `sorted()` plutôt que `.sort()`.)

Solution 4.2 —

```

rendements = [0.02, -0.01, 0.035, 0.007, -0.015]
print(max(rendements))                # → 0.035
print(min(rendements))                # → -0.015
print(sum(rendements))                # → 0.037000000000000005
moyenne = sum(rendements) / len(rendements)
print(round(moyenne, 4))               # → 0.0074

```

Tu reconnais le `...000005` ? C'est l'imprécision des flottants du chapitre 3 qui refait surface — la somme *mathématique* vaut 0.037, la somme *flottante* en est infinitésimalement éloignée. D'où le `round` (ou un format `:.4f`) systématique à l'affichage.

Solution 4.3 — Sortie: `[1, 2, 3]` `[1, 2, 3]` `[3, 1, 2]`. `b = a` fait de `b` un second nom du *même* objet : trier via `b` trie donc aussi « `a` » (c'est le même). `c = a.copy()` avait créé un objet indépendant *avant* le tri : il conserve l'ordre initial.

Solution 4.4 —

```

tickers = 'AAPL;MSFT;GOOG;AAPL;NVDA;MSFT'
liste = tickers.split(';')
print(liste)                          # → ['AAPL', 'MSFT', 'GOOG', 'AAPL', 'NVDA', 'MSFT']
distincts = set(liste)
print(len(distincts))                 # → 4
print(', '.join(sorted(distincts)))    # → AAPL, GOOG, MSFT, NVDA

```

`sorted()` appliqué à un set renvoie une *liste* triée — indispensable ici, car `join` a besoin d'un ordre et le set n'en a pas.

Solution 4.5 —

```

clients_2025 = ['Ana', 'Bob', 'Chloe', 'Ana']
clients_2026 = ['Bob', 'Dan', 'Chloe', 'Dan']

s25 = set(clients_2025)
s26 = set(clients_2026)

print(s25.intersection(s26))          # → {'Bob', 'Chloe'}      fidèles
print(s25.difference(s26))             # → {'Ana'}             perdus

```



```
print(s26.difference(s25))    # → {'Dan'}           nouveaux
print(len(s25.union(s26)))   # → 4                distincts au total
```

Les ensembles absorbent d'abord les doublons, puis les quatre questions deviennent quatre opérations ensemblistes directes — exactement le cas d'usage vanté dans la vidéo. (L'affichage des sets peut varier en ordre : normal.)

Fiche récap — Chapitre 4

Syntaxe	Effet
<code>[a, b, c]</code>	liste : ordonnée, muable ; <code>len</code> , <code>l[i]</code> , <code>l[-1]</code> , <code>l[a:b]</code>
<code>l.append(x)</code> / <code>l.insert(i, x)</code>	ajoute à la fin / à l'index <i>i</i> (une liste = 1 seul élément imbriqué)
<code>l.extend(autre)</code>	ajoute les éléments un à un (fusion)
<code>l.remove(x)</code> / <code>l.pop()</code>	retire par valeur / retire et renvoie le dernier (<code>pop(0)</code> : le 1)
<code>l.reverse()</code> / <code>l.sort()</code>	en place , renvoient <code>None</code> ; <code>sort(reverse=True)</code> décroissant
<code>sorted(l)</code>	renvoie une copie triée, original intact
<code>min</code> / <code>max</code> / <code>sum</code>	agrégats
<code>l.index(x)</code>	position (<code>ValueError</code> si absent) ; <code>x in l</code> → booléen
<code>for x in l:</code> / <code>enumerate(l, start=1)</code>	parcours / parcours numéroté
<code>'</code> , <code>'.join(l)</code> ↔ <code>s.split('')</code>	liste → chaîne ↔ chaîne → liste (join : méthode du séparateur)
<code>b = a</code> vs <code>b = a.copy()</code>	alias du même objet vs vraie copie (mutabilité !)
<code>(a, b, c)</code>	tuple : immuable ; lecture OK, modification → <code>TypeError</code>
<code>{a, b, c}</code>	set : sans doublons, non ordonné, <code>in</code> ultra-rapide ($O(1)$)
<code>s1.intersection(s2)</code> / <code>.difference</code> / <code>.union</code>	$n \cap \setminus \cup$
Vides	<code>[]</code> , <code>()</code> , mais <code>set()</code> — car <code>{}</code> = dict vide

CHAPITRE 5

Dictionnaires : travailler par clé-valeur

Vidéo source : [Python Tutorial for Beginners 5: Dictionaries - Working with Key-Value Pairs](#) — Corey Schafer

Objectifs : créer et manipuler des dictionnaires, accéder aux valeurs sans se faire surprendre par les clés absentes, ajouter/modifier/supprimer des entrées, et parcourir clés et valeurs.

5.1 L'idée : associer des clés à des valeurs

Le **dictionnaire** (`dict`) stocke des paires **clé** → **valeur**. Dans d'autres langages, on parle de *hashmap* ou de *tableau associatif*. L'analogie de Corey est la bonne : dans un dictionnaire papier, le **mot** est la clé, la **définition** est la valeur — on ne cherche pas « la 3 412 définition », on cherche « le mot *ergodique* ».

Là où la liste indexe par position (0, 1, 2, ...), le dictionnaire indexe par **clé signifiante** :

```
student = {'name': 'John', 'age': 25, 'courses': ['math', 'compsci']}
print(student)
# → {'name': 'John', 'age': 25, 'courses': ['math', 'compsci']}
```

Syntaxe : accolades, paires **clé**: **valeur** séparées par des virgules. Les **valeurs** peuvent être de n'importe quel type — ici une chaîne, un entier, et même une liste. Les **clés** doivent être d'un type immuable : le plus souvent des chaînes, parfois des entiers (les tuples marchent aussi ; les listes, non — elles sont muables).

Depuis la vidéo — Depuis Python 3.7, les dictionnaires **conservent l'ordre d'insertion** des clés (garanti par le langage). À l'époque de la vidéo, l'ordre était officiellement imprévisible ; c'est pour ça que certains affichages de la vidéo sortent mélangés, alors que chez toi ils sortiront dans l'ordre d'écriture.

5.2 Accéder aux valeurs — et le cas de la clé absente

```
print(student['name'])      # → John
print(student['courses'])  # → ['math', 'compsci']
```

Mais si la clé n'existe pas :

```
# print(student['phone'])  #   KeyError: 'phone'
```

Le programme s'arrête net. Pour un accès « prudent », la méthode `get` :

```
print(student.get('name'))           # → John
print(student.get('phone'))          # → None           (pas d'erreur)
print(student.get('phone', 'not found')) # → not found   (valeur par défaut)
```

`None` est l'objet Python qui représente « rien / pas de valeur » — on le reverra au chapitre 6 (il est « falsy ») et partout ailleurs.

Quand utiliser quoi ? `student['phone']` quand l'absence de la clé est un *bug* (mieux vaut une erreur immédiate et claire) ; `student.get('phone', défaut)` quand l'absence est un *cas normal* à gérer. Les deux réflexes sont bons — l'important est de choisir consciemment.

5.3 Ajouter et modifier

L'assignation par crochets fait les deux : elle **crée** la clé si elle n'existe pas, l'**écrase** sinon :

```
student['phone'] = '555-5555'      # ajout
student['name'] = 'Jane'           # modification
print(student)
# → {'name': 'Jane', 'age': 25, 'courses': ['math', 'compsci'], 'phone': '555-5555'}
```

Pour modifier/ajouter **plusieurs** entrées d'un coup, `update` prend un dictionnaire :

```
student.update({'name': 'Jane', 'age': 26, 'phone': '555-5555'})
print(student['age']) # → 26
```

`update` écrase les clés existantes et crée les autres — pratique notamment pour fusionner des informations venant d'une autre source.

5.4 Supprimer : `del` et `pop`

```
del student['age'] # supprime la paire
```

Ou bien, si on veut **récupérer la valeur** au passage :

```
student = {'name': 'John', 'age': 25, 'courses': ['math', 'compsci']}
age = student.pop('age')
print(age) # → 25
print(student) # → {'name': 'John', 'courses': ['math', 'compsci']}
```

Même philosophie que le `pop` des listes (chapitre 4) : retirer et renvoyer. `del` comme `pop` lèvent une `KeyError` si la clé n'existe pas (`pop` accepte un défaut : `student.pop('age', None)`).

5.5 Inspecter : `len`, `keys`, `values`, `items`

```
student = {'name': 'John', 'age': 25, 'courses': ['math', 'compsci']}

print(len(student))          # → 3                               (nombre de CLÉS)
print(student.keys())        # → dict_keys(['name', 'age', 'courses'])
print(student.values())      # → dict_values(['John', 25, ['math', 'compsci']])
print(student.items())       # → dict_items([('name', 'John'), ('age', 25), ('courses',
['math', 'compsci'])])
```

`items()` renvoie les paires sous forme de **tuples** (clé, valeur) — c'est la passerelle vers les boucles.

5.6 Parcourir un dictionnaire

Piège — La boucle « nue » sur un dictionnaire ne parcourt que les **clés** :

```
for key in student:
    print(key)
# → name
# → age
# → courses
```

Pour avoir clé **et** valeur, on boucle sur `items()` en déballant le tuple dans deux variables :

```
for key, value in student.items():
    print(key, value)
# → name John
# → age 25
# → courses ['math', 'compsci']
```

(Ce « déballage » `key, value` est exactement le mécanisme déjà vu avec `enumerate` au chapitre 4.) Et si seul l'un des deux t'intéresse : `for v in student.values(): ...`.

Pour aller plus loin (le lien avec les sets) — Sous le capot, un dict est la même structure qu'un set : une table de hachage sur les clés. D'où : recherche d'une clé en $O(1)$ en moyenne (aussi rapide sur un million d'entrées que sur dix), unicité des clés (réassigner = écraser), et exigence d'immuabilité des clés (il faut pouvoir les hacher). Un dict, c'est un set de clés qui transportent chacune une valeur.

5.7 Un vrai cas d'usage : compter

Pour ancrer le réflexe « dictionnaire », voici l'exemple canonique (il annonce les boucles du chapitre 7) — compter les occurrences :

```
notes = ['A', 'B', 'A', 'C', 'B', 'A']
compteur = {}

for note in notes:
    compteur[note] = compteur.get(note, 0) + 1

print(compteur)    # → {'A': 3, 'B': 2, 'C': 1}
```

La ligne clé combine tout le chapitre : `get(note, 0)` lit le compte actuel *ou* 0 si la note apparaît pour la première fois, puis l'assignation crée/écrase. Trois lignes, et ça marche pour n'importe quelles données.

Exercices

Exercice 5.1 ● — Crée un dictionnaire `actif` décrivant une action : `ticker` 'AAPL', `prix` 189.5, `devise` 'USD', et `secteurs` (liste de deux chaînes). Affiche le prix, ajoute une clé `pe_ratio` à 29.1, corrige le prix à 191.2, puis affiche le nombre d'entrées du dictionnaire.

Exercice 5.2 ● — Sur le dictionnaire de 5.1 : affiche `actif['dividende']` d'abord avec une méthode qui renvoie 'n/a' si la clé manque, puis retire `pe_ratio` en récupérant sa valeur dans une variable.

Exercice 5.3 ●● — Soit `taux = {'EUR': 1.0, 'USD': 0.92, 'GBP': 1.17}` (contre-valeur d'1 unité en euros). Affiche une ligne par devise au format `1 USD = 0.92 EUR` en bouclant sur `items()`. Puis ajoute `'JPY': 0.0061` et affiche la liste des devises connues, triée.

Exercice 5.4 ●●● — Prédise la sortie exacte avant d'exécuter :

```
config = {'mode': 'test', 'seuil': 10}
sauvegarde = config
config.update({'mode': 'prod'})
del config['seuil']
print(sauvegarde)
```

Que faudrait-il changer pour que `sauvegarde` reste `{'mode': 'test', 'seuil': 10}` ?

Exercice 5.5 ●●●● —

`transactions = [('AAPL', 3), ('MSFT', 1), ('AAPL', 2), ('NVDA', 5), ('MSFT', 4)]` : liste de tuples (`ticker`, `quantité`). Construis avec une boucle le dictionnaire `positions` qui agrège les quantités par ticker (attendu : `{'AAPL': 5, 'MSFT': 5, 'NVDA': 5}`), puis affiche chaque position au format `AAPL : 5 titres`.

Solutions détaillées

Solution 5.1 —

```

actif = {
    'ticker': 'AAPL',
    'prix': 189.5,
    'devise': 'USD',
    'secteurs': ['technologie', 'grand public'],
}
print(actif['prix'])      # → 189.5
actif['pe_ratio'] = 29.1
actif['prix'] = 191.2
print(len(actif))        # → 5

```

(Écrire le dictionnaire sur plusieurs lignes, une paire par ligne, est le style standard dès qu'il grossit — la virgule finale est légale et même recommandée.)

Solution 5.2 —

```

print(actif.get('dividende', 'n/a'))  # → n/a
pe = actif.pop('pe_ratio')
print(pe)                             # → 29.1
print('pe_ratio' in actif)            # → False

```

Bonus glissé ici : `in` teste l'appartenance **aux clés** d'un dict — même opérateur qu'au chapitre 4.

Solution 5.3 —

```

taux = {'EUR': 1.0, 'USD': 0.92, 'GBP': 1.17}
for devise, valeur in taux.items():
    print(f"1 {devise} = {valeur} EUR")
# → 1 EUR = 1.0 EUR
# → 1 USD = 0.92 EUR
# → 1 GBP = 1.17 EUR

taux['JPY'] = 0.0061
print(sorted(taux.keys()))  # → ['EUR', 'GBP', 'JPY', 'USD']

```

(`sorted(taux)` seul suffirait : itérer un dict, c'est itérer ses clés.)

Solution 5.4 — Sortie : `{'mode': 'prod'}`. Comme pour les listes (chapitre 4, section 4.8),

`sauvegarde = config` ne copie rien : deux noms, un seul objet — l'`update` et le `del` se « voient » donc via `sauvegarde`. Pour une vraie photographie : `sauvegarde = config.copy()` (copie superficielle, suffisante ici).

Solution 5.5 —

```

transactions = [('AAPL', 3), ('MSFT', 1), ('AAPL', 2), ('NVDA', 5), ('MSFT', 4)]

positions = {}
for ticker, quantite in transactions:
    positions[ticker] = positions.get(ticker, 0) + quantite

print(positions)  # → {'AAPL': 5, 'MSFT': 5, 'NVDA': 5}

```

```
for ticker, quantite in positions.items():
    print(f"{ticker} : {quantite} titres")
# → AAPL : 5 titres
# → MSFT : 5 titres
# → NVDA : 5 titres
```

Le motif `d[k] = d.get(k, 0) + x` (section 5.7) est à connaître par cœur : c'est l'agrégation par clé, omniprésente en traitement de données.

Fiche récap — Chapitre 5

Syntaxe	Effet
<code>{'k': v, ...}</code>	dictionnaire clé → valeur ; clés immuables (str, int, tuple), valeurs libres
<code>d['k']</code>	lit la valeur — <code>KeyError</code> si absente
<code>d.get('k') / d.get('k', défaut)</code>	lit sans erreur → <code>None</code> / défaut si absente
<code>d['k'] = v</code>	crée ou écrase
<code>d.update({...})</code>	plusieurs ajouts/modifications d'un coup
<code>del d['k'] / v = d.pop('k')</code>	supprime / supprime et renvoie
<code>len(d)</code>	nombre de clés
<code>d.keys() / d.values() / d.items()</code>	vues des clés / valeurs / paires (tuples)
<code>for k in d:</code>	boucle sur les clés seulement
<code>for k, v in d.items():</code>	boucle clé + valeur (déballage de tuple)
<code>'k' in d</code>	test d'appartenance aux clés, $O(1)$
<code>d[k] = d.get(k, 0) + x</code>	motif d'agrégation/comptage par clé
Depuis 3.7	l'ordre d'insertion des clés est préservé

CHAPITRE 6

Conditions et booléens : if, elif, else

Vidéo source : [Python Tutorial for Beginners 6: Conditionals and Booleans - If, Else, and Elif Statements](#) — Corey Schafer

Objectifs : faire prendre des décisions à un programme avec `if` / `elif` / `else`, combiner des conditions avec `and` / `or` / `not`, distinguer égalité (`==`) et identité (`is`), et connaître la liste exacte des valeurs « fausses ».

6.1 L'instruction `if`

Jusqu'ici, nos programmes exécutaient toutes leurs lignes, dans l'ordre, à chaque fois. Les **conditionnelles** introduisent les bifurcations : exécuter un bloc *seulement si* une condition est vraie. Une condition est une expression qui s'évalue en booléen — `True` ou `False`.

Forme minimale (avec un booléen codé en dur, pour voir la mécanique) :

```
if True:
    print('La condition est vraie')
# → La condition est vraie
```

Avec `if False:`, le bloc ne s'exécute pas — le programme continue simplement après.

Anatomie : le mot-clé `if`, une condition, un **deux-points**, puis un bloc **indenté** (4 espaces, standard). L'indentation n'est pas cosmétique : c'est *elle* qui définit ce qui appartient au bloc — Python n'a ni accolades ni `end`.

En pratique, la condition est une comparaison :

```
language = 'python'

if language == 'python':
    print('le langage est Python')
# → le langage est Python
```

Piège (rappel du chapitre 3, car il frappe surtout ici) — `==` compare, `=` assigne. Écrire

`if language = 'python':` est une erreur de syntaxe — Python te protège, mais le message peut dérouter.

6.2 `else` et `elif`

`else` fournit le chemin « sinon » ; `elif` (contraction d'*else if*) enchaîne d'autres tests :

```
language = 'java'

if language == 'python':
    print('le langage est Python')
```



```

elif language == 'java':
    print('le langage est Java')
elif language == 'javascript':
    print('le langage est JavaScript')
else:
    print('pas de correspondance')
# → le langage est Java

```

Fonctionnement : Python évalue les conditions **de haut en bas**, exécute le **premier** bloc dont la condition est vraie, et saute tout le reste. Le `else` final (optionnel) attrape tous les cas restants. On peut chaîner autant d'`elif` qu'on veut.

Piège d'indentation — `elif` et `else` se placent au **même niveau** que le `if` (revenus « à la ligne de base », dit Corey), pas à l'intérieur de son bloc. Un `else` indenté d'un cran de trop change complètement le sens du programme — ou provoque une `SyntaxError`.

Au passage, Corey répond à une question fréquente des gens venant d'autres langages : Python n'a pas de `switch / case`, parce que la chaîne `if / elif / else` est jugée suffisamment claire pour ce rôle.

Depuis la vidéo — Python 3.10 a introduit `match / case` (*structural pattern matching*), qui ressemble à un `switch` surpuissant. C'est un outil de niveau intermédiaire ; `if / elif / else` reste la base et couvre l'immense majorité des besoins.

6.3 Combiner des conditions : `and`, `or`, `not`

Trois opérateurs logiques, qui s'écrivent en toutes lettres (pas de `&&` / `||`) :

```

user = 'admin'
logged_in = True

if user == 'admin' and logged_in:
    print('page admin')
else:
    print('mauvais identifiants')
# → page admin

```

`and` exige que **les deux** conditions soient vraies : avec `logged_in = False`, on passerait dans le `else`. `or` se contente d'**au moins une** :

```

user = 'admin'
logged_in = False

if user == 'admin' or logged_in:

```

```
print('page admin')
# → page admin      (la première condition suffit)
```

`not` inverse un booléen :

```
logged_in = False

if not logged_in:
    print('Veuillez vous connecter')
else:
    print('Bienvenue')
# → Veuillez vous connecter
```

Lecture naturelle : « si *non* connecté... ». Corey le reconnaît, `not` déroute un peu au début — pense-le comme la négation logique $\neg$: `not False` vaut `True`.

En notation logique : `and` = $\wedge$, `or` = $\vee$ (ou **inclusif** : les deux vraies $\rightarrow$ vrai aussi), `not` = $\neg$. Les parenthèses groupent les conditions complexes : `if (a and b) or c:` .

Pour aller plus loin — Python autorise le **chaînage des comparaisons**, comme en maths : `if 0 < x < 10:` teste $x \in]0, 10[$ — strictement équivalent à `x > 0 and x < 10` , mais plus lisible. Rare dans les autres langages, à adopter sans hésiter. (On l'a déjà croisé dans `1 <= month <= 12` au chapitre 8 des fonctions — pardon, on va le croiser !)

6.4 Égalité vs identité : `==` et `is`

Deux questions différentes qu'on confond facilement :

- `a == b` : « a et b ont-ils la **même valeur** ? »
- `a is b` : « a et b sont-ils le **même objet** en mémoire ? »

```
a = [1, 2, 3]
b = [1, 2, 3]

print(a == b)    # → True      mêmes valeurs
print(a is b)    # → False     deux objets distincts !
```

Deux listes construites séparément sont égales mais pas identiques. La fonction `id()` — l'« adresse » de l'objet — le montre : `id(a)` et `id(b)` diffèrent, et `a is b` équivaut exactement à `id(a) == id(b)` . En revanche :

```
b = a            # b devient un alias du même objet (souvenir du chapitre 4 !)
print(a is b)    # → True
```

C'est la formalisation de ce qu'on a vu avec les listes partagées (section 4.8) : `b = a` crée une identité, pas une copie.

Usage pratique de `is` : essentiellement les tests contre `None` — l’idiome consacré est `if x is None:` / `if x is not None:` (il n'existe qu'un seul objet `None` dans tout le programme).

6.5 Truthiness : ce que Python considère comme faux

Une condition n'a pas besoin d'être un booléen : **tout objet** peut s'évaluer en vrai/faux. La vidéo donne la liste exacte des valeurs **fausses** (*falsy*) :

- `False`
- `None`
- zéro sous tout type numérique : `0`, `0.0`
- toute séquence vide : `''`, `[]`, `()`
- tout mapping vide : `{}` (dict vide — et set vide aussi)

Tout le reste est vrai (*truthy*) : `'test'`, `[0]`, `-1`, `3.14` ... (Certaines classes personnalisées peuvent définir leur propre falsité — chapitre 23.)

```
condition = []

if condition:
    print('évalué à vrai')
else:
    print('évalué à faux')
# → évalué à faux
```

L'intérêt pratique, souligné par Corey : tester « non vide » sans appeler la moindre méthode. Pour vérifier qu'une liste de résultats contient quelque chose :

```
resultats = ['x', 'y']

if resultats:
    # idiomatique — plutôt que len(resultats) > 0
    print(len(resultats), 'résultat(s)')
else:
    print('aucun résultat')
# → 2 résultat(s)
```

Piège — La truthiness ne distingue pas *pourquoi* c'est faux : `0`, `''` et `None` tombent tous dans le `else`. Si tu dois distinguer « valeur absente » (`None`) de « valeur nulle » (`0` — parfaitement légitime pour un prix, un solde, un rendement !), teste explicitement `if x is None:` et non `if not x:`.

Exercices

Exercice 6.1 ● — Écris un programme avec une variable `note` (sur 20) qui affiche : « Très bien » si `note ≥ 16`, « Bien » si `note ≥ 14`, « Assez bien » si `note ≥ 12`, « Admis » si `note ≥ 10`, « Ajourné » sinon. Utilise le chaînage `elif` et teste avec 17, 12 et 8. Pourquoi l'ordre des tests est-il important ?

Exercice 6.2 ● — Une variable `annee` contient un millésime. Affiche si l'année est bissextile : divisible par 4 ET (non divisible par 100 OU divisible par 400). Teste avec 2024 (oui), 1900 (non), 2000 (oui). (Tu écris ici la formule qu'on encapsulera en fonction au chapitre 8.)

Exercice 6.3 ●● — Prédis la sortie de chaque `print`, puis vérifie :

```
a = [1, 2]
b = [1, 2]
c = b
print(a == b, a is b)
print(b is c)
print(a != b or a is b)
print(not [])
print(bool(''), bool('faux'), bool(0.0), bool(-1))
```

Exercice 6.4 ●● — Un ordre de bourse est valide si : `quantite > 0`, ET `ticker` non vide, ET (`type_ordre` vaut `'marche'` OU (`type_ordre` vaut `'limite'` ET `prix_limite` n'est pas `None`)). Écris le test `if` correspondant, qui affiche « ordre valide » ou « ordre rejeté ». Vérifie avec : (`'AAPL'`, 10, `'marche'`, `None`) → valide ; (`'AAPL'`, 10, `'limite'`, `None`) → rejeté ; (`''`, 10, `'marche'`, `None`) → rejeté.

Exercice 6.5 ●●● — On lit `solde = 0` (compte à zéro) et `solde_inconnu = None` (donnée manquante). Montre que `if not solde:` traite les deux cas de la même façon, puis écris le test qui distingue proprement trois cas : « donnée manquante », « compte vide », « compte approvisionné ». Quel opérateur est indispensable et pourquoi ?

Solutions détaillées

Solution 6.1 —

```
note = 12

if note >= 16:
    print('Très bien')
elif note >= 14:
    print('Bien')
elif note >= 12:
    print('Assez bien')
elif note >= 10:
    print('Admis')
else:
```

```
print('Ajourné')
# → Assez bien
```

L'ordre décroissant est essentiel : Python s'arrête à la **première** condition vraie. En commençant par `note >= 10`, un 17 afficherait « Admis » — techniquement vrai, pédagogiquement vexant.

Solution 6.2 —

```
annee = 2000

if annee % 4 == 0 and (annee % 100 != 0 or annee % 400 == 0):
    print(f'{annee} est bissextile')
else:
    print(f'{annee} n'est pas bissextile')
# → 2000 est bissextile
```

Attention à la priorité : `and` passe avant `or`, donc sans parenthèses la condition se lirait (divisible par 4 et non séculaire) ou divisible par 400. Ici, les deux formules coïncident — mais uniquement parce que « divisible par 400 » implique « divisible par 4 » (exercice de logique : le vérifier !). Ne compte jamais sur ce genre de coïncidence : parenthèse ce que tu penses. 2024 → bissextile ; 1900 → non (séculaire non divisible par 400) ; 2000 → oui.

Solution 6.3 —

```
print(a == b, a is b)      # → True False    mêmes valeurs, objets distincts
print(b is c)              # → True          c = b : alias du même objet
print(a != b or a is b)   # → False          False or False
print(not [])              # → True          liste vide falsy, not l'inverse
print(bool(''), bool('faux'), bool(0.0), bool(-1))
# → False True False True
```

Le dernier : `'faux'` est une chaîne **non vide** donc truthy (son contenu importe peu !), et `-1` est truthy — seul le zéro numérique est falsy.

Solution 6.4 —

```
ticker = 'AAPL'
quantite = 10
type_ordre = 'limite'
prix_limite = None

if quantite > 0 and ticker and (type_ordre == 'marche' or
                                (type_ordre == 'limite' and prix_limite is not None)):
    print('ordre valide')
else:
    print('ordre rejeté')
# → ordre rejeté
```

Trois idiomes du chapitre réunis : `ticker` tout seul pour « non vide » (truthiness), `is not None` pour la donnée manquante, et les parenthèses pour imposer la structure logique. Les deux autres jeux de valeurs donnent bien « valide » puis « rejeté ».

Solution 6.5 —

```
solde = 0
solde_inconnu = None

print(not solde)          # → True
print(not solde_inconnu)  # → True ← indistinguables par la truthiness !

valeur = solde  # essayer aussi avec solde_inconnu et 1250.0

if valeur is None:
    print('donnée manquante')
elif valeur == 0:
    print('compte vide')
else:
    print('compte approvisionné')
# → compte vide
```

L'opérateur indispensable est `is` (test d'identité avec `None`) : `0` et `None` sont tous deux falsy, mais seul `None` est `None`. Tester `is None` en premier permet ensuite de comparer numériquement sans risque.

Fiche récap — Chapitre 6

Syntaxe	Effet
<code>if cond:</code> + bloc indenté	exécute le bloc si <code>cond</code> est vraie
<code>elif cond:</code> / <code>else:</code>	tests en cascade (premier vrai gagne) / cas restant — au même niveau que <code>if</code>
<code>==</code> <code>!=</code> <code>></code> <code><</code> <code>>=</code> <code><=</code>	comparaisons → booléens
<code>and</code> / <code>or</code> / <code>not</code>	$\wedge$ / $\vee$ (inclusif) / $\neg$ — en toutes lettres
<code>0 < x < 10</code>	chaînage des comparaisons, comme en maths
<code>a is b</code>	identité (même objet) $\Leftrightarrow \text{id}(a) == \text{id}(b)$; $\neq$ égalité <code>==</code>
<code>x is None</code> / <code>x is not None</code>	l'idiome pour tester « pas de valeur »
Falsy	<code>False</code> , <code>None</code> , <code>0</code> , <code>0.0</code> , <code>''</code> , <code>[]</code> , <code>()</code> , <code>{}</code> — tout le reste est truthy
<code>if ma_liste:</code>	test « non vide » idiomatique
Pas de switch	<code>if</code> / <code>elif</code> / <code>else</code> (et <code>match</code> / <code>case</code> depuis 3.10)

CHAPITRE 7

Boucles et itérations : for et while

Vidéo source : [Python Tutorial for Beginners 7: Loops and Iterations - For/While Loops](#) — Corey Schafer

Objectifs : répéter des traitements avec `for` et `while`, contrôler le déroulement avec `break` et `continue`, générer des séquences de nombres avec `range`, et savoir créer (et arrêter !) une boucle infinie.

7.1 La boucle `for`

On l'a entrevue au chapitre 4 ; formalisons. `for` itère sur les éléments d'un itérable (liste, chaîne, tuple, dict...) :

```
nums = [1, 2, 3, 4, 5]

for num in nums:
    print(num)
# → 1
# → 2
# → 3
# → 4
# → 5
```

À chaque tour, la variable de boucle `num` reçoit l'élément suivant, et le corps indenté s'exécute. Après le dernier élément, la boucle se termine et le programme continue en dessous.

7.2 `break` : sortir de la boucle

`break` interrompt **complètement** la boucle, immédiatement :

```
nums = [1, 2, 3, 4, 5]

for num in nums:
    if num == 3:
        print('Trouvé !')
        break
    print(num)
# → 1
# → 2
# → Trouvé !
```

Déroulé : 1 et 2 passent le test et s'affichent ; à 3, la condition déclenche l'affichage de « Trouvé ! » puis `break` — le `print(num)` de ce tour est sauté, et 4 et 5 ne sont jamais traités. Corey fait remarquer un détail d'ordre : si le `print(num)` était placé *au-dessus* du `if`, le 3 se serait affiché avant l'interruption. La position des instructions dans le corps compte.

Usage type : chercher un élément et arrêter dès qu'on l'a trouvé — inutile de parcourir le reste.

7.3 continue : sauter au tour suivant

`continue` abandonne le tour en cours et passe directement à l'itération suivante :

```
nums = [1, 2, 3, 4, 5]

for num in nums:
    if num == 3:
        print('Trouvé !')
        continue
    print(num)
# → 1
# → 2
# → Trouvé !
# → 4
# → 5
```

Même programme, un mot changé : cette fois 4 et 5 sont bien traités — seul le 3 a « sauté » son `print(num)`. `break` = sortir ; `continue` = passer son tour. Usage type de `continue` : filtrer des cas à ignorer (lignes vides d'un fichier, valeurs manquantes...) sans imbriquer tout le reste du corps dans un `else`.

7.4 Boucles imbriquées

Un corps de boucle peut contenir... une autre boucle :

```
nums = [1, 2, 3]

for num in nums:
    for letter in 'abc':
        print(num, letter)
# → 1 a
# → 1 b
# → 1 c
# → 2 a
# → 2 b
# → 2 c
# → 3 a
# → 3 b
# → 3 c
```

Pour **chaque** valeur externe, la boucle interne se déroule **entièrement** : on énumère le produit cartésien 1, 2, 3 × a, b, c, soit $3 \times 3 = 9$ lignes.

Piège — Corey met en garde : les combinaisons explosent vite. Deux boucles de 10 000 éléments = 10^8 tours. Si un traitement imbriqué traîne, le premier réflexe est de compter les itérations — et souvent, un set ou un dictionnaire (recherche en $O(1)$, chapitres 4-5) élimine la boucle interne.

7.5 range : boucler N fois

Pour itérer sur des entiers sans construire de liste à la main, la fonction intégrée `range` :

```
for i in range(10):
    print(i)
# → 0, 1, 2, ..., 9    (chacun sur sa ligne)
```

`range(10)` produit les entiers de **0 inclus à 10 exclu** — la même convention « fin exclue » que le slicing (chapitre 2). Pour 1 à 10 :

```
for i in range(1, 11):
    print(i)
# → 1, 2, ..., 10
```

Pour aller plus loin — Forme complète : `range(début, fin, pas)`. `range(0, 100, 10)` → 0, 10, ..., 90 ; `range(10, 0, -1)` → compte à rebours de 10 à 1. Et rappel du chapitre 4 : pour parcourir une liste avec son index, `enumerate(liste)` bat `range(len(liste))` en lisibilité.

7.6 La boucle while

`for` itère sur une collection ; `while` répète tant qu'une condition reste vraie :

```
x = 0

while x < 10:
    print(x)
    x += 1
# → 0, 1, 2, ..., 9
```

Mécanique : Python évalue `x < 10` ; si vrai, il exécute le corps (afficher, incrémenter), puis **réévalue** la condition, et ainsi de suite. Quand `x` atteint 10, la condition devient fausse et la boucle s'arrête.

Piège majeur — L'incrément `x += 1` est vital : sans lui, `x` reste à 0, la condition reste vraie, et la boucle tourne **pour toujours**. Dans un `while`, quelque chose dans le corps doit faire évoluer la condition (ou un `break` doit exister). C'est LA différence mentale avec `for`, qui s'arrête tout seul à la fin de la collection.

`while` accepte aussi `break` :

```
x = 0

while x < 10:
    if x == 5:
        break
```

```
print(x)
x += 1
# → 0, 1, 2, 3, 4
```

7.7 Boucles infinies : accident... et outil

L'accident d'abord. Si tu lances par mégarde une boucle sans sortie, le programme est bloqué. Le geste de survie, montré dans la vidéo : **Ctrl+C** dans le terminal envoie un `KeyboardInterrupt` qui tue le processus. À connaître *avant* d'en avoir besoin.

Mais la boucle infinie est aussi un **motif volontaire** : `while True:` + un `break` interne :

```
x = 0

while True:
    if x == 5:
        break
    print(x)
    x += 1
# → 0, 1, 2, 3, 4
```

Ici c'est artificiel, mais Corey explique le vrai usage : les programmes qui attendent indéfiniment quelque chose — une saisie utilisateur, une valeur particulière, une connexion réseau — et qui sortent par `break` quand la condition d'arrêt survient au *milieu* du traitement plutôt qu'en tête de boucle. Beaucoup de serveurs et d'interfaces tournent littéralement dans un `while True`.

Pour aller plus loin — Exemple concret du motif, avec la fonction `input()` (qui lit une saisie clavier) :

```
while True:
    reponse = input('Commande (q pour quitter) : ')
    if reponse == 'q':
        break
    print(f'Vous avez tapé : {reponse}')
```

Anecdote de syntaxe : `for` et `while` acceptent une clause `else`, exécutée si la boucle se termine *sans* `break`. C'est rare et déroutant — sache que ça existe, sans en abuser.

Exercices

Exercice 7.1 ● — Affiche la table de 7 (de $7 \times 1 = 7$ à $7 \times 10 = 70$) avec une boucle `for` et une f-string.

Exercice 7.2 ● — Parcours `nombres = [12, 7, 0, 3, -8, 15, 4]` et affiche chaque nombre pair, mais arrête tout dès qu'un nombre négatif apparaît. Quels nombres s'affichent ? (Combine `%`, `continue` et `break` — attention à l'ordre des tests.)

Exercice 7.3 ●● — Somme des multiples : calcule la somme des entiers de 1 à 1000 divisibles par 3 ou par 5 (boucle `for` + accumulateur). Vérifie ensuite ton résultat par la formule des sommes arithmétiques si le cœur t'en dit — Python doit donner 234168.

Exercice 7.4 ●● — Un capital de 1 000 € est placé à 5 % par an. Avec une boucle `while`, détermine au bout de combien d'années entières le capital dépasse 2 000 € (doublement). Affiche l'année et le capital final. (Réponse attendue : 15 ans — la « règle des 72 » donne $\approx 14,4$, cohérent.)

Exercice 7.5 ●●● — Suite de Collatz : partant de `n = 27`, répète — si `n` est pair, `n = n // 2` ; sinon, `n = 3 * n + 1` — jusqu'à atteindre 1. Compte le nombre d'étapes et le plus grand terme rencontré (attendus : 111 étapes, maximum 9232). Structure ton code avec `while True` et `break`, puis réécris-le avec `while n != 1` : laquelle des deux versions préfères-tu, et pourquoi ?

Solutions détaillées

Solution 7.1 —

```
for i in range(1, 11):
    print(f'7 × {i} = {7 * i}')
# → 7 × 1 = 7
# ...
# → 7 × 10 = 70
```

Solution 7.2 —

```
nombres = [12, 7, 0, 3, -8, 15, 4]

for n in nombres:
    if n < 0:
        break
    if n % 2 != 0:
        continue
    print(n)
# → 12
# → 0
```

Le test `break` doit venir **avant** le test de parité : -8 est pair, donc en inversant les tests, le `continue` le sauterait avant que le `break` ait pu s'exécuter... et 4 s'afficherait à tort. (Et oui, 0 est pair : $0 \bmod 2 = 0$.)

Solution 7.3 —

```
total = 0
for n in range(1, 1001):
```

```

if n % 3 == 0 or n % 5 == 0:
    total += n
print(total)    # → 234168

```

Le motif **accumulateur** (initialiser avant la boucle, enrichir dans la boucle, exploiter après) est l'un des plus fondamentaux de la programmation. Vérification par inclusion-exclusion :

$$S_3 + S_5 - S_{15} = 3 \frac{333 \cdot 334}{2} + 5 \frac{200 \cdot 201}{2} - 15 \frac{66 \cdot 67}{2} = 166833 + 100500 - 33165 = 234168 . \checkmark$$

Solution 7.4 —

```

capital = 1000
annees = 0

while capital <= 2000:
    capital *= 1.05
    annees += 1

print(f'{annees} ans — capital final : {capital:.2f} €')
# → 15 ans — capital final : 2078.93 €

```

La condition évolue à chaque tour (le capital croît), donc la boucle termine — le contrat du `while` est rempli. Sanity check mathématique : $1000 \times 1.05^{14} \approx 1979,93$; 2000 ; $1000 \times 1.05^{15} \approx 2078,93$. ✓

Solution 7.5 —

```

n = 27
etapes = 0
maximum = n

while True:
    if n == 1:
        break
    if n % 2 == 0:
        n = n // 2
    else:
        n = 3 * n + 1
    etapes += 1
    if n > maximum:
        maximum = n

print(etapes, maximum)    # → 111 9232

```

Version `while n != 1:` — même corps sans le `if n == 1: break` — plus courte et plus déclarative : la condition d'arrêt se lit dans l'en-tête. Ici, elle est préférable ; `while True` + `break` prend l'avantage quand la condition d'arrêt ne peut se tester qu'au *milieu* du corps (après une saisie, une lecture réseau...). Savoir choisir entre les deux, c'est exactement le contenu de la section 7.7. (Personne ne sait prouver que cette suite atteint toujours 1 — conjecture de Collatz, ouverte depuis 1937. Ton `while` repose sur une conjecture !)

Fiche récap — Chapitre 7

Syntaxe	Effet
<code>for x in iterable:</code>	un tour par élément ; s'arrête tout seul
<code>break</code>	sort complètement de la boucle
<code>continue</code>	abandonne le tour en cours, passe au suivant
Boucles imbriquées	produit cartésien — attention à l'explosion combinatoire
<code>range(n)</code>	0, 1, ..., n-1 (fin exclue)
<code>range(a, b)</code> / <code>range(a, b, pas)</code>	a → b-1 / avec pas (négatif possible)
<code>while cond:</code>	répète tant que <code>cond</code> est vraie — le corps DOIT faire évoluer <code>cond</code>
<code>while True: + break</code>	boucle infinie volontaire (attente d'un événement)
Ctrl+C	<code>KeyboardInterrupt</code> — tue une boucle infinie accidentelle
Motif accumulateur	<code>total = 0</code> avant ; <code>total += ...</code> dedans ; résultat après

CHAPITRE 8

Les fonctions

Vidéo source : [Python Tutorial for Beginners 8: Functions](#) — Corey Schafer

Objectifs : définir des fonctions, comprendre `return`, les paramètres (obligatoires, par défaut), `*args` / `**kwargs` et le déballage, la portée locale, et le principe DRY — jusqu'à un exemple réaliste emprunté à la bibliothèque standard.

8.1 Définir et appeler

Une **fonction** regroupe des instructions sous un nom, pour exécuter une tâche à la demande. Définition : mot-clé `def`, un nom (mêmes conventions `snake_case` que les variables), des parenthèses, deux-points, corps indenté.

```
def hello_func():
    pass
```

`pass` est l'instruction « ne rien faire » : elle permet de laisser un corps vide sans erreur de syntaxe — un vrai emplacement réservé pendant qu'on structure son code.

L'appel se fait avec les **parenthèses** : `hello_func()`. Et elles changent tout :

```
print(hello_func)    # → <function hello_func at 0x...>    (l'objet fonction lui-même)
hello_func()         # exécute la fonction (ici : rien)
```

Sans parenthèses, on désigne la fonction *en tant qu'objet* (utile plus tard — passer une fonction à `sorted(key=...)`, par exemple) ; avec, on l'**exécute**.

```
def hello_func():
    print('Hello Function!')

hello_func()    # → Hello Function!
```

8.2 Pourquoi ? Le principe DRY

Corey le démontre par l'absurde : afficher quatre fois le message = quatre `print` copiés-collés. Le jour où « le chef » demande de changer le texte, il faut retrouver et modifier chaque copie. Avec la fonction : quatre appels `hello_func()`, et **une seule** définition à modifier.

C'est le principe **DRY** — *Don't Repeat Yourself* : toute logique doit vivre à un seul endroit. Les fonctions sont l'outil n°1 pour y parvenir ; si tu te surprends à copier-coller du code, c'est presque toujours qu'une fonction manque.

8.3 `return` : renvoyer une valeur

Une fonction qui se contente d'afficher est un cul-de-sac. La puissance vient de `return` : la fonction *calcule et renvoie* une valeur, que l'appelant utilise comme il veut :

```
def hello_func():
    return 'Hello Function.'

print(hello_func())          # → Hello Function.
print(hello_func().upper())  # → HELLO FUNCTION.
```

La deuxième ligne est importante : `hello_func()` **vaut** une chaîne, donc on peut enchaîner dessus n'importe quelle méthode de chaîne. Corey propose la bonne image mentale : une fonction est une **boîte noire** — des entrées, une sortie ; pour l'utiliser, pas besoin de connaître la tuyauterie interne. Tu pratiques déjà : `len('test')` → 4, sans jamais avoir lu le code de `len`.

Piège — Une fonction sans `return` (ou dont l'exécution atteint la fin du corps) renvoie `None`. Confondre *afficher* et *renvoyer* est l'erreur classique : `print` montre à l'écran mais ne produit aucune valeur exploitable. Si `resultat = ma_fonction()` te donne `None`, vérifie que la fonction `return`-e bien quelque chose.

8.4 Paramètres et arguments

On rend la fonction paramétrable en déclarant des **paramètres** entre les parenthèses de `def` ; l'appelant fournit les **arguments** correspondants :

```
def hello_func(greeting):
    return f'{greeting} Function.'

print(hello_func('Hi'))          # → Hi Function.
# hello_func()                  # TypeError: hello_func() missing 1 required positional
                                # argument: 'greeting'
```

Valeurs par défaut : un paramètre peut devenir optionnel en recevant un défaut dans la signature :

```
def hello_func(greeting, name='You'):
    return f'{greeting}, {name}. Welcome!'

print(hello_func('Hi'))          # → Hi, You. Welcome!
print(hello_func('Hi', name='Mathias'))  # → Hi, Mathias. Welcome!
```

Au second appel, `name='Mathias'` est un **argument nommé** (*keyword argument*) : explicite et insensible à l'ordre — très lisible dès qu'une fonction a plusieurs paramètres.

Piège — Dans la signature, les paramètres **sans défaut viennent obligatoirement avant** ceux avec défaut : `def f(name='You', greeting)` est une `SyntaxError`. Logique : sinon, Python ne saurait pas à qui attribuer les arguments positionnels.

Portée locale : les paramètres et les variables créés dans une fonction n'existent **que dans la fonction**. À l'extérieur, `greeting` n'est pas défini — chaque fonction travaille dans sa bulle, ce qui est une protection : impossible d'écraser par accident les variables du programme principal. (Corey consacre une vidéo entière à la portée ; retiens pour l'instant : *local d'abord, global ensuite* pour la lecture, et une assignation dans la fonction crée toujours une locale.)

8.5 `*args` et `**kwargs` : accepter un nombre libre d'arguments

Certaines fonctions doivent accepter *autant* d'arguments positionnels et nommés qu'on veut. La syntaxe — déroutante au premier regard, avoue Corey — est :

```
def student_info(*args, **kwargs):
    print(args)
    print(kwargs)

student_info('Math', 'Art', name='John', age=22)
# → ('Math', 'Art')
# → {'name': 'John', 'age': 22}
```

Tout s'éclaire en regardant les types : `*args` ramasse les arguments **positionnels** en excès dans un **tuple** ; `**kwargs` (*keyword arguments*) ramasse les arguments **nommés** dans un **dictionnaire**. Les noms `args` et `kwargs` sont de pures conventions — ce sont `*` et `**` qui opèrent — mais respecte-les : tout le monde les attend.

Le déballage (*unpacking*), opération inverse : à l'appel, `*` et `**` éclatent une séquence / un dictionnaire en arguments individuels :

```
courses = ['Math', 'Art']
info = {'name': 'John', 'age': 22}

student_info(courses, info)      # sans étoiles : 2 arguments positionnels...
# → (['Math', 'Art'], {'name': 'John', 'age': 22})
# → {}

student_info(*courses, **info)  # avec étoiles : déballés !
# → ('Math', 'Art')
# → {'name': 'John', 'age': 22}
```

La première version passe la liste et le dict *comme deux objets* ; la seconde les déballé en `student_info('Math', 'Art', name='John', age=22)`. Ce mécanisme sert partout — par exemple pour transmettre des arguments d'une fonction à une autre sans les recopier.

8.6 Docstrings

Juste sous la ligne `def`, une chaîne triple-quotes documente la fonction : c'est la **docstring**.

```
def is_leap(year):
    """Return True for leap years, False for non-leap years."""
    ...
```

Ce n'est pas un simple commentaire : `help(is_leap)` l'affiche, les éditeurs la montrent en infobulle. Bonne pratique : une docstring pour toute fonction non triviale — une phrase qui dit *ce que* fait la fonction, ses entrées, sa sortie.

8.7 Exemple complet : jours dans le mois

Le clou de la vidéo — et un choix malin de Corey : ce code est **repris (légèrement simplifié) de la bibliothèque standard** elle-même. Lire du vrai code, c'est déjà en écrire.

```
# Nombre de jours par mois ; l'index 0 est un bouche-trou pour aligner janvier sur l'index 1
month_days = [0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]

def is_leap(year):
    """Return True for leap years, False for non-leap years."""
    return year % 4 == 0 and (year % 100 != 0 or year % 400 == 0)

def days_in_month(year, month):
    """Return number of days in that month in that year."""
    if not 1 <= month <= 12:
        return 'Invalid Month'
    if month == 2 and is_leap(year):
        return 29
    return month_days[month]

print(is_leap(2017))      # → False
print(is_leap(2020))      # → True
print(days_in_month(2017, 2))  # → 28
print(days_in_month(2020, 2))  # → 29
```

Tout le cours converge ici :

- la **règle bissextile** (déjà travaillée à l'exercice 6.2) : divisible par 4 **et** (pas séculaire **ou** divisible par 400) — directement en `and` / `or` avec les parenthèses qui traduisent la phrase ;
- **return d'une expression booléenne** : `return year % 4 == 0 and ...` renvoie directement `True` / `False` — pas besoin de `if ...: return True else: return False` ;
- la **comparaison chaînée** `1 <= month <= 12` (chapitre 6) et sa négation par `not` ;

- **l'astuce du 0 en tête de liste** : `month_days[1]` tombe sur janvier, `month_days[12]` sur décembre — on paie un entier fantôme pour s'épargner des `month - 1` partout ;
- **la composition** : `days_in_month` appelle `is_leap` — les fonctions se construisent les unes sur les autres ;
- **les retours multiples** : plusieurs `return` dans une fonction sont légaux ; le premier atteint gagne, et la fonction s'arrête là.

Dernier message de la vidéo, à encadrer : ce code existe déjà dans le module `calendar` (`calendar.isleap`, `calendar.monthrange`). **Avant de réinventer, cherche dans la bibliothèque standard** — testée, optimisée, débuggée par des milliers de gens. Transition parfaite vers le chapitre 9.

Exercices

Exercice 8.1 ● — Écris `aire_disque(rayon)` qui renvoie πr^2 (utilise `3.14159` ou, mieux, anticipe le chapitre 9 avec `math.pi` si tu veux). Docstring obligatoire. Affiche l'aire pour $r = 1$ et $r = 2.5$, la seconde arrondie à 2 décimales via f-string.

Exercice 8.2 ● — Écris `interets_composes(capital, taux, annees=1)` qui renvoie la valeur finale $C(1 + r)^n$. Appelle-la : sans préciser les années ; avec `annees=10` en argument nommé ; et vérifie qu'appeler `interets_composes()` sans rien lève bien une `TypeError` (lis le message d'erreur : que t'indique-t-il ?).

Exercice 8.3 ●● — Que vaut `resultat` ? Explique, puis corrige la fonction.

```
def double(x):
    print(x * 2)

resultat = double(21)
print(resultat)
```

Exercice 8.4 ●● — Écris `moyenne(*notes)` qui accepte un nombre quelconque de notes et renvoie leur moyenne (None s'il n'y a aucune note — pourquoi est-ce mieux que de laisser planter ?). Teste `moyenne(12, 15, 9)`, `moyenne(18)`, `moyenne()`. Puis déballe la liste `semestre = [11, 14, 16, 8]` dans un appel.

Exercice 8.5 ●●● — Écris `valeur_actuelle(flux, taux)` qui renvoie la valeur actualisée d'une liste de flux financiers : $\sum_{t=1}^n \frac{F_t}{(1+r)^t}$ (le 1 flux est actualisé sur 1 période). Utilise `enumerate(flux, start=1)`. Vérifie : `valeur_actuelle([100, 100, 1100], 0.05)` doit donner ≈ 1136.16 (une obligation à 3 ans, coupon 100, nominal 1000, actualisée à 5 %).

Solutions détaillées

Solution 8.1 —

```
def aire_disque(rayon):
    """Renvoie l'aire du disque de rayon donné."""
    return 3.14159 * rayon ** 2

print(aire_disque(1))          # → 3.14159
print(f'{aire_disque(2.5):.2f}') # → 19.63
```

Solution 8.2 —

```
def interets_composes(capital, taux, annees=1):
    """Valeur d'un capital placé à taux composé pendant annees."""
    return capital * (1 + taux) ** annees

print(interets_composes(1000, 0.05))          # → 1050.0
print(interets_composes(1000, 0.05, annees=10)) # → 1628.894626777442
# interets_composes()
# TypeError: interets_composes() missing 2 required positional arguments: 'capital' and 'taux'
```

Le message d'erreur est remarquablement précis : il nomme les paramètres manquants — et ne réclame pas `annees`, qui a son défaut. Apprendre à lire les `TypeError`, c'est la moitié du débogage.

Solution 8.3 — Affichage : `42` puis `None`. La fonction *affiche* le double mais ne le *renvoie* pas — donc l'appel vaut `None`, capturé dans `resultat`. C'est le piège de la section 8.3. Correction :

```
def double(x):
    return x * 2

resultat = double(21)
print(resultat)    # → 42
```

Solution 8.4 —

```
def moyenne(*notes):
    """Moyenne arithmétique des notes reçues, ou None si aucune."""
    if not notes:          # tuple vide → falsy (chapitre 6)
        return None
    return sum(notes) / len(notes)

print(moyenne(12, 15, 9))    # → 12.0
print(moyenne(18))           # → 18.0
print(moyenne())              # → None

semestre = [11, 14, 16, 8]
print(moyenne(*semestre))    # → 12.25
```

Sans le garde-fou, `moyenne()` lèverait `ZeroDivisionError: division by zero`. Renvoyer `None` transforme un plantage en cas gérable par l'appelant (`if m is None:` — chapitre 6) ; au chapitre 13, on verra l'alternative : lever une exception explicite.

Solution 8.5 —

```
def valeur_actuelle(flux, taux):
    """Valeur actualisée d'une liste de flux (le 1er flux est dans 1 période)."""
    total = 0
    for t, montant in enumerate(flux, start=1):
        total += montant / (1 + taux) ** t
    return total

va = valeur_actuelle([100, 100, 1100], 0.05)
print(f'{va:.2f}')    # → 1136.16
```

`enumerate(flux, start=1)` fournit exactement l'exposant d'actualisation t — le `start=1` du chapitre 4 trouve ici sa raison d'être financière. Sanity check : coupon 100 = taux 10 % sur nominal 1000, actualisé à 5 % → le prix doit dépasser le pair (1000). ✓ (Et 1136.16 est bien $\frac{100}{1.05} + \frac{100}{1.05^2} + \frac{1100}{1.05^3}$.)

Fiche récap — Chapitre 8

Syntaxe	Effet
<code>def f(...):</code> + corps indenté	définition ; <code>pass</code> = corps vide provisoire
<code>f</code> vs <code>f()</code>	l'objet fonction vs son exécution
<code>return expr</code>	renvoie la valeur et sort ; plusieurs <code>return</code> possibles ; sans → <code>None</code>
DRY	une logique = un seul endroit ; copier-coller ⇒ il manque une fonction
<code>def f(a, b=default):</code>	paramètre obligatoire puis optionnel — jamais l'inverse
<code>f(b=2, a=1)</code>	arguments nommés : lisibles, ordre libre
Portée	variables/paramètres locaux à la fonction
<code>*args</code> / <code>**kwargs</code>	ramassent les positionnels (tuple) / nommés (dict) en nombre libre
<code>f(*liste, **dico)</code>	déballage à l'appel : éclate en arguments individuels
<code>"""docstring"""</code>	documentation intégrée, lue par <code>help()</code>
Réflexe stdlib	avant de coder : la bibliothèque standard l'a peut-être déjà fait

CHAPITRE 9

Modules et bibliothèque standard

Vidéo source : [Python Tutorial for Beginners 9: Import Modules and Exploring The Standard Library](#) — Corey Schafer

Objectifs : découper son code en modules, maîtriser toutes les formes d' `import` , comprendre où Python cherche les modules (`sys.path` , `PYTHONPATH`), et faire un premier tour de la bibliothèque standard (`random` , `math` , `datetime` , `calendar` , `os`).

9.1 Ton premier module

Un **module** n'a rien de magique : c'est un fichier `.py` . Tout fichier Python peut être importé par un autre — c'est ainsi qu'on découpe un projet en morceaux réutilisables.

Deux fichiers dans le même dossier. D'abord `my_module.py` :

```
# my_module.py
print('imported my_module...')

test = 'Test String'

def find_index(to_search, target):
    """Find the index of a value in a sequence."""
    for i, value in enumerate(to_search):
        if value == target:
            return i
    return -1
```

(Une variable, une fonction — qui recycle `enumerate` du chapitre 4 et les retours multiples du chapitre 8.) Puis `intro.py` , qui l'importe :

```
# intro.py
import my_module

courses = ['history', 'math', 'physics', 'compsci']

index = my_module.find_index(courses, 'math')
print(index)
```

```
$ python intro.py
imported my_module...
1
```

Deux enseignements majeurs :

1. **Importer, c'est exécuter.** La ligne `import my_module` exécute *tout* le code de niveau supérieur du module — la preuve : son `print` s'affiche. C'est pour ça qu'un module bien élevé ne contient au niveau supérieur que des définitions (fonctions, classes, constantes), pas d'actions.
2. **L'accès est préfixé :** `my_module.find_index(...)`, `my_module.test`. Le module forme un espace de noms — pas de collision possible avec tes propres noms.

Pour aller plus loin — Après l'import, un dossier `__pycache__` apparaît, contenant du *bytecode* compilé (`.pyc`) : un cache que Python régénère au besoin. Il s'ignore (et ne se versionne pas). Et l'idiome `if __name__ == '__main__':`, que tu croieras partout, permet à un fichier d'avoir du code qui s'exécute *seulement* lancé directement, pas importé — Corey y consacre une vidéo dédiée.

9.2 Toutes les formes d'import

Alias de module — raccourcir un nom long :

```
import my_module as mm

index = mm.find_index(courses, 'math')
```

C'est la convention consacrée de l'écosystème scientifique : `import numpy as np`, `import pandas as pd` — tu les utiliseras constamment en finance quantitative.

Import ciblé — ne prendre que ce dont on a besoin, utilisable **sans préfixe** :

```
from my_module import find_index

index = find_index(courses, 'math')    # plus de préfixe
# print(test)                          # NameError : test n'a pas été importé
```

Plusieurs noms à la fois, avec alias éventuel :

```
from my_module import find_index as fi, test

print(fi(courses, 'math'))    # → 1
print(test)                   # → Test String
```

Piège de lisibilité — Les alias trop courts (`fi`) se paient à la relecture. Corey le dit : garde des noms qui restent compréhensibles pour quelqu'un d'autre — ou pour toi dans trois mois.

L'import à éviter :

```
from my_module import *      # à proscrire
```

Tout arrive sans préfixe... et plus personne ne sait d'où vient quoi. Si trois modules importés en `*` définissent chacun un `find_index`, lequel s'exécute ? Corey est net : c'est mal vu (*frowned upon*) et ça rend le débogage pénible. Les seuls imports `*` que tu verras sont dans de vieux tutoriels — pas dans du code sérieux.

9.3 Où Python cherche-t-il les modules ? `sys.path`

Comment `import my_module` a-t-il trouvé le fichier ? Python parcourt une liste de dossiers, visible dans `sys.path` :

```
import sys
print(sys.path)
# → ['C:\\Users\\mathias\\Documents\\python-cours', 'C:\\...\\python312.zip',
#     'C:\\...\\Lib', ..., 'C:\\...\\site-packages']
```

L'ordre de recherche :

1. le **dossier du script** en cours d'exécution — voilà pourquoi notre import marchait ;
2. les dossiers de la variable d'environnement `PYTHONPATH` (si définie) ;
3. la **bibliothèque standard** ;
4. `site-packages` — les paquets tiers installés (chapitre 10).

Si le module n'est dans aucun de ces dossiers : `ModuleNotFoundError: No module named 'my_module'`.

Solution 1 (à la main, pour comprendre) — ajouter le dossier à la liste avant l'import :

```
import sys
sys.path.append(r'C:\\Users\\mathias\\Desktop\\my_modules')

import my_module
```

Ça marche, mais Corey le qualifie lui-même de « pas très joli » : le chemin est codé en dur dans le script, et il faudrait le dupliquer partout.

Solution 2 (propre) — définir `PYTHONPATH`. Sous Windows : Démarrer → « variables d'environnement » → *Variables d'environnement...* → *Nouvelle* → nom `PYTHONPATH`, valeur `C:\\Users\\mathias\\Desktop\\my_modules` → OK, puis rouvrir le terminal. (Sur Mac : une ligne `export PYTHONPATH="..."` dans `~/.zprofile`.) Le dossier apparaît alors dans `sys.path` de tous tes scripts. Les IDE ont en plus leurs propres réglages de chemins — cherche « *ton éditeur* + python path ».

Depuis la vidéo — En 2026, on règle surtout ce problème autrement : en travaillant *dans un projet* dont le script principal vit à la racine, et en installant ses dépendances dans un environnement virtuel (chapitre 11). `PYTHONPATH` reste utile à connaître, mais si tu en dépends lourdement, c'est souvent le signe d'une organisation de projet à revoir.

9.4 Visite guidée de la bibliothèque standard

La **bibliothèque standard** (*standard library*) est la collection de modules livrée avec Python — « piles incluses », dit le slogan. Elle est **déjà installée** : il suffit d'importer. Échantillon de la vidéo :

`random` — aléatoire :

```
import random

courses = ['history', 'math', 'physics', 'compsci']
random_course = random.choice(courses)
print(random_course)    # → physics    (par exemple : varie à chaque exécution)
```

`math` — fonctions mathématiques :

```
import math

rads = math.radians(90)
print(rads)          # → 1.5707963267948966    ( $\pi/2$ )
print(math.sin(rads)) # → 1.0
```

(Tu y trouveras aussi `math.pi`, `math.e`, `math.sqrt`, `math.log`, `math.exp`... — ta boîte à outils d'analyse. Pour le calcul vectoriel sérieux, ce sera NumPy, hors stdlib.)

`datetime` et `calendar` — dates :

```
import datetime
import calendar

today = datetime.date.today()
print(today)          # → 2026-08-24    (la date du jour)

print(calendar.isleap(2017)) # → False
print(calendar.isleap(2020)) # → True
```

Clin d'œil de Corey : `calendar.isleap`, c'est exactement notre `is_leap` du chapitre 8 — la preuve en direct que la stdlib avait déjà la réponse, testée et optimisée. Le chapitre 17 approfondira `datetime`.

`os` — parler au système d'exploitation :

```
import os

print(os.getcwd())    # → C:\Users\mathias\Documents\python-cours    (répertoire courant)
```

Le chapitre 18 lui est entièrement consacré.

9.5 Lire le code source de la stdlib

Où vivent ces modules ? Chaque module connaît son propre fichier :

```
import os
print(os.__file__)    # → C:\...\Python312\Lib\os.py
```

Ouvre ce dossier `Lib` : la bibliothèque standard est là, **en clair**. Le conseil de Corey est excellent : lire ce code, écrit par des développeurs chevronnés, est un accélérateur d'apprentissage — on y voit les conventions, les docstrings, les astuces du métier. (Certains modules sont en C, invisibles — mais l'essentiel est en Python lisible.)

Et la démonstration finale, l'easter egg le plus célèbre du langage :

```
import antigravity
```

... ouvre dans ton navigateur la BD *xkcd* n° 353 sur Python. Va lire le fichier `antigravity.py` : trois lignes. Même les blagues de la stdlib sont du code lisible.

Exercices

Exercice 9.1 ● — Crée `finance_utils.py` contenant une constante `TAUX_DEFAULT = 0.03` et ta fonction `interets_composes` du chapitre 8. Depuis un second fichier, importe le module avec l'alias `fu` et calcule la valeur de 2 000 € sur 5 ans au taux par défaut du module.

Exercice 9.2 ● — Réécris l'import de 9.1 pour n'importer *que* `interets_composes` et `TAUX_DEFAULT`, sans préfixe. Cite deux raisons pour lesquelles `from finance_utils import *` serait une mauvaise idée.

Exercice 9.3 ●● — Ajoute en tête de `finance_utils.py` la ligne `print('module finance_utils chargé')`. Dans le fichier principal, importe-le **deux fois** (deux lignes `import`). Combien de fois le message s'affiche-t-il ? Exécute, constate, et explique (indice : Python met les modules importés en cache dans `sys.modules`).

Exercice 9.4 ●● — Avec les bons modules de la stdlib : tire un échantillon de 5 lancers de dé (`random.randint(1, 6)` dans une compréhension ou une boucle), calcule leur moyenne, puis affiche $\sqrt{2}$, $e^{0.05}$ et $\ln(2)$ avec `math` (arrondis à 4 décimales). Bonus : que fait `random.seed(42)` placé avant les tirages ? Relance deux fois pour le voir.

Exercice 9.5 ●●● — Sans exécuter : ce projet contient `stats.py` (avec une fonction `mean`) et `main.py` dans le même dossier. `main.py` commence par `import stats`. Ton collègue le lance depuis un *autre* dossier avec `python C:\projet\main.py`... et l'import fonctionne quand même. Explique pourquoi, en citant la règle précise de `sys.path`. Puis : il renomme `stats.py` en `statistics.py` — quel problème subtil vient-il de créer ? (Indice : la stdlib a un module `statistics`.)

Solutions détaillées

Solution 9.1 —

```
# finance_utils.py
TAUX_DEFAULT = 0.03

def interets_composes(capital, taux, annees=1):
    """Valeur d'un capital placé à taux composé pendant annees."""
    return capital * (1 + taux) ** annees
```

```
# principal.py
import finance_utils as fu

valeur = fu.interets_composes(2000, fu.TAUX_DEFAULT, annees=5)
print(f'{valeur:.2f} €')    # → 2318.55 €
```

Solution 9.2 —

```
from finance_utils import interets_composes, TAUX_DEFAULT

valeur = interets_composes(2000, TAUX_DEFAULT, annees=5)
```

Contre l'import `*` : 1) on ne sait plus, en lisant le code, d'où vient chaque nom (débogage et relecture pénibles) ; 2) risque de collision silencieuse — si deux modules définissent `TAUX_DEFAULT`, le dernier importé écrase l'autre sans le moindre avertissement.

Solution 9.3 — Le message s'affiche **une seule fois**. Au premier `import`, Python exécute le module et le range dans le cache `sys.modules` ; au second, il le retrouve dans le cache et ne réexécute rien. C'est voulu : un module utilisé par vingt fichiers d'un projet n'est chargé qu'une fois.

Solution 9.4 —

```
import random
import math

lancers = []
for _ in range(5):          # _ : convention pour « variable inutilisée »
    lancers.append(random.randint(1, 6))
print(lancers)              # → [6, 1, 1, 6, 3]    (par exemple)
print(sum(lancers) / len(lancers))  # → 3.4      (par exemple)

print(round(math.sqrt(2), 4))    # → 1.4142
print(round(math.exp(0.05), 4))  # → 1.0513
print(round(math.log(2), 4))     # → 0.6931
```

`random.seed(42)` fixe l'état initial du générateur **pseudo**-aléatoire : les tirages deviennent reproductibles à l'identique d'une exécution à l'autre — indispensable pour déboguer une simulation Monte-Carlo ou publier un résultat vérifiable.

Solution 9.5 — Règle n° 1 de `sys.path` : Python ajoute en tête **le dossier contenant le script exécuté** — pas le dossier courant du terminal. `main.py` vit dans `C:\projet`, donc `C:\projet` est dans `sys.path`, et `stats.py` y est trouvé, peu importe d'où on lance la commande. — Le renommage en `statistics.py`, lui, **masque le module `statistics` de la stdlib** : comme le dossier du script passe *avant* la bibliothèque standard dans `sys.path`, tout `import statistics` du projet (y compris ceux faits en interne par d'autres bibliothèques !) chargera le fichier local. Bugs incompréhensibles garantis. Règle pratique : **ne jamais donner à ses fichiers le nom d'un module standard** (`statistics.py`, `random.py`, `math.py`, `csv.py`, `json.py`...).

Fiche récap — Chapitre 9

Syntaxe	Effet
module	= un fichier <code>.py</code> ; l'importer exécute son code top-level (une seule fois, cache <code>sys.modules</code>)
<code>import mod</code>	accès préfixé : <code>mod.fonction</code> , <code>mod.variable</code>
<code>import mod as m</code>	alias (<code>np</code> , <code>pd</code> ... conventions à respecter)
<code>from mod import f, x</code>	accès direct sans préfixe (le reste du module non importé)
<code>from mod import f as fi</code>	alias ciblé — rester lisible
<code>from mod import *</code>	à proscrire : origine des noms intraçable, collisions
<code>sys.path</code>	ordre de recherche : dossier du script → <code>PYTHONPATH</code> → stdlib → site-packages
<code>ModuleNotFoundError</code>	module hors de ces dossiers ; fix : organisation du projet, <code>PYTHONPATH</code>
<code>mod.__file__</code>	où vit le module — la stdlib se lit !
stdlib essentiels	<code>random</code> (choice, randint, seed), <code>math</code> (pi, sqrt, sin, log), <code>datetime</code> , <code>calendar</code> (isleap), <code>os</code>
nommage	jamais de fichier au nom d'un module standard (<code>statistics.py</code> ...)

PARTIE III

Python au quotidien

CHAPITRE 10

pip : installer des paquets tiers

Vidéo source : [Python Tutorial: pip - An in-depth look at the package management system](#) — Corey Schafer

Objectifs : installer, lister, mettre à jour et désinstaller des paquets avec pip, et maîtriser le duo `pip freeze / requirements.txt` qui rend un projet reproductible.

10.1 Au-delà de la stdlib : PyPI et pip

La bibliothèque standard (chapitre 9) couvre énormément — mais l'écosystème Python vit surtout dans les **paquets tiers** : NumPy, pandas, matplotlib, requests... Ils sont hébergés sur **PyPI** (*Python Package Index*, pypi.org), et s'installent avec **pip**, le gestionnaire de paquets livré avec Python.

Toutes les commandes de ce chapitre se tapent **dans le terminal** (pas dans l'interpréteur Python !) — c'est une confusion classique de débutant : `pip install ...` dans le `>>>` provoque une `SyntaxError`.

10.2 S'orienter : `pip help`

Corey commence par le réflexe santé : pip s'auto-documente.

```
pip help          → liste des commandes (install, uninstall, list, freeze, ...)
pip help install  → toutes les options d'une commande (-r, --upgrade, --user, ...)
```

Trouver un paquet : la vidéo montre `pip search`, qui interrogeait PyPI depuis le terminal.

Depuis la vidéo — `pip search` est **désactivé** depuis 2021 (l'API de PyPI a fermé, victime de son succès). Aujourd'hui, on cherche directement sur pypi.org ou sur Google ; la page PyPI d'un paquet donne sa commande d'installation exacte, sa doc et sa popularité.

10.3 Le cycle de vie d'un paquet

Installer :

```
C:\> pip install requests
Collecting requests
...
Successfully installed requests-2.32.3 certifi-... charset_normalizer-... idna-...
urllib3-...
```

Remarque : pip a installé `requests` **et ses dépendances** — les paquets dont il a lui-même besoin. C'est tout l'intérêt d'un gestionnaire de paquets.

Lister ce qui est installé, avec les versions :

```
C:\> pip list
Package            Version
-----
certifi            2026.x.x
pip                24.x
requests           2.32.3
...
```

Repérer ce qui a vieilli :

```
C:\> pip list --outdated
Package    Version    Latest    Type
-----
setuptools 68.0.0     75.1.0    wheel
```

(`-o` en abrégé.) Pour chaque paquet dépassé : version installée, dernière version disponible.

Mettre à jour un paquet :

```
C:\> pip install -U requests
```

Le `-U` majuscule (ou `--upgrade`) réinstalle la dernière version.

Désinstaller :

```
C:\> pip uninstall requests
...
Proceed (Y/n)? y
Successfully uninstalled requests-2.32.3
```

(pip demande confirmation ; il ne retire pas automatiquement les dépendances devenues orphelines.)

Depuis la vidéo — La forme recommandée aujourd'hui est `python -m pip install ...` (ou `py -m pip ...` sous Windows) : elle garantit que pip installe pour le Python que tu crois, quand plusieurs versions cohabitent. `pip ...` tout court marche très bien tant que ton PATH est propre ; retiens simplement que `python -m pip` est la forme « anti-ambiguïté ». Pense aussi à mettre à jour pip lui-même de temps en temps :

```
python -m pip install -U pip.
```

10.4 pip freeze et requirements.txt : la reproductibilité

Le passage le plus important de la vidéo. `pip freeze` affiche les paquets installés au format exact `paquet==version` :

```
C:\> pip freeze
certifi==2026.x.x
requests==2.32.3
...
```

Ce format n'est pas décoratif : c'est celui du fichier `requirements.txt`, la liste des dépendances d'un projet. On le génère par redirection :

```
C:\> pip freeze > requirements.txt
```

Et dans l'autre sens, **n'importe qui** (un collègue, un serveur, toi sur une autre machine) reconstitue l'environnement à l'identique :

```
C:\> pip install -r requirements.txt
```

Le `-r` lit le fichier et installe chaque paquet à la **version épinglée**. C'est LA réponse au « ça marche chez moi » : mêmes versions partout, comportement identique. Corey insiste : dès qu'un projet a plus de deux dépendances, envoyer un `requirements.txt` écrase toute autre méthode (« installe à peu près ces paquets... » = catastrophe assurée).

Piège — Les versions comptent vraiment : entre deux versions majeures d'une bibliothèque, des fonctions changent de nom ou de comportement. Un `requirements.txt` sans versions (`requests` au lieu de `requests==2.32.3`) redevient une loterie. `pip freeze` épinglé tout, c'est sa raison d'être.

10.5 Tout mettre à jour d'un coup ?

pip n'a **pas** de commande native « upgrade all ». La vidéo montre le contournement historique, un one-liner glané sur Stack Overflow, qui enchaîne des commandes shell :

```
pip freeze --local | grep -v '^-e' | cut -d = -f 1 | xargs -n1 pip install -U
```

Décorticage (tel que Corey le fait) : `pip freeze --local` liste les paquets (seulement locaux dans un venv) → `grep -v '^-e'` écarte les installations « editable » → `cut -d = -f 1` garde le nom avant le `==` → `xargs -n1 pip install -U` lance la mise à jour paquet par paquet. Honnêteté remarquable de Corey au passage : « je ne suis pas certain de ce que fait ce grep » — et il cite sa source. Deux réflexes de bon développeur : citer d'où vient un code, et ne pas prétendre comprendre ce qu'on n'a pas vérifié.

Depuis la vidéo — Ce one-liner est Unix (`grep/cut/xargs` n'existent pas dans CMD Windows). Et surtout, la *pratique* a changé : on ne met plus à jour « tout, aveuglément » — on travaille par projet dans un environnement virtuel (chapitre 11) avec des versions épinglées, qu'on met à jour délibérément. La question « comment tout upgrader » disparaît presque d'elle-même avec une bonne organisation.

10.6 Où pip installe-t-il ?

Par défaut, dans le dossier `site-packages` de ton installation Python — le 4^e emplacement de `sys.path` (chapitre 9) : voilà le lien entre `pip install` et `import`. Tout s'installe *globalement*... ce qui devient vite un problème : deux projets qui exigent deux versions différentes du même paquet ne peuvent pas cohabiter dans un espace global unique. La solution est le sujet du chapitre suivant : **les environnements virtuels**.

Exercices

Exercice 10.1 ● — Dans un terminal : affiche la version de pip (`pip --version`), la liste de tes paquets, puis installe le paquet `cowsay` (petit et inoffensif). Vérifie qu'il apparaît dans `pip list`, teste-le (`python -c "import cowsay; cowsay.cow('Python !')"`), puis désinstalle-le.

Exercice 10.2 ● — Génère le `requirements.txt` de ton installation actuelle. Ouvre-le dans ton éditeur : combien de lignes ? Repère un paquet que tu n'as jamais installé toi-même — d'où vient-il probablement ?

Exercice 10.3 ●● — Ton binôme t'envoie ce `requirements.txt` : `requests==2.32.3` et `python-dateutil==2.9.0`. Écris la commande unique qui installe le tout, puis la commande qui vérifie que les versions installées correspondent. Pourquoi `pip install requests python-dateutil` (sans fichier) n'est-il pas équivalent ?

Exercice 10.4 ●●● — Question de compréhension : ton script plante avec `ModuleNotFoundError: No module named 'requests'` alors que `pip install requests` répond « Requirement already satisfied ». Donne deux causes plausibles et la commande de diagnostic qui départage (indice : chapitre 9, `sys.path` — et pense « plusieurs Python installés »).

Solutions détaillées

Solution 10.1 —

```
pip --version          → pip 24.x from C:\...\site-packages\pip (python 3.12)
pip list              → tableau paquet/version
pip install cowsay     → Successfully installed cowsay-6.1
python -c "import cowsay; cowsay.cow('Python !')"
```

→ une vache ASCII parle

```
pip uninstall cowsay  → Proceed (Y/n)? y
```

Solution 10.2 — `pip freeze > requirements.txt`. Le fichier contient typiquement bien plus de lignes que de paquets installés volontairement : la différence, ce sont les **dépendances** installées automatiquement (installer `requests` amène `certifi`, `urllib3`, `idna`, `charset-normalizer`...). `pip freeze` photographie tout, dépendances comprises — c'est ce qui rend la reconstruction fidèle.

Solution 10.3 —


```
pip install -r requirements.txt
pip list          (ou pip freeze, à comparer au fichier)
```

Sans le fichier, `pip install requests python-dateutil` installerait les **dernières** versions du moment, pas celles épinglées — dans six mois, ton binôme et toi n'auriez plus le même environnement, et un comportement différent entre vos machines ne serait plus une surprise mais une certitude.

Solution 10.4 — Causes classiques : **1)** plusieurs installations de Python — `pip` a installé dans le Python A, mais `python` (ou l'éditeur/IDE) exécute le Python B ; **2)** le script tourne dans un environnement virtuel (chapitre 11) où le paquet n'est pas installé — ou inversement, `pip` a installé dans le venv et le script tourne hors venv. Diagnostic :

```
python -c "import sys; print(sys.executable)"    → QUEL python exécute ?
pip --version                                   → pour QUEL python pip installe-t-il ?
```

Si les deux chemins divergent, tout est expliqué. Le remède est la forme anti-ambiguïté de la section 10.3 : `python -m pip install requests` — `pip` devient *forcément* celui du `python` qu'on invoque.

Fiche récap — Chapitre 10

Commande (terminal!)	Effet
<code>pip help</code> / <code>pip help install</code>	aide générale / options d'une commande
<code>pip install paquet</code>	installe depuis PyPI (+ dépendances)
<code>pip install -U paquet</code>	met à jour (<code>--upgrade</code>)
<code>pip uninstall paquet</code>	désinstalle (confirmation demandée)
<code>pip list</code> / <code>pip list --outdated</code>	installés + versions / dépassés (installée vs dernière)
<code>pip freeze</code>	format <code>paquet==version</code>
<code>pip freeze > requirements.txt</code>	photographie l'environnement
<code>pip install -r requirements.txt</code>	reconstruit l'environnement à l'identique
<code>python -m pip ...</code>	forme anti-ambiguïté (le <code>pip</code> DU <code>python</code> invoqué)
Chercher un paquet	pypi.org (<code>pip search</code> est mort)
Où ça s'installe	<code>site-packages</code> (→ trouvé par <code>import</code> via <code>sys.path</code>)

CHAPITRE 11

Environnements virtuels : venv

Vidéo source : [Python Tutorial: VENV \(Windows\) - How to Use Virtual Environments with the Built-In venv Module](#) — Corey Schafer (une vidéo jumelle existe pour Mac/Linux)

Objectifs : comprendre pourquoi isoler les dépendances par projet, créer/activer/désactiver/supprimer un environnement virtuel, et adopter la structure de projet standard avec `requirements.txt`.

11.1 Le problème que venv résout

Fin du chapitre 10 : `pip install` range tout dans un unique `site-packages` global. Scénario inévitable : le projet A a été écrit pour `pandas 1.x`, le projet B exige `pandas 2.x`. Une seule installation globale $\Rightarrow$ mettre à jour pour B casse A. Et impossible de savoir quels paquets appartiennent à quel projet.

L'**environnement virtuel** (*virtual environment*) règle tout : c'est une installation Python « de poche », **propre à un projet**, avec son propre `site-packages`. Chaque projet a le sien ; les versions n'entrent plus jamais en collision. Règle d'or moderne, telle que Corey la formule : **un projet = un environnement virtuel**.

Le module `venv` qui fait ça est **dans la bibliothèque standard** depuis Python 3.3 — rien à installer. (D'autres outils existent — `virtualenv`, `conda`, `poetry`, `uv`... — mais Corey le dit et ça reste vrai : `venv` couvre l'immense majorité des besoins.)

11.2 Créer et activer (Windows)

Créer — dans un terminal, là où tu veux ranger l'environnement :

```
C:\Users\mathias> python -m venv projet_env
```

(`-m` exécute un module comme un programme — souvenir du chapitre 10 avec `python -m pip`.) Un dossier `projet_env` apparaît, contenant un interpréteur Python et un `site-packages` vierges.

Activer :

```
C:\Users\mathias> projet_env\Scripts\activate
(projet_env) C:\Users\mathias>
```

Le nom entre parenthèses en tête d'invite est le **signal** : cet environnement est actif. Désormais, dans ce terminal, `python` et `pip` désignent ceux de l'environnement. Preuve avec `where python` (l'équivalent Windows de `which`) :

```
(projet_env) C:\Users\mathias> where python
C:\Users\mathias\projet_env\Scripts\python.exe      ← en premier : celui du venv
C:\Users\mathias\AppData\Local\Programs\Python\Python312\python.exe
```

Piège Windows/PowerShell — La vidéo utilise l'invite CMD (`activate.bat`). Sous **PowerShell** (le terminal par défaut de VS Code), la commande est `projet_env\Scripts\Activate.ps1` — et si PowerShell refuse en parlant de « scripts désactivés », lance une fois : `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser`, puis réessaie.

L'isolation, en direct :

```
(projet_env) C:\> pip list
Package      Version
-----
pip          24.x
```

Quasi vide — seulement pip (et parfois setuptools) : l'environnement ne voit **pas** les paquets globaux. Installons :

```
(projet_env) C:\> pip install requests pytz
```

`pip list` montre maintenant requests, pytz et leurs dépendances — **dans le venv uniquement**. L'environnement hérite par ailleurs de la **version de Python** qui l'a créé (créé avec un 3.12, il tourne en 3.12 ; pour jongler entre versions de Python, c'est le lanceur `py -3.11 -m venv ...`).

Désactiver — revenir au Python global :

```
(projet_env) C:\> deactivate
C:\>
```

Supprimer — un venv est **jetable** : c'est juste un dossier.

```
C:\> rmdir /s /q projet_env
```

(`/s` : sous-dossiers ; `/q` : sans confirmation. La corbeille marche aussi.) Aucune cérémonie : tout se recrée en une minute grâce au `requirements.txt`.

11.3 La structure de projet standard

La vidéo assemble ensuite les bonnes pratiques en un schéma type, à adopter tel quel :

```
mon_projet\
├─ venv\           ← l'environnement, nommé « venv » par convention
├─ script.py       ← ton code
└─ requirements.txt ← les dépendances épinglées
```

Création :

```
C:\> mkdir mon_projet
C:\> cd mon_projet
C:\Users\...\mon_projet> python -m venv venv
C:\Users\...\mon_projet> venv\Scripts\activate
(venv) C:\Users\...\mon_projet> pip install requests pytz
(venv) C:\Users\...\mon_projet> pip freeze > requirements.txt
```

Les deux règles qui vont avec :

Règle 1 — Jamais de code dans le dossier venv. Le venv est jetable et régénérable ; ton code ne l'est pas. `script.py` vit à côté de `venv\`, jamais dedans.

Règle 2 — Le venv ne se partage pas, le requirements.txt si. Avec git : `venv/` va dans le `.gitignore` ; `requirements.txt` est versionné. Quiconque clone le projet refait l'environnement en deux commandes : `python -m venv venv` puis (après activation) `pip install -r requirements.txt`.

La boucle du chapitre 10 est bouclée : la vidéo le démontre en *supprimant* l'environnement puis en le reconstruisant à l'identique depuis le fichier — c'est le test ultime de reproductibilité, à faire une fois pour se convaincre.

11.4 Option avancée : voir les paquets globaux

Cas particulier montré en fin de vidéo : créer un venv qui **voit** les paquets globaux, tout en installant ses nouveautés localement :

```
python -m venv venv --system-site-packages
```

`pip list` montre alors le global + le local ; pour ne lister/geler que ce qui appartient au venv : `pip list --local` et `pip freeze --local`. L'isolation en écriture demeure : ce qu'on installe dans le venv n'apparaît jamais dans le global (la vidéo le vérifie en désactivant puis en relançant `pip list`). Corey est clair : option à réserver aux cas où réinstaller un gros paquet global serait pénible — par défaut, isolation complète.

Depuis la vidéo — Deux habitudes modernes en plus : VS Code détecte le dossier `venv` et propose de l'utiliser comme interpréteur du projet (statut en bas à droite — plus besoin d'activer à la main pour exécuter depuis l'éditeur) ; et si tu explores l'écosystème, l'outil `uv` (2024+) fait venv + pip beaucoup plus vite avec les mêmes concepts. Ce que tu viens d'apprendre reste la base que tous ces outils reproduisent.

Exercices

Exercice 11.1 ● — Crée un projet `essai_venv` avec la structure standard : dossier, venv, activation, installe `cowsay`, gèle dans `requirements.txt`, puis désactive. Vérifie ensuite que `python -c "import cowsay"` échoue hors venv (si `cowsay` n'est pas installé globalement) et réussit venv activé.

Exercice 11.2 ● — Terminal venv activé : que répondent `where python` et `pip -V` ? Terminal désactivé ? Explique en une phrase ce que l'activation modifie réellement (indice : la variable d'environnement `PATH`).

Exercice 11.3 ●● — Reproductibilité complète : supprime entièrement le dossier `venv` de l'exercice 11.1, puis reconstruis l'environnement depuis `requirements.txt`. Combien de commandes t'ont suffi ? Pourquoi la suppression était-elle sans danger ?

Exercice 11.4 ●●● — Diagnostic : ton binôme te dit « j'ai activé le venv, installé pandas, mais VS Code souligne `import pandas` en rouge et le script plante ». Propose les deux causes les plus probables et leurs remèdes (pense : quel interpréteur VS Code utilise-t-il ? dans quel terminal l'installation a-t-elle eu lieu ?).

Solutions détaillées

Solution 11.1 —

```
C:\> mkdir essai_venv
C:\> cd essai_venv
C:\essai_venv> python -m venv venv
C:\essai_venv> venv\Scripts\activate
(venv) C:\essai_venv> pip install cowsay
(venv) C:\essai_venv> pip freeze > requirements.txt
(venv) C:\essai_venv> python -c "import cowsay; print('ok')"      → ok
(venv) C:\essai_venv> deactivate
C:\essai_venv> python -c "import cowsay"
→ ModuleNotFoundError: No module named 'cowsay'
```

L'échec hors venv est la **preuve** de l'isolation — c'est le comportement désiré, pas un bug.

Solution 11.2 — Activé : `where python` place `...\venv\Scripts\python.exe` en tête, et `pip -V` affiche un chemin dans `...\venv\...\site-packages`. Désactivé : le Python global reprend la tête. L'activation ne fait « que » ça : elle **préfixe le PATH** du terminal avec `venv\Scripts`, si bien que `python` et `pip` résolvent d'abord vers le venv (et elle habille l'invite du `(venv)`). Rien de magique — c'est pour ça que fermer le terminal « désactive » aussi.

Solution 11.3 —

```
C:\essai_venv> rmdir /s /q venv
C:\essai_venv> python -m venv venv
C:\essai_venv> venv\Scripts\activate
(venv) C:\essai_venv> pip install -r requirements.txt
```

Trois commandes utiles (+ activation). Sans danger car le venv ne contient **aucune information originale** : tout ce qui compte (le code, la liste des dépendances) vit à côté. Le venv est du bétail, pas un animal de compagnie.

Solution 11.4 — Cause 1 : VS Code n'utilise pas l'interpréteur du venv. L'éditeur analyse `import pandas` avec le Python global (où pandas manque) → soulignage rouge, et « Run » exécute le mauvais Python. Remède :

`Ctrl+Shift+P` → *Python: Select Interpreter* → choisir `.\venv\Scripts\python.exe`. **Cause 2 : l'installation a eu lieu dans le mauvais terminal** — un terminal sans `(venv)` au moment du `pip install`, donc pandas est parti dans le global (ou dans un autre venv). Remède : réinstaller depuis un terminal où `(venv)` est visible, ou en forme anti-ambiguïté `venv\Scripts\python -m pip install pandas`. Diagnostic universel dans les deux cas : comparer `sys.executable` du script avec `pip -V`.

Fiche récap — Chapitre 11

Commande / règle	Effet
Problème résolu	dépendances par projet, plus de conflits de versions
<code>python -m venv venv</code>	crée l'environnement (module stdlib, Python ≥ 3.3)
<code>venv\Scripts\activate</code>	active (CMD) ; PowerShell : <code>Activate.ps1</code> ; le prompt affiche <code>(venv)</code>
<code>deactivate</code>	désactive ; fermer le terminal aussi
<code>where python</code> / <code>pip -V</code>	vérifier QUEL python/pip est actif
<code>rmdir /s /q venv</code>	supprime — un venv est jetable
Structure type	<code>projet\{venv\, script.py, requirements.txt}</code>
Règles d'or	1 projet = 1 venv ; jamais de code dans <code>venv\</code> ; versionner <code>requirements.txt</code> , ignorer <code>venv\</code>
Reconstruction	<code>python -m venv venv + pip install -r requirements.txt</code>
<code>--system-site-packages</code>	venv qui voit le global (cas particulier) ; <code>pip freeze --local</code>
Python du venv	= celui qui l'a créé (<code>py -3.11 -m venv ...</code> pour choisir)

CHAPITRE 12

Lire et écrire des fichiers

Vidéo source : [Python Tutorial: File Objects - Reading and Writing to Files](#) — Corey Schafer

Objectifs : ouvrir des fichiers proprement avec `with`, choisir le bon mode (`r / w / a / b`), lire de quatre façons différentes selon la taille du fichier, écrire sans détruire, et copier des fichiers binaires par blocs.

Fichier d'exemple pour tout le chapitre — `test.txt`, dans le dossier du script :

```
1) This is a test file
2) With multiple lines of data
3) Third line
4) Fourth line
```

12.1 Ouvrir... et surtout fermer : pourquoi `with`

Première manière, montrée pour comprendre — **pas** pour être imitée :

```
f = open('test.txt', 'r')

print(f.name)    # → test.txt
print(f.mode)    # → r

f.close()
```

`open()` renvoie un **objet fichier** (*file object*) ; le deuxième argument est le **mode** (lecture par défaut). Le problème de cette forme : il faut penser à `close()` — et un oubli ne pardonne pas. Fichier non fermé = ressource système qui fuit ; une application qui ouvre beaucoup de fichiers sans les fermer peut dépasser la limite de descripteurs et **planter**. Pire : si une erreur survient entre `open` et `close`, le `close` n'est jamais exécuté.

La forme correcte, à utiliser **systématiquement** : le **gestionnaire de contexte** (*context manager*) avec `with` :

```
with open('test.txt', 'r') as f:
    f_contents = f.read()
    print(f_contents)
# ← à la sortie du bloc, f est fermé AUTOMATIQUEMENT
```

Garanties du `with` : le fichier est fermé à la sortie du bloc, **même si une exception éclate au milieu**. On a encore accès à la variable après le bloc — mais le fichier, lui, est clos :

```
with open('test.txt', 'r') as f:
    pass
```

```
print(f.closed)    # → True
# f.read()         # ValueError: I/O operation on closed file.
```

Piège spécial francophone (absent de la vidéo, crucial sous Windows) — Précise l'**encodage** : `open('test.txt', 'r', encoding='utf-8')`. Sans lui, Windows utilise un encodage hérité (cp1252) : tes accents (é , à , €) se liront de travers ou feront planter la lecture d'un fichier UTF-8. Prends le réflexe `encoding='utf-8'` sur *tous* tes `open` en mode texte. (Les exemples ci-dessous l'omettent pour rester proches de la vidéo — ajoute-le dans ton code réel.)

12.2 Les modes d'ouverture

Mode	Signification
'r'	lecture (défaut) — erreur si le fichier n'existe pas
'w'	écriture — crée le fichier, ou ÉCRASE son contenu s'il existe
'a'	ajout (<i>append</i>) — écrit à la fin, sans toucher l'existant
'r+'	lecture + écriture
'rb' / 'wb'	idem <code>r / w</code> en binaire (images, PDF...)

12.3 Lire : quatre techniques, un curseur

Tout d'un coup — `read()` :

```
with open('test.txt', 'r') as f:
    f_contents = f.read()
    print(f_contents)    # → tout le fichier, en une chaîne
```

Simple, mais tout le fichier monte **en mémoire** — parfait pour un petit fichier, dangereux pour un gros.

Toutes les lignes en liste — `readlines()` :

```
with open('test.txt', 'r') as f:
    f_lines = f.readlines()
    print(f_lines)
# → ['1) This is a test file\n', '2) With multiple lines of data\n', ...]
```

Note les `\n` conservés en fin de chaque ligne. Même limite mémoire que `read()`.

Une ligne à la fois — `readline()` :

```
with open('test.txt', 'r') as f:
    print(f.readline(), end='')    # → 1) This is a test file
    print(f.readline(), end='')    # → 2) With multiple lines of data
```


Chaque appel avance d'une ligne — le fichier maintient un **curseur** de position. (Le `end=''` évite les lignes vides : la ligne lue contient déjà son `\n`, et `print` en ajoute un par défaut.)

La bonne méthode par défaut : itérer sur le fichier

```
with open('test.txt', 'r') as f:
    for line in f:
        print(line, end='')
```

L'objet fichier est **itérable ligne par ligne** : une seule ligne en mémoire à la fois, quelle que soit la taille du fichier — des gigaoctets de logs se parcourent ainsi sans effort. C'est la technique à privilégier.

Par blocs de taille fixe — `read(n)`, pour un contrôle total :

```
size_to_read = 100

with open('test.txt', 'r') as f:
    f_contents = f.read(size_to_read)
    while len(f_contents) > 0:
        print(f_contents, end='*')
        f_contents = f.read(size_to_read)
```

`read(100)` renvoie (au plus) 100 caractères et avance le curseur ; chaque appel reprend où le précédent s'était arrêté ; en fin de fichier, `read` renvoie une **chaîne vide** → la boucle `while` s'arrête (truthiness, chapitre 6 — `while f_contents:` marcherait aussi). L'astérisque de séparation ne sert qu'à *visualiser* les frontières de blocs dans la démo.

Contrôler le curseur — `tell()` et `seek()` :

```
with open('test.txt', 'r') as f:
    print(f.read(10))    # → 1) This is
    print(f.tell())      # → 10           position actuelle
    f.seek(0)            # retour au début
    print(f.read(10))    # → 1) This is   (relu depuis le début)
```

Sans le `seek(0)`, la seconde lecture aurait continué au caractère 11.

12.4 Écrire

Mode `'w'` :

```
with open('test2.txt', 'w') as f:
    f.write('Test')
    f.write('Test')
```

Résultat dans `test2.txt` : `TestTest` — comme en lecture, le curseur avance à chaque `write` (pas de retour à la ligne automatique ; écrire `'Test\n'` pour en obtenir).

Piège majeur — Ouvrir un fichier existant en `'w'` efface immédiatement tout son contenu, avant même le premier `write`. Pour ajouter sans détruire : mode `'a'`. Vérifie toujours deux fois le mode quand le fichier t'est précieux.

Curiosité instructive montrée dans la vidéo — `seek` en écriture ne « tronque » pas, il **superpose** :

```
with open('test2.txt', 'w') as f:
    f.write('Test')
    f.seek(0)
    f.write('R')
# test2.txt contient : Rest
```

Le `R` a remplacé le seul caractère à la position 0 — le reste (`est`) est resté. D'où : `seek` + `write`, c'est pour des formats à positions fixes ; pour « recommencer proprement », on rouvre en `'w'`.

12.5 Copier un fichier : lire l'un, écrire l'autre

Deux `with` imbriqués — un fichier en lecture (`rf`), un en écriture (`wf`) :

```
with open('test.txt', 'r') as rf:
    with open('test_copy.txt', 'w') as wf:
        for line in rf:
            wf.write(line)
```

Ligne à ligne : mémoire minimale, quelle que soit la taille. (Les deux `with` peuvent aussi s'écrire sur une seule ligne, séparés par une virgule.)

12.6 Le binaire : copier une image

Tentons la même chose avec une photo `bronx.jpg` :

```
# with open('bronx.jpg', 'r') as rf: # UnicodeDecodeError: 'utf-8'/'charmap' codec
can't decode byte...
```

Normal : une image n'est pas du texte — la « décoder » en caractères n'a pas de sens. On passe en **mode binaire** : `'rb'` et `'wb'`, où l'on manipule des **bytes** bruts :

```
with open('bronx.jpg', 'rb') as rf:
    with open('bronx_copy.jpg', 'wb') as wf:
        for line in rf:
            wf.write(line)
```

Ça fonctionne... mais itérer « par lignes » un fichier qui n'en a pas est bancal. La version propre — et le motif générique de traitement des gros fichiers — est la **copie par blocs** (*chunks*), reprise de la section 12.3 :

```

chunk_size = 4096

with open('bronx.jpg', 'rb') as rf:
    with open('bronx_copy.jpg', 'wb') as wf:
        rf_chunk = rf.read(chunk_size)
        while len(rf_chunk) > 0:
            wf.write(rf_chunk)
            rf_chunk = rf.read(chunk_size)

```

4096 octets lus, écrits, jusqu'à ce que `read` renvoie des bytes vides. Mémoire constante (4 Ko), fichiers de n'importe quelle taille.

Pour aller plus loin — Dans la vraie vie, copier un fichier se fait en une ligne avec `shutil.copyfile(src, dst)` (`stdlib`) — mais le motif *lire par blocs / écrire au fil de l'eau* que tu viens d'apprendre est celui de tous les traitements de flux : téléchargements, hachage de fichiers, encodage... Il fallait l'écrire une fois à la main. Mentionnons aussi `pathlib` (chapitre 18) : `Path('test.txt').read_text(encoding='utf-8')` pour les cas simples.

Exercices

Exercice 12.1 ● — Crée `notes.txt` depuis Python : trois lignes (`Analyse I`, `Probabilités`, `Optimisation`), une par `write`. Puis rouvre-le en lecture et affiche son contenu. Enfin ajoute `Statistiques` **sans effacer** l'existant et vérifie.

Exercice 12.2 ● — Sur le `test.txt` du chapitre : affiche uniquement la **première** ligne, puis, dans un *second* `with`, le nombre **total** de lignes du fichier (compteur + itération — pas `readlines`, imagine le fichier énorme).

Exercice 12.3 ●● — Prédise le contenu final de `sortie.txt`, puis vérifie :

```

with open('sortie.txt', 'w') as f:
    f.write('ABCDEF')
with open('sortie.txt', 'w') as f:
    f.write('123')
with open('sortie.txt', 'a') as f:
    f.write('456')

```

Exercice 12.4 ●● — Écris un programme qui copie `notes.txt` vers `notes_numerotees.txt` en préfixant chaque ligne de son numéro : `1. Analyse I`, etc. (`enumerate(rf, start=1)` fonctionne sur un fichier ouvert !)

Exercice 12.5 ●●● — Sans l'exécuter, explique le bug de ce code censé afficher deux fois le fichier, puis donne deux corrections différentes :

```

with open('test.txt', 'r') as f:
    print(f.read())
    print(f.read())

```

Solutions détaillées

Solution 12.1 —

```
with open('notes.txt', 'w', encoding='utf-8') as f:
    f.write('Analyse I\n')
    f.write('Probabilités\n')
    f.write('Optimisation\n')

with open('notes.txt', 'r', encoding='utf-8') as f:
    print(f.read())
# → Analyse I
#   Probabilités
#   Optimisation

with open('notes.txt', 'a', encoding='utf-8') as f:
    f.write('Statistiques\n')
```

Les `\n` sont à ta charge en écriture, et le `encoding='utf-8'` protège « Probabilités » de la charabia-isation sous Windows.

Solution 12.2 —

```
with open('test.txt', 'r') as f:
    print(f.readline(), end='')    # → 1) This is a test file

with open('test.txt', 'r') as f:
    count = 0
    for line in f:
        count += 1
    print(count)                  # → 4
```

L'itération ligne à ligne garde une mémoire constante — c'est tout l'intérêt face à `len(f.readlines())`, qui chargerait le fichier entier. (Version d'expert, pour plus tard : `sum(1 for _ in f)`.)

Solution 12.3 — Contenu final : `123456`. Le premier `'w'` écrit `ABCDEF` ; le deuxième `'w'` **écrase tout** dès l'ouverture puis écrit `123` ; le `'a'` ajoute `456` à la suite. Toute la différence `w/a` tient dans cet exercice.

Solution 12.4 —

```
with open('notes.txt', 'r', encoding='utf-8') as rf:
    with open('notes_numerotees.txt', 'w', encoding='utf-8') as wf:
        for i, line in enumerate(rf, start=1):
            wf.write(f'{i}. {line}')

# notes_numerotees.txt :
# 1. Analyse I
# 2. Probabilités
# 3. Optimisation
# 4. Statistiques
```

`line` contient déjà son `\n` final : le réutiliser tel quel évite d'en gérer un soi-même.

Solution 12.5 — Le second `print(f.read())` affiche... une **chaîne vide**. Après le premier `read()`, le curseur est en fin de fichier ; le second `read()` ne trouve plus rien à lire. Correction 1 — rembobiner : `f.seek(0)` entre les deux lectures. Correction 2 — lire une fois, réutiliser : `contenu = f.read()` puis deux `print(contenu)` (préférable : une seule lecture disque). Ce « piège du curseur » explique l'immense majorité des « mon fichier est vide ?! » des débutants.

Fiche récap — Chapitre 12

Syntaxe	Effet
<code>with open(path, mode, encoding='utf-8') as f:</code>	ouverture auto-fermée (même sur exception) — LA forme à utiliser
Modes	<code>'r'</code> lire (défaut) ; <code>'w'</code> écrire (écrase !) ; <code>'a'</code> ajouter ; <code>'r+'</code> ; + <code>'b'</code> = binaire
<code>f.read()</code> / <code>f.read(n)</code>	tout (petits fichiers) / n caractères, reprend au curseur
<code>f.readline()</code> / <code>f.readlines()</code>	une ligne / liste de toutes les lignes (avec leurs <code>\n</code>)
<code>for line in f:</code>	itération ligne à ligne, mémoire constante — le défaut à adopter
<code>f.tell()</code> / <code>f.seek(0)</code>	position du curseur / déplacement (0 = début)
<code>f.write(s)</code>	écrit (pas de <code>\n</code> automatique) ; en <code>'w'</code> + <code>seek</code> , superpose sans tronquer
Copie texte	deux <code>with</code> imbriqués, <code>for line in rf: wf.write(line)</code>
Binaire	<code>'rb'</code> / <code>'wb'</code> (sinon <code>UnicodeDecodeError</code>) ; copie par blocs <code>read(4096)</code> en boucle
Fin de fichier	<code>read</code> renvoie <code>''</code> / <code>b''</code> (falsy) → condition d'arrêt naturelle
Windows/accents	toujours <code>encoding='utf-8'</code> en mode texte

CHAPITRE 13

Gérer les erreurs : try / except

Vidéo source : [Python Tutorial: Using Try/Except Blocks for Error Handling](#) — Corey Schafer

Objectifs : intercepter les exceptions proprement, comprendre pourquoi un `except` nu est un piège, ordonner plusieurs `except`, exploiter `else` et `finally`, et lever ses propres exceptions avec `raise`.

13.1 Pourquoi gérer les erreurs

Depuis le début du cours, tu as croisé des **exceptions** : `IndexError`, `KeyError`, `TypeError`, `ValueError`, `FileNotFoundError`... Quand l'une d'elles survient sans être gérée, Python s'arrête net et affiche un **traceback** — précieux pour *toi* en développement, catastrophique pour l'utilisateur final : programme mort, message cryptique qui expose tes entrailles de code.

Le bloc `try / except` change la donne : il permet d'**anticiper** les points fragiles (fichier absent, saisie invalide, réseau en panne...), de réagir avec élégance, et de laisser le programme continuer.

13.2 La forme de base

Le fil rouge de la vidéo : ouvrir un fichier qui n'existe pas.

```
# f = open('testfile.txt')
# FileNotFoundError: [Errno 2] No such file or directory: 'testfile.txt'
```

Version protégée :

```
try:
    f = open('testfile.txt')
except FileNotFoundError:
    print('Sorry. This file does not exist')
# → Sorry. This file does not exist
```

Mécanique : Python exécute le bloc `try` ; à la **première** exception, il saute immédiatement au bloc `except` correspondant, l'exécute, puis **le programme continue normalement**. Pas de traceback, pas d'arrêt.

13.3 Le piège de l'`except` nu

On voit souvent (trop souvent) :

```
try:
    f = open('testfile.txt')
```

```
except:                                # attrape TOUT
    print('Sorry. This file does not exist')
```

Corey démontre pourquoi c'est dangereux avec une expérience remarquable. Le fichier existe maintenant, mais on introduit une *autre* erreur dans le `try` :

```
try:
    f = open('test.txt')                # ce fichier EXISTE
    var = bad_var                        # variable jamais définie → NameError
except:
    print('Sorry. This file does not exist')
# → Sorry. This file does not exist      (?!)
```

Le message affirme que le fichier n'existe pas... alors qu'il s'est ouvert sans problème ! L'`except` nu a gobé le `NameError` — une erreur qu'on n'avait pas prévue — et l'a déguisé en problème de fichier. **Le bug est masqué ET mal étiqueté** : le pire des deux mondes, et des heures de débogage en perspective.

D'où la règle : **toujours nommer l'exception qu'on attrape**. Avec `except FileNotFoundError:`, le `NameError`, lui, remonte avec son traceback exact :

```
try:
    f = open('test.txt')
    var = bad_var
except FileNotFoundError:
    print('Sorry. This file does not exist')
# NameError: name 'bad_var' is not defined    ← le VRAI problème, visible
```

C'est le comportement sain : on gère ce qu'on a prévu, et l'imprévu **doit** faire du bruit.

13.4 Plusieurs `except`, et l'ordre qui compte

On peut chaîner les gestionnaires, du **plus spécifique au plus général** :

```
try:
    f = open('testfile.txt')
    var = bad_var
except FileNotFoundError:
    print('Sorry. This file does not exist')
except Exception:
    print('Sorry. Something went wrong')
```

Fichier manquant → premier bloc ; toute autre erreur → second (`Exception` est la classe mère de presque toutes les exceptions — un filet de sécurité *explicite*, très différent d'un `except` nu par accident).

Piège d'ordre — Python teste les `except` **de haut en bas** et exécute le premier qui correspond. Si `except Exception` venait en premier, il attraperait *aussi* les `FileNotFoundError`, et le bloc spécifique deviendrait du code mort. Spécifique d'abord, général ensuite — toujours.

Récupérer l'objet exception avec `as` — pour afficher le vrai message d'erreur sans arrêter le programme :

```
try:
    f = open('testfile.txt')
except FileNotFoundError as e:
    print(e)
except Exception as e:
    print(e)
# → [Errno 2] No such file or directory: 'testfile.txt'
```

`e` est l'objet exception lui-même ; l'afficher donne le message natif de Python — souvent plus informatif que notre texte maison, idéal pour les journaux (*logs*).

13.5 `else` et `finally` : le quatuor complet

```
try:
    f = open('test.txt')
except FileNotFoundError as e:
    print(e)
else:
    print(f.read())
    f.close()
finally:
    print('Executing Finally...')
```

`else` s'exécute **seulement si le `try` n'a levé aucune exception**. Pourquoi ne pas mettre `f.read()` directement dans le `try` ? Réponse subtile de la vidéo : le `try` doit contenir *uniquement* le code dont on surveille l'échec. Si `read()` était dans le `try` et échouait pour une raison imprévue, notre `except` risquerait de l'attraper et d'afficher un message trompeur — on retomberait dans le piège de la section 13.3. `else` = « le code qui dépend du succès, mais qu'on ne surveille pas ».

`finally` s'exécute **dans tous les cas** : succès, exception attrapée, exception non attrapée, même un `return` au milieu. C'est la place du **nettoyage** : fermer une connexion à une base de données, libérer une ressource, supprimer un fichier temporaire — ce qui doit arriver *quoi qu'il advienne*.

Tableau de synthèse :

Scénario	try	except	else	finally
aucune exception	✓	—	✓	✓
exception attrapée	✓ (interrompu)	✓	—	✓

Scénario	try	except	else	finally
exception non attrapée	✓ (interrompu)	—	—	✓ (puis traceback)

13.6 Lever ses propres exceptions : `raise`

On peut déclencher une exception volontairement — pour signaler qu'une règle *métier* est violée, même si Python, lui, n'y voit rien d'illégal :

```
try:
    f = open('corrupt_file.txt')
    if f.name == 'corrupt_file.txt':
        raise Exception
except FileNotFoundError as e:
    print(e)
except Exception as e:
    print('Error!')
else:
    print(f.read())
    f.close()
finally:
    print('Executing Finally...')
# → Error!
# → Executing Finally...
```

Le fichier s'ouvre très bien — c'est *notre* code qui décide que ce nom de fichier est inacceptable et lève l'exception, aussitôt attrapée par le `except Exception`. Et `finally` s'exécute, comme promis.

En pratique, on lève des exceptions **précises et documentées** :

```
def taux_periodique(taux_annuel, periodes):
    if periodes <= 0:
        raise ValueError(f'periodes doit être > 0, reçu {periodes}')
    return (1 + taux_annuel) ** (1 / periodes) - 1
```

Pour aller plus loin — Faut-il renvoyer `None` (solution 8.4) ou lever une exception quand une entrée est invalide ? Les deux se défendent ; la culture Python penche pour l'exception : impossible à ignorer silencieusement, message explicite, traceback qui pointe le fautif. Réserve `None` aux absences *normales* (« pas trouvé »), l'exception aux situations *anormales* (« données invalides »). Tu peux aussi définir tes propres types d'exceptions — il suffit d'hériter d' `Exception` (mécanisme du chapitre 22).

Les exceptions courantes à reconnaître (tu les as presque toutes déjà rencontrées dans ce cours) :

Exception	Cause typique	Vue au chapitre
<code>ValueError</code>	bonne forme, mauvaise valeur : <code>int('abc')</code>	3

Exception	Cause typique	Vue au chapitre
<code>TypeError</code>	types incompatibles : <code>'a' * 'b'</code> ; mauvais nombre d'arguments	3, 8
<code>IndexError</code>	index hors limites : <code>liste[99]</code>	2, 4
<code>KeyError</code>	clé absente d'un dict	5
<code>NameError</code>	variable jamais définie	13
<code>ZeroDivisionError</code>	division par zéro	8
<code>FileNotFoundError</code>	fichier introuvable	12, 13

Exercices

Exercice 13.1 ● — Demande un nombre avec `nombre = float(input('Un nombre : '))` protégé par un `try / except ValueError` qui affiche « Ce n'est pas un nombre. » au lieu de planter. Teste avec `3.14` puis `abc`.

Exercice 13.2 ● — Écris `inverse(x)` qui renvoie $1/x$, et un appel protégé qui gère séparément `ZeroDivisionError` (« division par zéro ») et `TypeError` (« entrée non numérique » — teste avec `inverse('a')`). Pourquoi deux `except` distincts plutôt qu'un seul `except Exception` ?

Exercice 13.3 ●● — Prédis la sortie complète de ce programme dans les trois cas `x = 4`, `x = 0`, `x = 'a'`, puis vérifie :

```
def teste(x):
    try:
        r = 10 / x
    except ZeroDivisionError:
        print('div zéro')
    else:
        print('résultat', r)
    finally:
        print('fin')

teste(4); teste(0); teste('a')
```

Exercice 13.4 ●● — Reprends la lecture de fichier du chapitre 12 : écris `lire_config(chemin)` qui renvoie le contenu du fichier, ou la chaîne `'[config absente]'` si le fichier n'existe pas — mais qui laisse remonter toute autre erreur. Justifie ce choix en une phrase (règle 13.3).

Exercice 13.5 ●●● — Écris `volatilite(rendements)` qui lève `ValueError` avec un message explicite si la liste contient moins de 2 éléments, et sinon renvoie l'écart-type *échantillon* $\sqrt{\frac{1}{n-1} \sum (r_i - \bar{r})^2}$ (chapitres 7-8 : boucles/accumulateurs, ou compréhension si tu as lu en avance). Montre l'appel protégé qui affiche le message d'erreur via `as e`. Vérifie : `volatilite([0.01, 0.03, -0.02, 0.02])` ≈ 0.0216 , et `volatilite([0.01])` → message.

Solutions détaillées

Solution 13.1 —

```
try:
    nombre = float(input('Un nombre : '))
except ValueError:
    print("Ce n'est pas un nombre.")
else:
    print(f'Le double vaut {nombre * 2}')
```

`float('abc')` lève `ValueError` → message propre au lieu du traceback. (L'usage de `else` pour le code « en cas de succès » : section 13.5.)

Solution 13.2 —

```
def inverse(x):
    return 1 / x

for valeur in [4, 0, 'a']:
    try:
        print(inverse(valeur))
    except ZeroDivisionError:
        print('division par zéro')
    except TypeError:
        print('entrée non numérique')

# → 0.25
# → division par zéro
# → entrée non numérique
```

Deux `except` distincts car les **réactions** doivent différer : à terme, l'un pourrait substituer une valeur par défaut, l'autre re-demander la saisie. Un `except Exception` unique écraserait ces deux cas en un seul message flou — et gober aussi les erreurs *imprévues*, qu'on veut voir remonter.

Solution 13.3 —

```
teste(4)    → résultat 2.5    puis    fin
teste(0)    → div zéro       puis    fin
teste('a')  → fin            puis x  TypeError: unsupported operand type(s) for /: 'int' and 'str'
```

Le troisième cas est le plus instructif : le `TypeError` n'est **pas** attrapé (on ne gère que `ZeroDivisionError`), mais `finally` s'exécute *quand même* avant que l'exception ne remonte — exactement la ligne 3 du tableau de la section 13.5.

Solution 13.4 —

```
def lire_config(chemin):
    try:
        with open(chemin, encoding='utf-8') as f:
            return f.read()
    except FileNotFoundError:
        return '[config absente]'
```

On n'attrape **que** `FileNotFoundError`, cas prévu et gérable (une config absente a un sens : valeurs par défaut). Un `PermissionError` ou un disque en erreur sont des situations anormales que masquer serait dangereux : on les laisse remonter, fidèle à « gérer le prévu, laisser crier l'imprévu ».

Solution 13.5 —

```
def volatilité(rendements):
    """Écart-type échantillon d'une liste de rendements."""
    n = len(rendements)
    if n < 2:
        raise ValueError(f'Il faut au moins 2 rendements, reçu {n}')
    moyenne = sum(rendements) / n
    somme_carres = 0
    for r in rendements:
        somme_carres += (r - moyenne) ** 2
    return (somme_carres / (n - 1)) ** 0.5

try:
    print(round(volatilité([0.01, 0.03, -0.02, 0.02]), 4))    # → 0.0216
    print(volatilité([0.01]))
except ValueError as e:
    print('Erreur :', e)

# → 0.0216
# → Erreur : Il faut au moins 2 rendements, reçu 1
```

Le `raise ValueError(...)` avec message *incluant la valeur fautive* est un cadeau au futur débogueur (souvent toi). Noter : la validation se fait **en tête de fonction** — on échoue tôt (*fail fast*), avant tout calcul. (La `stdlib` fait ce calcul via `statistics.stdev` — réflexe du chapitre 9 ; ici on s'entraîne.)

Fiche récap — Chapitre 13

Syntaxe	Effet
<code>try: ... except TypeErreur:</code>	saute au gestionnaire à la 1 ^{ère} exception du try ; le programme continue ensuite
<code>except:</code> nu	attrape tout, masque et mal-étiquette les bugs — toujours nommer
Plusieurs <code>except</code>	testés de haut en bas : spécifique d'abord , Exception en dernier filet
<code>except ... as e</code>	l'objet exception ; <code>print(e)</code> → message natif

Syntaxe	Effet
<code>else:</code>	exécuté si aucune exception — le code qui dépend du succès
<code>finally:</code>	exécuté dans tous les cas — nettoyage (fermer connexions/ressources)
<code>raise ValueError('msg')</code>	lever soi-même ; message explicite avec la valeur fautive ; <i>fail fast</i>
Philosophie	gérer les échecs prévus , laisser remonter l'imprévu bruyamment
À reconnaître	ValueError, TypeError, IndexError, KeyError, NameError, ZeroDivisionError, FileNotFoundError

CHAPITRE 14

Les compréhensions

Vidéo source : [Python Tutorial: Comprehensions - How they work and why you should be using them](#) — Corey Schafer

Objectifs : écrire des compréhensions de listes, de dictionnaires et de sets, les préférer à `map` / `filter` / `lambda`, découvrir `zip`, et croiser les expressions génératrices.

14.1 L'idée, et la méthode de la vidéo

Une **compréhension de liste** (*list comprehension*) est une syntaxe compacte et lisible pour **construire une liste**. La pédagogie de Corey est excellente et on va la suivre à la lettre : pour chaque exemple, d'abord la **boucle classique** (que tu sais déjà écrire depuis le chapitre 7), puis la **compréhension équivalente** — et tu constateras qu'elle se lit comme la phrase d'intention.

Matériau de base :

```
nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Exemple 1 — copier chaque élément. Intention : « je veux `n` pour chaque `n` dans `nums` ».

```
# boucle
my_list = []
for n in nums:
    my_list.append(n)

# compréhension
my_list = [n for n in nums]
print(my_list)  # → [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

La compréhension est la phrase : « je veux `n` — `for n in nums` pour chaque `n` dans `nums` ». Trois lignes-rituel (créer vide, boucler, append) fondues en une expression.

Exemple 2 — transformer. « Je veux `n*n` pour chaque `n` dans `nums` » :

```
my_list = [n * n for n in nums]
print(my_list)  # → [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

La partie gauche (avant le `for`) est l'**expression de sortie** — c'est là qu'on transforme. En notation mathématique, tu reconnais la définition d'ensemble/famille en compréhension — d'où le nom : $n^2 \mid n \in \text{nums}$.

14.2 La parenthèse `map` / `lambda` — et pourquoi on s'en passe

L'« ancienne » façon fonctionnelle de faire l'exemple 2 :

```
my_list = list(map(lambda n: n * n, nums))
print(my_list)  # → [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

`lambda` crée une petite fonction anonyme, `map` l'applique à chaque élément. Ça marche — mais l'argument de Corey est imparable : un débutant lit `[n * n for n in nums]` sans formation ; `map(lambda ...)` exige de connaître deux concepts de plus pour un résultat identique. Son verdict : « 99 % du temps », un `map` / `filter` + `lambda` se réécrit en compréhension plus claire. C'est le consensus Python moderne — retiens que `map` / `filter` / `lambda` existent (tu en croiseras), écris des compréhensions.

14.3 Filtrer avec `if`

« Je veux `n` pour chaque `n` dans `nums` si `n` est pair » :

```
# boucle
my_list = []
for n in nums:
    if n % 2 == 0:
        my_list.append(n)

# compréhension
my_list = [n for n in nums if n % 2 == 0]
print(my_list)  # → [2, 4, 6, 8, 10]
```

Le `if` final filtre les éléments (le `n % 2` sort tout droit du chapitre 3). Équivalent fonctionnel — même critique de lisibilité : `list(filter(lambda n: n % 2 == 0, nums))`.

Et on peut combiner transformation et filtre : `[n * n for n in nums if n % 2 == 0]` → les carrés des seuls nombres pairs. Version mathématique : $n^2 \mid n \in \text{nums}, n \equiv 0 \pmod{2}$.

14.4 Doubles boucles : le produit cartésien

« Je veux la paire (`lettre`, `chiffre`) pour chaque lettre de `'abcd'` et chaque chiffre de `range(4)` » :

```
# boucle
my_list = []
for letter in 'abcd':
    for num in range(4):
        my_list.append((letter, num))

# compréhension
my_list = [(letter, num) for letter in 'abcd' for num in range(4)]
print(my_list)
# → [('a', 0), ('a', 1), ('a', 2), ('a', 3), ('b', 0), ('b', 1), ('b', 2), ('b', 3),
#      ('c', 0), ('c', 1), ('c', 2), ('c', 3), ('d', 0), ('d', 1), ('d', 2), ('d', 3)]
```

Les deux `for` s'écrivent dans le même ordre que la boucle imbriquée : le premier est la boucle externe. C'est `'abcd' × 0, 1, 2, 3` — seize tuples (l'explosion combinatoire du chapitre 7 s'applique ici aussi !).

Piège — Les noms utilisés dans l'expression de sortie doivent être ceux des `for` : `[(letter, x) for letter in 'abcd' for num in range(4)]` → `NameError` (ou pire : capture silencieuse d'une variable extérieure qui s'appellerait `x`). Une compréhension est autosuffisante : tout nom qu'elle émet doit y être défini.

14.5 `zip`, puis les compréhensions de dictionnaires

Nouvel outil d'abord : `zip` apparie deux séquences élément par élément, comme une fermeture éclair :

```
names = ['Bruce', 'Clark', 'Peter', 'Diana']
heroes = ['Batman', 'Superman', 'Spiderman', 'Wonder Woman']

print(list(zip(names, heroes)))
# → [('Bruce', 'Batman'), ('Clark', 'Superman'), ('Peter', 'Spiderman'), ('Diana', 'Wonder Woman')]
```

« Je veux un dict `{name: hero}` pour chaque paire de `zip(names, heroes)` » :

```
# boucle
my_dict = {}
for name, hero in zip(names, heroes):
    my_dict[name] = hero

# compréhension de dictionnaire
my_dict = {name: hero for name, hero in zip(names, heroes)}
print(my_dict)
# → {'Bruce': 'Batman', 'Clark': 'Superman', 'Peter': 'Spiderman', 'Diana': 'Wonder Woman'}
```

Reconnais les marqueurs : **accolades + deux-points** `clé: valeur` — la signature du dict (chapitre 5). Et le filtre s'ajoute pareil :

```
my_dict = {name: hero for name, hero in zip(names, heroes) if name != 'Peter'}
print(my_dict)
# → {'Bruce': 'Batman', 'Clark': 'Superman', 'Diana': 'Wonder Woman'}
```

(`zip` s'arrête à la plus courte des séquences ; il en accepte trois ou plus. Avec `dict(zip(names, heroes))` on obtient l'appariement brut — la compréhension gagne dès qu'il y a transformation ou filtre.)

14.6 Compréhensions de sets

```
nums = [1, 1, 2, 1, 3, 4, 3, 4, 5, 5, 6, 7, 8, 7, 9, 9]

my_set = {n for n in nums}
print(my_set)  # → {1, 2, 3, 4, 5, 6, 7, 8, 9}
```


Accolades sans deux-points = set (doublons éliminés, chapitre 4). Utile avec une transformation :

`{n % 3 for n in nums}` → les restes distincts. (Pour une simple déduplication, `set(nums)` suffit.)

14.7 Expressions génératrices

Dernière venue, la **generator expression** — même syntaxe, entre **parenthèses** :

```
nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

my_gen = (n * n for n in nums)
print(my_gen)           # → <generator object <genexpr> at 0x...>

for i in my_gen:
    print(i)             # → 1, 4, 9, ..., 100 (un par ligne)
```

Un **générateur** ne calcule rien à l'avance : il produit ses valeurs **une à la fois, à la demande** (*lazy evaluation*) — l'équivalent de la fonction à `yield` que Corey montre en parallèle et détaille dans une vidéo dédiée. Conséquence : mémoire constante là où la liste matérialise tout. Usage idiomatique : alimenter directement une fonction agrégante —

```
total = sum(n * n for n in range(1, 101))
print(total)      # → 338350      ( $\sum n^2$  pour  $n=1..100$ , sans jamais créer la liste)
```

Pour aller plus loin — Un générateur s'épuise : après l'avoir parcouru une fois, il est vide (comme le curseur de fichier du chapitre 12 !). Et il ne connaît pas `len()`. Règle simple : résultat à réutiliser/indexer → liste ; consommation unique immédiate (sum, max, for) → générateur.

14.8 Le mot de la fin : lisibilité

Les compréhensions sont un sommet d'élégance **tant qu'elles tiennent en une pensée**. Une compréhension à trois `for` et deux `if`, illisible, trahit l'esprit de l'outil : reviens alors à la boucle explicite — c'est Corey lui-même qui le suggère. Le juge de paix est toujours le lecteur.

Exercices

Exercice 14.1 ● — En compréhensions : 1) les cubes de 1 à 10 ; 2) les multiples de 7 entre 1 et 100 ; 3) les longueurs des mots de `['si', 'python', 'savait', 'compter']` (attendu : `[2, 6, 6, 7]`).

Exercice 14.2 ● — Traduis en une compréhension chacune de ces boucles :

```
# a)
res = []
```

```

for x in range(20):
    if x % 2 == 1:
        res.append(x ** 2)

# b)
paires = []
for i in range(3):
    for j in range(3):
        if i != j:
            paires.append((i, j))

```

Exercice 14.3 ●● — `tickers = ['aapl', 'msft', 'nvda']` et `prix = [189.5, 412.3, 118.7]`. Construis en une ligne chacun : 1) le dict `{'AAPL': 189.5, ...}` (tickers en majuscules) ; 2) la liste des tickers (majuscules) dont le prix dépasse 150 ; 3) le set des prix arrondis à l'entier. Vérifie : `{'AAPL': 189.5, 'MSFT': 412.3, 'NVDA': 118.7}`, `['AAPL', 'MSFT']`, et — surprise à expliquer — `{412, 190, 119}`.

Exercice 14.4 ●● — Rendements mensuels `r = [0.01, -0.02, 0.03, 0.005, -0.01]`. En une expression chaque fois : 1) les rendements *positifs* ; 2) les facteurs bruts `1 + r_i` ; 3) le facteur cumulé $\prod (1 + r_i)$ avec `math.prod` et une expression génératrice ; 4) le dict `{numéro_de_mois: rendement}` (mois numérotés depuis 1 — `enumerate` !).

Exercice 14.5 ●●● — La matrice **A** à 2 lignes et 3 colonnes ci-dessous est stockée comme liste de listes : `A = [[1, 2, 3], [4, 5, 6]]` (une sous-liste par ligne). En compréhensions (imbriquées) : 1) la matrice **2A** ; 2) la liste aplatie `[1, 2, 3, 4, 5, 6]` ; 3) la transposée `[[1, 4], [2, 5], [3, 6]]`. Pour la transposée, écris d'abord la double boucle, puis la compréhension — et commente la gymnastique d'ordre des `for`.

Solutions détaillées

Solution 14.1 —

```

cubes = [n ** 3 for n in range(1, 11)]
# → [1, 8, 27, 64, 125, 216, 343, 512, 729, 1000]
mult7 = [n for n in range(1, 101) if n % 7 == 0]
# → [7, 14, 21, 28, 35, 42, 49, 56, 63, 70, 77, 84, 91, 98]
longueurs = [len(mot) for mot in ['si', 'python', 'savait', 'compter']]
# → [2, 6, 6, 7]

```

Solution 14.2 —

```

res = [x ** 2 for x in range(20) if x % 2 == 1]
# → [1, 9, 25, 49, 81, 121, 169, 225, 289, 361]
paires = [(i, j) for i in range(3) for j in range(3) if i != j]
# → [(0, 1), (0, 2), (1, 0), (1, 2), (2, 0), (2, 1)]

```

Le `if` final coexiste sans problème avec les deux `for` — l'ordre de lecture reste celui de la boucle d'origine.

Solution 14.3 —

```

tickers = ['aapl', 'msft', 'nvda']
prix = [189.5, 412.3, 118.7]

cotations = {t.upper(): p for t, p in zip(tickers, prix)}
# → {'AAPL': 189.5, 'MSFT': 412.3, 'NVDA': 118.7}
chers = [t.upper() for t, p in zip(tickers, prix) if p > 150]
# → ['AAPL', 'MSFT']
arrondis = {round(p) for p in prix}
# → {412, 190, 119} (l'ordre d'affichage d'un set peut varier)

```

Trois structures, trois paires de délimiteurs, un seul schéma mental. La « surprise » annoncée : `round(189.5)` → 190... qui n'en est pas une si tu te souviens de l'arrondi bancaire (chapitre 3) — 189.5 est à mi-distance exacte, l'entier pair le plus proche est 190. (Avec 188.5, on aurait obtenu 188 !) `round(118.7)` → 119 et `round(412.3)` → 412, sans subtilité.

Solution 14.4 —

```

import math
r = [0.01, -0.02, 0.03, 0.005, -0.01]

positifs = [x for x in r if x > 0]           # → [0.01, 0.03, 0.005]
bruts = [1 + x for x in r]                  # → [1.01, 0.98, 1.03, 1.005, 0.99]
cumule = math.prod(1 + x for x in r)        # → 1.0143459...
par_mois = {m: x for m, x in enumerate(r, start=1)}
# → {1: 0.01, 2: -0.02, 3: 0.03, 4: 0.005, 5: -0.01}
print(round(cumule, 6))                     # → 1.014346

```

Le 3) est l'usage-roi de l'expression génératrice : `math.prod` consomme les facteurs au fil de l'eau, aucune liste intermédiaire. (Performance cumulée $\approx +1,43\%$ sur les 5 mois — et non la somme naïve des rendements, $+1,5\%$: les produits ne commutent pas avec les sommes, le composé est légèrement rogné par la volatilité.)

Solution 14.5 —

```

A = [[1, 2, 3], [4, 5, 6]]

double = [[2 * x for x in ligne] for ligne in A]
# → [[2, 4, 6], [8, 10, 12]]

aplatie = [x for ligne in A for x in ligne]
# → [1, 2, 3, 4, 5, 6]

# transposée – d'abord la boucle :
T = []
for j in range(len(A[0])):           # pour chaque colonne j...
    colonne = []
    for ligne in A:                  # ...collecter A[i][j] sur toutes les lignes
        colonne.append(ligne[j])
    T.append(colonne)

# puis la compréhension :

```

```
T = [[ligne[j] for ligne in A] for j in range(len(A[0]))]
# → [[1, 4], [2, 5], [3, 6]]
```

Gymnastique à bien voir : dans `aplatie`, les `for` se lisent **de gauche à droite comme la boucle imbriquée** (`for ligne in A` externe, `for x in ligne` interne). Dans la transposée, en revanche, la structure est *une compréhension DANS une compréhension* : l'externe construit les nouvelles lignes (indexées par `j`), l'interne parcourt `A`. Deux constructions différentes qui se ressemblent — quand ça devient dur à relire... boucle explicite, comme le préconise la section 14.8. (En pratique matricielle réelle : NumPy, `A.T` — mais savoir le faire à la main, c'est comprendre l'indexation.)

Fiche récap — Chapitre 14

Syntaxe	Construit
<code>[expr for x in it]</code>	liste — expr appliquée à chaque élément
<code>[expr for x in it if cond]</code>	liste filtrée ($f(x) \mid x \in E, P(x)$)
<code>[expr for x in it1 for y in it2]</code>	double boucle (externe $\rightarrow$ interne, même ordre qu'imbriquées)
<code>{cle: val for ... }</code>	dictionnaire (accolades + <code>:</code>)
<code>{expr for ... }</code>	set (accolades sans <code>:</code>)
<code>(expr for ...)</code>	générateur : paresseux, mémoire constante, consommation unique
<code>sum(expr for ...)</code>	agrégation directe sans liste intermédiaire
<code>zip(a, b)</code>	appariement 1-à-1 en tuples (s'arrête à la plus courte)
vs <code>map</code> / <code>filter</code> / <code>lambda</code>	compréhension plus lisible « 99 % du temps »
Noms	l'expression n'emploie que les variables des <code>for</code>
Limite	illisible (trop imbriquée) $\Rightarrow$ revenir à la boucle

CHAPITRE 15

CSV : lire et écrire des données tabulaires

Vidéo source : [Python Tutorial: CSV Module - How to Read, Parse, and Write CSV Files](#) — Corey Schafer

Objectifs : lire et écrire des fichiers CSV avec le module `csv`, gérer en-têtes et délimiteurs, et passer aux versions dictionnaire (`DictReader` / `DictWriter`), plus lisibles et plus robustes.

15.1 Le format CSV

CSV = *comma separated values* : du texte brut où chaque ligne est un enregistrement et chaque champ est séparé par une virgule. Première ligne (usuelle) : l'**en-tête** avec les noms de colonnes. C'est LE format d'échange tabulaire universel — exports bancaires, données de marché, Excel... Fichier d'exemple du chapitre, `names.csv` :

```
first_name,last_name,email
John,Doe,john-doe@bogusemail.com
Mary,Smith-Robinson,maryjacobs@bogusemail.com
Dave,Smith,davesmith@bogusemail.com
```

Pourquoi un module dédié ? On serait tenté de lire le fichier (chapitre 12) et de `split(',')` chaque ligne. Ça casse au premier champ *contenant* une virgule (`"Paris, France"`), aux champs entre guillemets, aux guillemets échappés... Le module `csv` de la `stdlib` gère tous ces cas — Corey est formel : ne parse jamais du CSV à la main.

15.2 Lire : `csv.reader`

```
import csv

with open('names.csv', 'r') as csv_file:
    csv_reader = csv.reader(csv_file)

    for line in csv_reader:
        print(line)
# → ['first_name', 'last_name', 'email']
# → ['John', 'Doe', 'john-doe@bogusemail.com']
# → ['Mary', 'Smith-Robinson', 'maryjacobs@bogusemail.com']
# → ['Dave', 'Smith', 'davesmith@bogusemail.com']
```

Tout le savoir des chapitres précédents s'assemble : `with open` (ch. 12), itération (ch. 7), et chaque ligne arrive en **liste** (ch. 4). L'accès aux champs se fait par index — l'email est en position 2 :

```
with open('names.csv', 'r') as csv_file:
    csv_reader = csv.reader(csv_file)

    for line in csv_reader:
        print(line[2])
```

```
# → email
# → john-doe@bogusemail.com
# → maryjacobs@bogusemail.com
# → davesmith@bogusemail.com
```

Petit défaut : la ligne d'en-tête (`email`) est traitée comme une donnée. Pour la **sauter**, on avance l'itérateur d'un cran avec `next()` avant la boucle :

```
with open('names.csv', 'r') as csv_file:
    csv_reader = csv.reader(csv_file)

    next(csv_reader)                # consomme la ligne d'en-tête

    for line in csv_reader:
        print(line[2])
# → john-doe@bogusemail.com
# → maryjacobs@bogusemail.com
# → davesmith@bogusemail.com
```

(`next()`) fait avancer n'importe quel itérateur — même mécanique que le curseur de fichier du chapitre 12.)

15.3 Écrire : `csv.writer`

Recopions le fichier en changeant le délimiteur — un lecteur ouvert, un écrivain ouvert, et `writerow` ligne à ligne :

```
with open('names.csv', 'r') as csv_file:
    csv_reader = csv.reader(csv_file)

    with open('new_names.csv', 'w', newline='') as new_file:
        csv_writer = csv.writer(new_file, delimiter='\t')

        for line in csv_reader:
            csv_writer.writerow(line)
```

`new_names.csv` contient désormais les champs séparés par des **tabulations** (`'\t'`).

Piège Windows (à ne jamais oublier) — Le `newline=''` dans l'`open` d'écriture : sans lui, sous Windows, chaque enregistrement est suivi d'une **ligne vide** (double saut de ligne). La vidéo, tournée sur Mac, ne le montre pas — la doc officielle du module l'impose. Symptôme classique « une ligne sur deux est vide » ⇒ c'est ça.

Pourquoi changer de délimiteur ? Démonstration lumineuse de la vidéo avec le tiret : `delimiter='-'` pose problème car `Smith-Robinson` **contient** un tiret. Regarde comme le module s'en sort :

```
first_name-last_name-email
John-Doe-john-doe@bogusemail.com
```

```
Mary-"Smith-Robinson"-maryjacobs@bogusemail.com
...
```

Il a mis les guillemets tout seul autour de "Smith-Robinson" pour que la relecture ne coupe pas ce champ en deux. C'est exactement le genre de subtilité qu'un `split('-')` maison raterait — la raison d'être du module.

Relire un fichier à délimiteur spécial : il faut le préciser aussi en lecture. Oublions-le d'abord (l'erreur est instructive) :

```
with open('new_names.csv', 'r') as csv_file:
    csv_reader = csv.reader(csv_file)          # délimiteur virgule par défaut !
    for line in csv_reader:
        print(line)
# → ['first_name\tlast_name\temail']          ← une liste à UN seul élément par ligne...
```

Le module cherchait des virgules, n'en a pas trouvés : chaque ligne entière devient un unique champ. Correction : `csv.reader(csv_file, delimiter='\t')` — et tout rentre dans l'ordre.

15.4 La version dictionnaire : DictReader

Accéder à l'email par `line[2]` fonctionne, mais tu relis le code dans un mois : que signifie `2` ? La solution moderne du module : **DictReader**, qui utilise automatiquement l'en-tête comme clés et livre chaque ligne en **dictionnaire** :

```
with open('names.csv', 'r') as csv_file:
    csv_reader = csv.DictReader(csv_file)

    for line in csv_reader:
        print(line['email'])
# → john-doe@bogusemail.com
# → maryjacobs@bogusemail.com
# → davesmith@bogusemail.com
```

Trois gains d'un coup : plus de `next()` (l'en-tête est consommé comme noms de colonnes), un code **auto-documenté** (`line['email']` dit ce qu'il fait), et l'insensibilité à l'ordre des colonnes (si quelqu'un insère une colonne, `line[2]` casse en silence, `line['email']` continue de viser juste).

Depuis la vidéo — La vidéo affiche des `OrderedDict(...)` : c'était le type renvoyé à l'époque. Depuis Python 3.8, **DictReader** renvoie des **dict ordinaires** (devenus ordonnés en 3.7 — chapitre 5) : `{'first_name': 'John', 'last_name': 'Doe', 'email': 'john-doe@bogusemail.com'}`.

15.5 DictWriter : écrire par dictionnaires

L'écrivain symétrique exige un paramètre : **fieldnames**, la liste des colonnes (leur ordre dans le fichier de sortie), et `writeheader()` écrit la ligne d'en-tête :

```

with open('names.csv', 'r') as csv_file:
    csv_reader = csv.DictReader(csv_file)

    with open('new_names.csv', 'w', newline='') as new_file:
        fieldnames = ['first_name', 'last_name', 'email']
        csv_writer = csv.DictWriter(new_file, fieldnames=fieldnames, delimiter='\t')

        csv_writer.writeheader()

        for line in csv_reader:
            csv_writer.writerow(line)

```

Le cas d'usage roi : transformer à la volée. Exemple de la vidéo — recopier le fichier **sans** la colonne email :

```

with open('names.csv', 'r') as csv_file:
    csv_reader = csv.DictReader(csv_file)

    with open('new_names.csv', 'w', newline='') as new_file:
        fieldnames = ['first_name', 'last_name']          # email exclu

        csv_writer = csv.DictWriter(new_file, fieldnames=fieldnames)
        csv_writer.writeheader()

        for line in csv_reader:
            del line['email']                             # retirer la clé (chapitre 5)
            csv_writer.writerow(line)

```

`new_names.csv` :

```

first_name,last_name
John,Doe
Mary,Smith-Robinson
Dave,Smith

```

(Sans le `del`, `DictWriter` leverait `ValueError: dict contains fields not in fieldnames: 'email'` — il refuse les clés non déclarées, sauf à lui passer `extrasaction='ignore'`. À l'inverse, une clé *manquante* est écrite comme champ vide.)

Piège de données — Tout ce que lit le module csv est **du texte** : `line['prix']` vaut `'189.5'` (une chaîne !). Toute arithmétique exige la conversion du chapitre 3 : `float(line['prix'])`. L'oubli se manifeste par des concaténations farfelues ou un `TypeError`.

Pour aller plus loin (ta future boîte à outils) — Pour *analyser* des données tabulaires (statistiques, jointures, séries temporelles), l'outil standard est **pandas** (`pip install pandas`) : `pd.read_csv('names.csv')` charge le fichier en DataFrame typé. Le module `csv` reste l'outil léger et sans dépendance des scripts d'ingestion/transformation — et comprendre son fonctionnement t'expliquera ce que pandas fait sous le capot.

Exercices

Fichier de travail des exercices — crée `cotations.csv` :

```
ticker,prix,quantite
AAPL,189.5,12
MSFT,412.3,4
NVDA,118.7,25
AIR.PA,142.8,10
```

Exercice 15.1 ● — Avec `csv.reader` : affiche chaque ligne (liste), puis relance en sautant l'en-tête et n'affiche que les tickers.

Exercice 15.2 ● — Avec `DictReader` : affiche `AAPL : 189.5 USD` pour chaque ligne (f-string). Pourquoi n'a-t-on pas eu besoin de `next()` ?

Exercice 15.3 ●● — Calcule la **valeur totale du portefeuille** ($\sum \text{prix} \times \text{quantité}$) avec `DictReader`. Attendu : `8318.70` (attention au piège de la section 15.5 !). Affiche « Valeur du portefeuille : 8318.70 € ».

Exercice 15.4 ●● — Produis `portefeuille_valorise.csv` : mêmes colonnes plus une colonne `valeur` (prix × quantité, arrondie à 2 décimales), séparateur **point-virgule** (le CSV « à la française » qu'Excel FR ouvre bien). Relis-le pour vérifier — sans oublier le délimiteur à la relecture.

Exercice 15.5 ●●● — Ton courtier exporte `ordres.csv` avec les colonnes `date,ticker,sens,montant` ; certaines lignes ont un montant **vide** (ordre annulé). Écris un script qui : ignore les lignes à montant vide (truthiness !), agrège par ticker le montant net (achats **A** positifs, ventes **V** négatifs — motif du chapitre 5), et écrit `net_par_ticker.csv` (colonnes `ticker,net`) trié par ticker. Données de test :

```
date,ticker,sens,montant
2026-01-05,AAPL,A,1000
2026-01-06,MSFT,A,2000
2026-01-07,AAPL,V,400
2026-01-08,AAPL,A,
2026-01-09,MSFT,V,500
2026-01-10,AAPL,A,250
```

Solutions détaillées

Solution 15.1 —

```
import csv

with open('cotations.csv', 'r') as f:
    reader = csv.reader(f)
    for line in reader:
        print(line)
# → ['ticker', 'prix', 'quantite'] puis une liste par ligne de données

with open('cotations.csv', 'r') as f:
    reader = csv.reader(f)
    next(reader)
    for line in reader:
        print(line[0])
# → AAPL  MSFT  NVDA  AIR.PA  (un par ligne)
```

Solution 15.2 —

```
with open('cotations.csv', 'r') as f:
    reader = csv.DictReader(f)
    for line in reader:
        print(f"{line['ticker']} : {line['prix']} USD")
```

`DictReader` consomme lui-même la première ligne pour en faire ses clés — l'en-tête ne réapparaît donc jamais comme donnée.

Solution 15.3 —

```
with open('cotations.csv', 'r') as f:
    reader = csv.DictReader(f)
    total = 0
    for line in reader:
        total += float(line['prix']) * int(line['quantite'])

print(f'Valeur du portefeuille : {total:.2f} €')
# → Valeur du portefeuille : 8318.70 €
```

Le double casting est le cœur de l'exercice : sans lui, `'189.5' * '12'` lève `TypeError` (on ne multiplie pas deux chaînes). Détail : `float × int → float`, parfait ici.

Solution 15.4 —

```
with open('cotations.csv', 'r') as f:
    reader = csv.DictReader(f)

    with open('portefeuille_valorise.csv', 'w', newline='') as out:
```

```

fieldnames = ['ticker', 'prix', 'quantite', 'valeur']
writer = csv.DictWriter(out, fieldnames=fieldnames, delimiter=';')
writer.writeheader()

for line in reader:
    line['valeur'] = round(float(line['prix']) * int(line['quantite']), 2)
    writer.writerow(line)

# relecture de contrôle :
with open('portefeuille_valorise.csv', 'r') as f:
    for line in csv.DictReader(f, delimiter=';'):
        print(line['ticker'], line['valeur'])
# → AAPL 2274.0 / MSFT 1649.2 / NVDA 2967.5 / AIR.PA 1428.0

```

On enrichit le dict lu d'une clé `valeur` (chapitre 5), déclarée dans `fieldnames`. Et le délimiteur se précise **aux deux bouts** — l'oubli en relecture reproduit l'erreur « liste à un élément » de la section 15.3.

Solution 15.5 —

```

import csv

net = {}
with open('ordres.csv', 'r') as f:
    for line in csv.DictReader(f):
        if not line['montant']:
            continue
        montant = float(line['montant'])
        if line['sens'] == 'V':
            montant = -montant
        net[line['ticker']] = net.get(line['ticker'], 0) + montant

with open('net_par_ticker.csv', 'w', newline='') as out:
    writer = csv.writer(out)
    writer.writerow(['ticker', 'net'])
    for ticker in sorted(net):
        writer.writerow([ticker, net[ticker]])

# net_par_ticker.csv :
# ticker,net
# AAPL,850.0
# MSFT,1500.0

```

Récapitulatif express de la moitié du cours : `continue` sur champ vide (ch. 7 + 6), agrégation `get(k, 0)` (ch. 5), tri des clés (ch. 4), écriture csv. AAPL : $1000 - 400 + 250 = 850$ ✓ ; MSFT : $2000 - 500 = 1500$ ✓ — la ligne au montant vide n'a rien cassé.

Fiche récap — Chapitre 15

Syntaxe	Effet
<code>import csv</code>	module <code>stdlib</code> — jamais de <code>split(',')</code> maison
<code>csv.reader(f)</code>	itérateur de listes ; champs par index
<code>next(reader)</code>	saute l'en-tête (avance l'itérateur d'un cran)
<code>csv.writer(f, delimiter='\\t') + writerow(liste)</code>	écriture ; délimiteur au choix ; guillemets automatiques si un champ contient le délimiteur
<code>open(..., 'w', newline='')</code>	obligatoire sous Windows en écriture CSV (sinon lignes vides)
Délimiteur en lecture	à re-préciser (<code>delimiter='\\t'</code>), sinon « liste à 1 élément »
<code>csv.DictReader(f)</code>	itérateur de dicts (clés = en-tête, auto-consommé) — lisible et robuste
<code>csv.DictWriter(f, fieldnames=[...])</code>	+ <code>writeheader()</code> , <code>writerow(dict)</code> ; clés non déclarées → <code>ValueError</code>
Types	tout champ lu est une chaîne → <code>float()</code> / <code>int()</code> avant calcul
Analyse lourde	→ pandas (<code>pd.read_csv</code>) ; <code>csv</code> pour les scripts légers

CHAPITRE 16

JSON : données structurées et APIs

Vidéo source : [Python Tutorial: Working with JSON Data using the json Module](#) — Corey Schafer

Objectifs : convertir du JSON en objets Python et inversement (`loads` / `dumps`), lire et écrire des fichiers JSON (`load` / `dump`), et exploiter une réponse d'API réelle.

16.1 Le format JSON

JSON (*JavaScript Object Notation*) est LE format d'échange de données structurées : réponses d'APIs web, fichiers de configuration, stockage léger. Né de JavaScript, il est aujourd'hui indépendant de tout langage — et sa syntaxe te sera étrangement familière : elle ressemble énormément aux dictionnaires et listes Python.

Le module de la stdlib s'appelle `json`. Exemple de la vidéo — une chaîne JSON décrivant des personnes :

```
import json

people_string = '''
{
    "people": [
        {
            "name": "John Smith",
            "phone": "615-555-7164",
            "emails": ["johnsmith@bogusemail.com", "john.smith@work-place.com"],
            "has_license": false
        },
        {
            "name": "Jane Doe",
            "phone": "560-555-5153",
            "emails": null,
            "has_license": true
        }
    ]
}
'''
```

(Triple quotes du chapitre 2 pour la chaîne multi-lignes.) Différences visibles avec Python : guillemets **doubles obligatoires** , `true` / `false` en minuscules, `null` au lieu de `None`.

16.2 JSON → Python : `json.loads`

```
data = json.loads(people_string)
print(type(data))    # → <class 'dict'>
```

`loads` (*load string*) **parse** la chaîne et construit les objets Python correspondants, selon cette table de conversion :

JSON	Python
object <code>{}</code>	<code>dict</code>
array <code>[]</code>	<code>list</code>
string	<code>str</code>
number (entier)	<code>int</code>
number (réel)	<code>float</code>
<code>true</code> / <code>false</code>	<code>True</code> / <code>False</code>
<code>null</code>	<code>None</code>

La preuve dans les données : `has_license: false` devient `False` (majuscule), `emails: null` devient `None`. Et ensuite... tout est du Python standard, que tu sais déjà manipuler depuis les chapitres 4-7 :

```
print(type(data['people']))    # → <class 'list'>

for person in data['people']:
    print(person['name'])
# → John Smith
# → Jane Doe
```

Naviguer dans du JSON = enchaîner accès par clé (dicts) et boucles/index (listes). Rien de plus.

16.3 Python → JSON : `json.dumps`

Sens inverse : sérialiser un objet Python en chaîne JSON. Cas pratique de la vidéo — retirer les numéros de téléphone avant de réexporter :

```
for person in data['people']:
    del person['phone']

new_string = json.dumps(data)
print(new_string)
# → {"people": [{"name": "John Smith", "emails": [...], "has_license": false}, ...]}
```

Compact mais illisible. Deux paramètres arrangent tout :

```
new_string = json.dumps(data, indent=2)
print(new_string)
# → {
#     "people": [
#         {
#             "name": "John Smith",
#             ...
```

`indent=2` indente joliment (pour les humains) ; `sort_keys=True` trie les clés alphabétiquement (pour des sorties stables et comparables) :

```
new_string = json.dumps(data, indent=2, sort_keys=True)
```

Piège francophone — Par défaut, `dumps` échappe les non-ASCII : "é" devient "é". C'est du JSON valide mais illisible. Ajoute `ensure_ascii=False` pour garder les accents : `json.dumps(data, indent=2, ensure_ascii=False)`.

16.4 Fichiers JSON : `load` et `dump` (sans « s »)

Mnémotechnique donnée par le module lui-même : `loads / dumps = String` ; `load / dump = fichier`. Fichier d'exemple `states.json` :

```
{
  "states": [
    {"name": "Alabama", "abbreviation": "AL", "area_codes": ["205", "251", "256"]},
    {"name": "Alaska", "abbreviation": "AK", "area_codes": ["907"]}
  ]
}
```

Lire un fichier JSON — `open` du chapitre 12 + `json.load` :

```
with open('states.json', encoding='utf-8') as f:
    data = json.load(f)

for state in data['states']:
    print(state['name'], state['abbreviation'])
# → Alabama AL
# → Alaska AK
```

Écrire un fichier JSON — transformer puis `json.dump(objet, fichier)` :

```
for state in data['states']:
    del state['area_codes']

with open('new_states.json', 'w', encoding='utf-8') as f:
    json.dump(data, f, indent=2)
```

`new_states.json` contient la version épurée, indentée. Le JSON est ainsi le moyen le plus simple de **persister** l'état d'un programme (configs, caches, résultats intermédiaires) : `dump` à la fin, `load` au démarrage.

16.5 Cas réel : une réponse d'API

Le clou de la vidéo : consommer une **API web** — Corey interroge une API Yahoo Finance de taux de change et la parse en direct. L'API précise de la vidéo a disparu depuis, mais le **schéma de travail est éternel**, le voici étape par étape :

1. **Récupérer** la réponse (la vidéo utilise `urlopen` de la `stdlib`) :

```
from urllib.request import urlopen
import json

with urlopen('https://api.exemple.com/taux') as response:
    source = response.read()

data = json.loads(source)
```

1. **Explorer** la structure — le réflexe clé quand on découvre une API : tout ré-afficher indenté pour *voir* l'arborescence :

```
print(json.dumps(data, indent=2))
```

1. **Naviguer** jusqu'aux données utiles et les **réorganiser** pour son besoin. Dans la vidéo : construire un dictionnaire `{nom_devise: taux}` pour des conversions directes. Simulons la même logique sur une réponse type :

```
data = json.loads('''
{
    "base": "USD",
    "list": {
        "resources": [
            {"resource": {"fields": {"name": "USD/EUR", "price": "0.9204"}}},
            {"resource": {"fields": {"name": "USD/GBP", "price": "0.7823"}}},
            {"resource": {"fields": {"name": "USD/JPY", "price": "147.11"}}}
        ]
    }
}
''')
```

```
usd_rates = {}
for item in data['list']['resources']:
    fields = item['resource']['fields']
    usd_rates[fields['name']] = float(fields['price'])

print(usd_rates)
# → {'USD/EUR': 0.9204, 'USD/GBP': 0.7823, 'USD/JPY': 147.11}
```

```
montant_usd = 50
print(f"{montant_usd} USD = {montant_usd * usd_rates['USD/EUR']:.2f} EUR")
# → 50 USD = 46.02 EUR
```


Note le `float(...)` : cette API (comme beaucoup) livre les prix **en chaînes** — le piège des types, encore lui (chapitres 3 et 15). Et remarque la gymnastique `data['list']['resources'][i]['resource']['fields']` : les APIs réelles sont profondes ; on descend étage par étage, en s'aidant du `dumps(indent=2)` de l'étape 2.

Idee d'application donnée par Corey, très saine : un script qui télécharge les taux une fois par jour, les `dump` dans un fichier local, que le reste du programme `load` — on épargne l'API et on gagne en vitesse.

Depuis la vidéo — Pour requêter des APIs en 2026, tout le monde utilise le paquet tiers **requests** (`pip install requests` — chapitre 10) plutôt qu'`urllib` : `r = requests.get(url)` puis `data = r.json()` (le parsing JSON est même intégré !). Corey a d'ailleurs une vidéo dédiée à **requests**. Pour les données de marché : les APIs sérieuses (et souvent payantes) type Alpha Vantage, Twelve Data, ou le paquet `yfinance`, répondent toutes... en JSON. Ce chapitre est ta clé d'entrée.

Exercices

Exercice 16.1 ● — Donne, sans exécuter, le type Python de chaque valeur après `json.loads` : `{"a": 1, "b": 2.5, "c": "2.5", "d": [1, 2], "e": {"x": null}, "f": true}`. Vérifie ensuite avec une boucle sur `.items()` et `type()`.

Exercice 16.2 ● — Construis en Python le dictionnaire `profil` : ton prénom, ta ville, une liste `objectifs` (2-3 chaînes), et `etudiant` (booléen). Écris-le dans `profil.json` (indenté, accents préservés), puis relis le fichier et affiche le premier objectif.

Exercice 16.3 ●● — Le fichier `config.json` contient `{"seuil": 0.05, "actifs": ["AAPL", "MSFT"], "verbose": true}`. Écris un script qui charge la config, ajoute `"NVDA"` aux actifs, passe `verbose` à `false`, et réécrit le fichier **sans perdre l'indentation**. Pourquoi ce trio `load`→`modifier`→`dump` est-il si courant ?

Exercice 16.4 ●● — Une API de cotations renvoie : `'{"quotes": [{"symbol": "AAPL", "price": "189.50"}, {"symbol": "MSFT", "price": "412.30"}, {"symbol": "NVDA", "price": null}]}'`. Construis le dict `{symbol: prix_en_float}` en **ignorant** les cotations à prix `null` (chapitre 6 : quel test exactement ?). Attendu : `{'AAPL': 189.5, 'MSFT': 412.3}`.

Exercice 16.5 ●●● — Aller-retour complet : reprends `cotations.csv` du chapitre 15 et convertis-le en `cotations.json` de la forme `{"positions": [{"ticker": ..., "prix": ..., "quantite": ..., "valeur": ...}, ...]}` avec les types **numériques** corrects (prix float, quantité int, valeur calculée) — puis recharge le JSON et vérifie que la somme des valeurs retombe sur les 8318.70 € du chapitre 15. (Tu viens d'écrire un convertisseur CSV→JSON : un vrai petit outil.)

Solutions détaillées

Solution 16.1 —

```
import json

data = json.loads('{ "a": 1, "b": 2.5, "c": "2.5", "d": [1, 2], "e": {"x": null}, "f": true }')
for key, value in data.items():
    print(key, type(value))
# → a <class 'int'>
# → b <class 'float'>
# → c <class 'str'>          ← "2.5" entre guillemets : chaîne !
# → d <class 'list'>
# → e <class 'dict'>        (et data['e']['x'] est None)
# → f <class 'bool'>
```

Le piège est `c` : des guillemets dans le JSON ⇒ chaîne en Python, même si le contenu « ressemble » à un nombre.

Solution 16.2 —

```
import json

profil = {
    'prenom': 'Mathias',
    'ville': 'Paris',
    'objectifs': ['finance quantitative', 'optimisation', 'automatiser mes analyses'],
    'etudiant': True,
}

with open('profil.json', 'w', encoding='utf-8') as f:
    json.dump(profil, f, indent=2, ensure_ascii=False)

with open('profil.json', encoding='utf-8') as f:
    relu = json.load(f)

print(relu['objectifs'][0])  # → finance quantitative
```

`ensure_ascii=False` + `encoding='utf-8'` : le duo qui préserve les accents à l'écriture ET à la lecture.

Solution 16.3 —

```
import json

with open('config.json', encoding='utf-8') as f:
    config = json.load(f)

config['actifs'].append('NVDA')
config['verbose'] = False

with open('config.json', 'w', encoding='utf-8') as f:
    json.dump(config, f, indent=2, ensure_ascii=False)
```

Le trio est le **cycle de vie de tout état persistant** : un programme qui doit se souvenir de quelque chose entre deux exécutions (préférences, avancement, cache) charge au démarrage, travaille en mémoire sur des dicts/listes ordinaires, sauvegarde en sortie. JSON rend ce cycle trivial — c'est exactement pour ça qu'il est partout.

Solution 16.4 —

```
import json

reponse = '{"quotes": [{"symbol": "AAPL", "price": "189.50"}, {"symbol": "MSFT", "price": "412.30"}, {"symbol": "NVDA", "price": null}]}'
data = json.loads(reponse)

prix = {}
for quote in data['quotes']:
    if quote['price'] is None:
        continue
    prix[quote['symbol']] = float(quote['price'])

print(prix)    # → {'AAPL': 189.5, 'MSFT': 412.3}
```

Le bon test est `is None` (chapitre 6) : `null` JSON → `None` Python. Un `if not quote['price']` marcherait ici mais éliminerait aussi un prix de `0` — pour un actif, discutable ; pour un taux, dangereux. On teste *précisément* ce qu'on veut exclure.

Solution 16.5 —

```
import csv
import json

positions = []
with open('cotations.csv') as f:
    for line in csv.DictReader(f):
        prix = float(line['prix'])
        quantite = int(line['quantite'])
        positions.append({
            'ticker': line['ticker'],
            'prix': prix,
            'quantite': quantite,
            'valeur': round(prix * quantite, 2),
        })

with open('cotations.json', 'w', encoding='utf-8') as f:
    json.dump({'positions': positions}, f, indent=2, ensure_ascii=False)

# contrôle :
with open('cotations.json', encoding='utf-8') as f:
    data = json.load(f)

total = sum(p['valeur'] for p in data['positions'])
print(f'{total:.2f}')    # → 8318.70
```

La conversion des types se fait à l'**entrée** (côté CSV, où tout est chaîne) ; le JSON, lui, sait stocker de vrais nombres — c'est sa supériorité structurelle sur le CSV, avec l'imbrication. La vérification par somme (`sum` + expression génératrice, chapitre 14) boucle la boucle avec le chapitre 15.

Fiche récap — Chapitre 16

Syntaxe	Effet
<code>json.loads(chaîne)</code>	JSON → objets Python (<code>s = string</code>)
<code>json.dumps(obj)</code>	Python → chaîne JSON ; <code>indent=2</code> , <code>sort_keys=True</code> , <code>ensure_ascii=False</code>
<code>json.load(f)</code> / <code>json.dump(obj, f)</code>	idem, directement sur un fichier ouvert
Conversions	object→dict, array→list, string→str, number→int/float, true/false→True/False, null→None
Explorer une API	<code>print(json.dumps(data, indent=2))</code> puis descendre clé par clé
Types piégeux	nombres livrés en chaînes → <code>float()</code> ; prix <code>null</code> → tester <code>is None</code>
Persistence	load au démarrage → travailler en mémoire → dump en sortie
APIs en pratique	paquet <code>requests</code> (<code>r.json()</code>) ; réponses profondes = navigation dict/list

CHAPITRE 17

Dates et heures : le module datetime

Vidéo source : [Python Tutorial: Datetime Module - How to work with Dates, Times, Timedeltas, and Timezones](#) — Corey Schafer

Objectifs : manier les quatre objets `date`, `time`, `datetime` et `timedelta`, faire de l'arithmétique temporelle, formater (`strftime`) et parser (`strptime`) des dates, et travailler proprement avec les fuseaux horaires.

17.1 La carte du module

`datetime` (stdlib) fournit plusieurs classes, à ne pas confondre :

Classe	Contient	Exemple
<code>datetime.date</code>	année, mois, jour	2026-08-24
<code>datetime.time</code>	heure, minute, seconde, μ s	12:30:45
<code>datetime.datetime</code>	les deux	2026-08-24 12:30:45
<code>datetime.timedelta</code>	une durée	7 jours

Et une distinction transversale : un objet temporel est **naïf** (*naive*, sans fuseau horaire) ou **conscient** (*aware*, avec fuseau et heure d'été). Le naïf est plus simple et suffit souvent ; l'aware est sans ambiguïté — indispensable dès que plusieurs fuseaux entrent en jeu.

17.2 Les dates : `datetime.date`

```
import datetime

d = datetime.date(2016, 7, 24)
print(d)      # → 2016-07-24
```

Piège immédiat — Pas de zéros de tête : `datetime.date(2016, 07, 24)` est une `SyntaxError` (en Python, un littéral commençant par `0` est interdit — héritage de la notation octale). On écrit `7`, pas `07`.

La date du jour et ses composantes :

```
today = datetime.date.today()
print(today)      # → 2026-08-24
print(today.year)  # → 2026
```

```
print(today.month)    # → 8
print(today.day)      # → 24
```

Le jour de la semaine, avec deux conventions coexistantes :

```
print(today.weekday())    # → 0    lundi=0 ... dimanche=6
print(today.isoweekday()) # → 1    lundi=1 ... dimanche=7 (norme ISO 8601)
```

(Le 24 août 2026 est un lundi.) Retenir le duo évite une classe entière de bugs « décalés d'un jour ».

17.3 Les durées : `timedelta`, et l'arithmétique des dates

Un `timedelta` représente un **écart de temps** ; il se combine algébriquement avec les dates :

```
tdelta = datetime.timedelta(days=7)

print(today + tdelta)    # → 2026-08-31    (dans une semaine)
print(today - tdelta)    # → 2026-08-17    (il y a une semaine)
```

L'algèbre à connaître — élégante et cohérente :

- `date + timedelta` → `date`
- `date - timedelta` → `date`
- `date - date` → `timedelta`

Le troisième cas est le plus utile — exemple de la vidéo, les jours jusqu'à un événement :

```
bday = datetime.date(2026, 9, 24)
till_bday = bday - today

print(till_bday)          # → 31 days, 0:00:00
print(till_bday.days)     # → 31
print(till_bday.total_seconds()) # → 2678400.0
```

`days` est un attribut (les jours entiers), `total_seconds()` une méthode (la durée totale convertie en secondes).

`timedelta` accepte aussi `hours=`, `minutes=`, `seconds=`, `weeks=`... et les dépassements se propagent tout seuls (ajouter 12 h à 20 h bascule au lendemain).

Pour aller plus loin (finance) — `date - date` → `timedelta` rend triviaux les calculs de maturité : `(echeance - today).days / 365` te donne une durée en années pour actualiser. Les conventions de décompte réelles (30/360, ACT/ACT...) se codent par-dessus — mais la brique de base est là.

17.4 `datetime.time` et surtout `datetime.datetime`

`time` seul (heure sans date) sert rarement :

```
t = datetime.time(9, 30, 45, 100000)
print(t)          # → 09:30:45.100000
print(t.hour)     # → 9
```

Le vrai cheval de bataille est `datetime.datetime` — date + heure, tous les attributs, toute l'arithmétique :

```
dt = datetime.datetime(2026, 8, 24, 12, 30, 45, 100000)
print(dt)          # → 2026-08-24 12:30:45.100000
print(dt.date())   # → 2026-08-24
print(dt.time())   # → 12:30:45.100000
print(dt.year)     # → 2026

print(dt + datetime.timedelta(hours=12))
# → 2026-08-25 00:30:45.100000      (basculer de jour automatique)
```

Trois façons d'obtenir « maintenant », aux nuances près :

```
print(datetime.datetime.today())    # heure locale, naïf
print(datetime.datetime.now())      # heure locale, naïf — mais accepte un fuseau (voir 17.5)
print(datetime.datetime.utcnow())   # heure UTC... mais NAÏF quand même (tzinfo=None)
```

Piège — `utcnow()` donne les chiffres de l'heure UTC sans marquer l'objet comme UTC : il reste naïf, et se mélange mal avec le reste. La vidéo le signale déjà ; Python 3.12 l'a carrément **déprécié**. La bonne forme moderne :

`datetime.datetime.now(tz=...)` — section suivante.

17.5 Les fuseaux horaires

Le principe cardinal, martelé par Corey : **travailler en UTC en interne** (stockage, calculs, logs), et **convertir vers le fuseau local uniquement à l'affichage**. UTC est le temps de référence sans heure d'été : c'est lui qui évite les heures dupliquées/inexistantes des changements d'heure.

Depuis la vidéo — Corey utilise `pytz`, la bibliothèque tierce recommandée à l'époque par la doc Python elle-même. Depuis **Python 3.9**, la stdlib intègre `zoneinfo`, qui rend `pytz` inutile pour les nouveaux projets. Le cours montre la voie moderne (les concepts de la vidéo — UTC, aware/naïf, conversion — sont inchangés ; seule l'orthographe diffère : `pytz.timezone('US/Mountain')` → `ZoneInfo('America/Denver')`, et le `localize()` de `pytz` → `replace(tzinfo=...)`).

```
from zoneinfo import ZoneInfo
import datetime

# maintenant, en UTC, AWARE (la bonne façon) :
dt_utc = datetime.datetime.now(tz=datetime.timezone.utc)
print(dt_utc)    # → 2026-08-24 10:00:00.123456+00:00
```

Le suffixe `+00:00` est la signature d'un objet *aware* : son décalage par rapport à UTC est connu. **Convertir** vers un fuseau d'affichage — `astimezone` :

```
dt_paris = dt_utc.astimezone(ZoneInfo('Europe/Paris'))
print(dt_paris)    # → 2026-08-24 12:00:00.123456+02:00    (heure d'été : UTC+2)

dt_ny = dt_utc.astimezone(ZoneInfo('America/New_York'))
print(dt_ny)       # → 2026-08-24 06:00:00.123456-04:00
```

Même instant, trois habillages — c'est exactement ça, un fuseau. Les noms de zones (`'Europe/Paris'`, `'America/New_York'`, `'Asia/Tokyo'`...) viennent de la base IANA ; `zoneinfo.available_timezones()` les liste (l'équivalent du `pytz.all_timezones` de la vidéo).

Rendre aware un naïf (on sait par ailleurs quelle heure il représente) :

```
dt_naive = datetime.datetime(2026, 8, 24, 9, 30)           # 9h30... où ?
dt_aware = dt_naive.replace(tzinfo=ZoneInfo('Europe/Paris')) # 9h30 À PARIS
print(dt_aware)           # → 2026-08-24 09:30:00+02:00
print(dt_aware.astimezone(datetime.timezone.utc))
# → 2026-08-24 07:30:00+00:00
```

(Chez `pytz`, cette étape était `tz.localize(dt_naive)` — si tu lis du code ancien.) À l'inverse, convertir *directement* un naïf avec `astimezone` sans déclarer son fuseau, c'est laisser Python **supposer** l'heure locale de la machine — source d'ambiguïtés : déclare, puis convertis.

17.6 Formater et parser : `strftime` et `strptime`

Datetime → chaîne : `isoformat()` pour les machines, `strftime` pour les humains :

```
dt = datetime.datetime(2026, 8, 24, 12, 30, 45)

print(dt.isoformat())           # → 2026-08-24T12:30:45    (stockage, tri, APIs)
print(dt.strftime('%B %d, %Y')) # → August 24, 2026
print(dt.strftime('%d/%m/%Y %H:%M')) # → 24/08/2026 12:30
```

Chaîne → datetime : `strptime`, avec le format qui décrit la chaîne d'entrée :

```
dt = datetime.datetime.strptime('July 26, 2016', '%B %d, %Y')
print(dt)    # → 2016-07-26 00:00:00
```


Mnémotechnique de la vidéo : `strftime` = Format (vers chaîne) ; `strptime` = Parse (depuis chaîne). Les codes principaux :

Code	Sens	Exemple
<code>%Y / %y</code>	année 4 / 2 chiffres	2026 / 26
<code>%m / %d</code>	mois / jour, zéro-paddés	08 / 24
<code>%B / %b</code>	nom du mois complet / abrégé	August / Aug
<code>%A / %a</code>	jour de semaine complet / abrégé	Monday / Mon
<code>%H / %M / %S</code>	heure / minute / seconde	12 / 30 / 45

Corey avoue les rechercher à chaque fois dans la doc — fais pareil, personne ne les connaît par cœur. Et souviens-toi du chapitre 2 : ces codes marchent **directement dans une f-string** : `f'{dt:%d/%m/%Y}'`.

Piège — `strptime` est strict : la chaîne doit correspondre *exactement* au format, sinon `ValueError: time data ... does not match format ...`. C'est une `ValueError` comme une autre : le chapitre 13 sait la gérer. Pour l'ISO (`'2026-08-24'`), plus simple : `datetime.date.fromisoformat('2026-08-24')`.

Exercices

Exercice 17.1 ● — Affiche : la date du jour ; le jour de la semaine en toutes lettres (`%A`) ; la date dans 45 jours ; et le nombre de jours restants jusqu'au 1 janvier 2027.

Exercice 17.2 ● — `naissance = datetime.date(2004, 3, 15)`. Calcule l'âge en années entières « à la louche » (`(today - naissance).days // 365`), puis explique en quoi cette formule est approximative (années bissextiles !) et pourquoi elle peut se tromper autour de la date d'anniversaire.

Exercice 17.3 ●● — Une obligation détachera ses coupons chaque 15 mai des années 2027 à 2030. Génère la liste de ces quatre `date` (compréhension !), puis affiche pour chacune : la date au format `15/05/2027` et la maturité en années depuis aujourd'hui (`(d - today).days / 365` , 2 décimales).

Exercice 17.4 ●● — La clôture de la bourse de New York est à 16 h 00, heure locale. Pour aujourd'hui : construis ce `datetime aware` (`America/New_York`), puis affiche l'heure correspondante à Paris et en UTC. Que donnerait le même exercice un jour de janvier — et pourquoi le décalage Paris/New York n'est-il pas constant dans l'année ?

Exercice 17.5 ●●● — Un fichier de log contient des lignes du type `2026-08-24 09:31:07 | ORDRE AAPL` (heure de Paris, naïve). Écris `vers_utc_iso(ligne)` qui extrait l'horodatage (slicing : les 19 premiers caractères), le parse avec `strptime`, le déclare heure de Paris, le convertit en UTC et renvoie l'ISO 8601. Attendu pour la ligne ci-dessus : `2026-08-24T07:31:07+00:00`. Gère la ligne malformée en laissant remonter une `ValueError` claire — pourquoi est-ce le bon choix ici (chapitre 13) ?

Solutions détaillées

Solution 17.1 —

```
import datetime

today = datetime.date.today()
print(today)                # → 2026-08-24
print(today.strftime('%A'))  # → Monday
print(today + datetime.timedelta(days=45))  # → 2026-10-08
print((datetime.date(2027, 1, 1) - today).days)  # → 130
```

Solution 17.2 —

```
naissance = datetime.date(2004, 3, 15)
age = (datetime.date.today() - naissance).days // 365
print(age)    # → 22
```

`(today - naissance).days` est **exact** (les `timedeltas` ne mentent pas) ; c'est la division par 365 qui néglige les années bissextiles : sur 22 ans, ~5 jours d'écart accumulés. Conséquence : dans la fenêtre de quelques jours autour de l'anniversaire, le quotient peut basculer trop tôt ou trop tard. Le calcul irréfutable compare année/mois/jour :

`age = today.year - naissance.year - ((today.month, today.day) < (naissance.month, naissance.day))` — comparaison de tuples, élégance Python.

Solution 17.3 —

```
today = datetime.date.today()
coupons = [datetime.date(annee, 5, 15) for annee in range(2027, 2031)]

for d in coupons:
    maturite = (d - today).days / 365
    print(f"{d.strftime('%d/%m/%Y')} - {maturite:.2f} ans")
# → 15/05/2027 - 0.72 ans
# → 15/05/2028 - 1.73 ans
# → 15/05/2029 - 2.73 ans
# → 15/05/2030 - 3.73 ans
```

(Regarde l'écart entre les deux premières : +1.01 an et non +1.00 — le 29 février 2028 s'est glissé dans l'intervalle. La division par 365 accuse le coup : clin d'œil à l'exercice 17.2.)

Solution 17.4 —

```
from zoneinfo import ZoneInfo
import datetime

today = datetime.date.today()
cloture_ny = datetime.datetime(today.year, today.month, today.day, 16, 0,
                                tzinfo=ZoneInfo('America/New_York'))
```

```
print(cloture_ny) # → 2026-08-24 16:00:00-04:00
print(cloture_ny.astimezone(ZoneInfo('Europe/Paris'))) # → 2026-08-24 22:00:00+02:00
print(cloture_ny.astimezone(datetime.timezone.utc)) # → 2026-08-24 20:00:00+00:00
```

En août : NY est en UTC−4 (EDT), Paris en UTC+2 → clôture à 22 h. En **janvier** : NY passe en UTC−5 (EST) et Paris en UTC+1 → 16 h NY = 22 h Paris aussi... mais pendant quelques semaines de mars et d'octobre-novembre, les deux pays ne changent pas d'heure à la même date : l'écart devient 5 h et la clôture tombe à 21 h. C'est exactement le chaos que la règle « tout en UTC, conversion à l'affichage » élimine.

Solution 17.5 —

```
from zoneinfo import ZoneInfo
import datetime

def vers_utc_iso(ligne):
    """Convertit l'horodatage Paris d'une ligne de log en ISO 8601 UTC."""
    horodatage = ligne[:19] # '2026-08-24 09:31:07'
    dt = datetime.datetime.strptime(horodatage, '%Y-%m-%d %H:%M:%S')
    dt_paris = dt.replace(tzinfo=ZoneInfo('Europe/Paris'))
    return dt_paris.astimezone(datetime.timezone.utc).isoformat()

print(vers_utc_iso('2026-08-24 09:31:07 | ORDRE AAPL'))
# → 2026-08-24T07:31:07+00:00
```

Ligne malformée → `strptime` lève de lui-même une `ValueError` au message limpide (« time data ... does not match format »). La laisser remonter est le bon choix du chapitre 13 : un horodatage illisible dans un log d'ordres n'est pas un cas « normal » à étouffer — c'est une anomalie de données que l'appelant doit voir (et décider : ignorer la ligne, alerter, stopper). La fonction reste pure : elle convertit ou elle crie.

Fiche récap — Chapitre 17

Syntaxe	Effet
<code>datetime.date(2026, 8, 24)</code>	date (pas de 07 : <code>SyntaxError</code>) ; <code>.today()</code> , <code>.year/.month/.day</code>
<code>.weekday()</code> / <code>.isoweekday()</code>	lundi=0...6 / lundi=1...7
<code>datetime.timedelta(days=7, hours=...)</code>	durée ; <code>date ± td → date</code> ; <code>date - date → timedelta</code> (<code>.days</code> , <code>.total_seconds()</code>)
<code>datetime.datetime(...)</code>	date+heure ; <code>.date()</code> , <code>.time()</code> ; même arithmétique
Maintenant	<code>now()</code> naïf local ; <code>now(tz=...)</code> aware (la bonne forme) ; <code>utcnow()</code> déprécié
Aware vs naïf	aware = suffixe <code>+02:00</code> ; naïf = ambigu
<code>ZoneInfo('Europe/Paris')</code>	fuseaux IANA (stdlib 3.9+ ; remplace pytz)

Syntaxe	Effet
<code>dt.astimezone(tz)</code>	conversion d'affichage ; naïf → d'abord <code>replace(tzinfo=...)</code>
Règle d'or	stocker/calculer en UTC, convertir à l'affichage
<code>dt.isoformat()</code>	ISO 8601 — stockage et échange
<code>strftime(fmt) / strptime(s, fmt)</code>	Format vers chaîne / Parse depuis chaîne ; codes <code>%Y %m %d %B %A %H %M %S</code> (doc !)

CHAPITRE 18

Naviguer dans le système : le module os

Vidéo source : [Python Tutorial: OS Module - Use Underlying Operating System Functionality](#) — Corey Schafer

Objectifs : se déplacer dans les dossiers, créer/renommer/supprimer fichiers et répertoires, lire leurs métadonnées, parcourir des arborescences entières avec `os.walk`, et construire des chemins portables avec `os.path`.

18.1 Le module et son exploration

`os` est la passerelle vers le **système d'exploitation** : système de fichiers, variables d'environnement, processus. Premier réflexe de la vidéo, déjà rodé au chapitre 2 :

```
import os

print(dir(os))  # → la (très longue) liste des fonctions disponibles
```

On n'en visite qu'une sélection — la plus utile au quotidien.

18.2 Se repérer et se déplacer

```
print(os.getcwd())  # → C:\Users\mathias\Documents\python-cours
                    # (current working directory : le dossier « où tu es »)

os.chdir(r'C:\Users\mathias\Desktop')  # change directory
print(os.getcwd())  # → C:\Users\mathias\Desktop
```

Le répertoire courant est celui où s'appliquent, par défaut, tous les chemins **relatifs** — c'est lui qui explique le classique « `FileNotFoundError` alors que le fichier existe » : le script ne « regardait » pas dans le bon dossier.

Aparté Windows — Note le `r'...'` (*raw string*) : dans une chaîne brute, les antislashes sont pris tels quels. Sans le `r`, `\u`, `\t`, `\n` ... seraient interprétés comme séquences d'échappement (chapitre 2) — `'C:\temp'` contient une tabulation ! Trois parades : chaîne brute `r'C:\temp'`, antislashes doublés `'C:\\temp'`, ou... slashes simples `'C:/temp'`, que Windows accepte très bien.

Lister le contenu d'un dossier :

```
print(os.listdir())  # → ['exemple.txt', 'projets', 'notes.md']  (fichiers ET dossiers)
```

(`os.listdir(chemin)` pour viser un autre dossier sans s'y déplacer.)

18.3 Créer et supprimer des dossiers

Deux paires de fonctions, un cran de puissance d'écart :

```
os.mkdir('os-demo-2')           # crée UN dossier
os.makedirs('os-demo/sub-1/sub-2') # crée toute la CHAÎNE de dossiers intermédiaires
```

`os.mkdir('a/b/c')` échoue (`FileNotFoundError`) si `a/b` n'existe pas ; `makedirs` construit tout le chemin.

Symétriquement :

```
os.rmdir('os-demo-2')           # supprime UN dossier (vide)
os.removedirs('os-demo/sub-1/sub-2') # supprime la chaîne (tous vides)
```

Le conseil de prudence de Corey mérite d'être retenu tel quel : préfère `rmdir`, qui supprime exactement ce que tu désignes — `removedirs` remonte la chaîne et en supprime plus que tu ne crois. Et ni l'un ni l'autre ne supprime un dossier **non vide** (`OSError`) : pour raser un arbre entier, c'est `shutil.rmtree` — puissant, définitif (pas de corbeille !), à manier en réfléchissant deux fois.

18.4 Renommer et inspecter

```
os.rename('test.txt', 'demo.txt') # renomme (ou déplace !) un fichier
```

Les **métadonnées** d'un fichier — `os.stat` :

```
stats = os.stat('demo.txt')
print(stats.st_size)      # → 4096           taille en octets
print(stats.st_mtime)     # → 1787245123.55 dernière modification... en timestamp Unix
```

Un *timestamp* est un nombre de secondes depuis le 1 janvier 1970 (UTC) — illisible tel quel. Le pont vers le chapitre 17 :

```
from datetime import datetime

mod_time = os.stat('demo.txt').st_mtime
print(datetime.fromtimestamp(mod_time)) # → 2026-08-24 14:32:05.550000
```

Cas d'usage donné dans la vidéo : détecter les fichiers modifiés récemment (sauvegardes incrémentales, tri de téléchargements...).

18.5 Parcourir une arborescence : `os.walk`

La fonction la plus puissante du module. `os.walk(chemin)` **descend récursivement** dans tous les sous-dossiers ; à chaque étage, elle produit un triplet (`dirpath`, `dirnames`, `filenames`) :

```

for dirpath, dirnames, filenames in os.walk(r'C:\Users\mathias\Desktop'):
    print('Current Path:', dirpath)
    print('Directories:', dirnames)
    print('Files:', filenames)
    print()
# → Current Path: C:\Users\mathias\Desktop
# Directories: ['projets', 'archives']
# Files: ['demo.txt']
#
# Current Path: C:\Users\mathias\Desktop\projets
# Directories: []
# Files: ['main.py', 'notes.md']
# ...

```

À chaque tour : le dossier visité, la liste de ses sous-dossiers, la liste de ses fichiers — puis `walk` visite chacun des sous-dossiers à son tour, jusqu'au fond. C'est le déballage de tuple à trois variables (chapitres 4-5) au service du système de fichiers.

Cas d'usage cité par Corey : retrouver un fichier égaré quelque part sous un dossier, inventorier un site web, indexer une bibliothèque. Motif type « chercher tous les `.py` » :

```

for dirpath, dirnames, filenames in os.walk(os.getcwd()):
    for filename in filenames:
        if filename.endswith('.py'):
            print(os.path.join(dirpath, filename))

```

(`endswith` : méthode de chaîne, cousine de celles du chapitre 2 ; `os.path.join` : section suivante.)

18.6 Variables d'environnement

```

print(os.environ.get('USERPROFILE'))    # → C:\Users\mathias      (Windows)
# sur Mac/Linux, la variable équivalente est HOME (celle de la vidéo)

```

`os.environ` se comporte comme un dictionnaire — d'où le `.get()` du chapitre 5, qui renvoie `None` au lieu de planter si la variable n'existe pas. Les variables d'environnement servent notamment à configurer un programme sans coder les secrets en dur (clés d'API, chemins propres à chaque machine) — tu retrouves aussi le `PYTHONPATH` du chapitre 9.

18.7 `os.path` : la boîte à outils des chemins

Assembler un chemin — le point le plus important de la section. La concaténation manuelle est un nid à bugs :

```

file_path = os.environ.get('USERPROFILE') + 'test.txt'
print(file_path)    # → C:\Users\mathias\test.txt    ← séparateur oublié !

```

La bonne fonction :

```
file_path = os.path.join(os.environ.get('USERPROFILE'), 'test.txt')
print(file_path)    # → C:\Users\mathias\test.txt
```

`os.path.join` place le bon séparateur, au bon endroit, pour le bon système (`\` Windows, `/` ailleurs) — la portabilité offerte.

Décomposer un chemin (fonctions purement textuelles : le fichier n'a pas besoin d'exister) :

```
p = r'C:\Users\mathias\rapports\resultats.txt'

print(os.path.basename(p))    # → resultats.txt           (le nom)
print(os.path.dirname(p))     # → C:\Users\mathias\rapports (le dossier)
print(os.path.split(p))       # → ('C:\\Users\\mathias\\rapports', 'resultats.txt')
print(os.path.splitext(p))    # → ('C:\\Users\\mathias\\rapports\\resultats', '.txt')
```

`splitext` (*split extension*) est la façon propre d'attraper ou de retirer une extension — infiniment plus fiable qu'un `split('.')` (pense à `rapport.tar.gz` ou `mon.fichier.txt`).

Tester l'existence et la nature :

```
print(os.path.exists(p))    # → True / False    (le chemin existe-t-il ?)
print(os.path.isdir(p))    # → False           (est-ce un dossier ?)
print(os.path.isfile(p))   # → True            (est-ce un fichier ?)
```

Et `dir(os.path)` t'en montrera d'autres du même tonneau.

Depuis la vidéo — pathlib — Depuis Python 3.6+, la stdlib offre une seconde interface, orientée objet : `pathlib`. Le tableau de correspondance :

```
from pathlib import Path

p = Path.home() / 'rapports' / 'resultats.txt'  # le / assemble les chemins !
p.name          # resultats.txt                (basename)
p.parent        # ...\\rapports                 (dirname)
p.suffix        # .txt                         (splitext[1])
p.exists(), p.is_file(), p.is_dir()
p.read_text(encoding='utf-8')                  # lecture en une ligne (petits fichiers)
```

Le code moderne penche vers `pathlib` (l'opérateur `/` est délicieux) ; `os` / `os.path` reste omniprésent dans le code existant et nécessaire pour `walk`, `environ`, `stat` ... Connaître les deux, écrire tes nouveaux scripts avec celui qui te rend le code le plus clair.

Exercices

Exercice 18.1 ● — Affiche ton répertoire courant, la liste de son contenu, puis crée l'arborescence `sandbox/data/raw` en un seul appel. Vérifie avec `os.path.isdir`, puis supprime proprement les trois niveaux (quelle fonction, et pourquoi est-ce sûr ici ?).

Exercice 18.2 ● — Pour le chemin `r'C:\Users\mathias\Downloads\rapport_final.v2.pdf'` : donne (sans exécuter, puis vérifie) `basename`, `dirname`, `splitext`. Que renvoie `splitext` exactement pour ce nom à deux points ?

Exercice 18.3 ●● — Écris un script qui liste les fichiers du répertoire courant avec, pour chacun, sa taille en Ko (1 décimale) et sa date de modification au format `24/08/2026 14:32`. (Combine `os.listdir`, `os.path.isfile`, `os.stat`, `datetime.fromtimestamp` et le `strftime` du chapitre 17.)

Exercice 18.4 ●● — Avec `os.walk` sur ton dossier de cours : compte le nombre total de fichiers `.py` et leur taille cumulée. Affiche « X fichiers Python, Y Ko au total ».

Exercice 18.5 ●●● — Le grand classique : un dossier `factures/` contient des fichiers nommés `facture-mars-2026.pdf`, `facture-avril-2026.pdf`... Écris un script qui les **renomme** en `2026-03-facture.pdf`, `2026-04-facture.pdf` (tri chronologique naturel !). Découpe chaque nom avec `splitext` + `split('-')`, convertis le mois via un dictionnaire `{'janvier': '01', ...}`, et assemble avec une f-string. Sécurité : affiche d'abord les renommages prévus (*dry run*), et n'exécute `os.rename` qu'après un `input('Confirmer ? (o/n) ')`. Pourquoi ce garde-fou est-il indispensable sur ce genre de script ?

Solutions détaillées

Solution 18.1 —

```
import os

print(os.getcwd())
print(os.listdir())

os.makedirs('sandbox/data/raw')
print(os.path.isdir('sandbox/data/raw'))    # → True

os.removedirs('sandbox/data/raw')           # remonte et supprime les 3 niveaux
print(os.path.exists('sandbox'))           # → False
```

`removedirs` est sûr ici parce qu'on vient de créer la chaîne et qu'elle est vide : il supprime exactement ce que `makedirs` avait créé. Dans un dossier de travail réel (où `sandbox` pourrait contenir autre chose), on préférerait des `rmdir` ciblés — et de toute façon, tout dossier non vide fait échouer l'opération, c'est le filet de sécurité.

Solution 18.2 —

```
p = r'C:\Users\mathias\Downloads\rapport_final.v2.pdf'
print(os.path.basename(p))    # → rapport_final.v2.pdf
print(os.path.dirname(p))     # → C:\Users\mathias\Downloads
print(os.path.splitext(p))    # → ('C:\Users\mathias\Downloads\rapport_final.v2',
'.pdf')
```

`splitext` coupe au **dernier** point : l'extension est `.pdf`, et le `.v2` reste dans la racine — exactement le comportement voulu, et ce qu'un `split('.')` naïf aurait raté.

Solution 18.3 —

```
import os
from datetime import datetime

for nom in os.listdir():
    if os.path.isfile(nom):
        stats = os.stat(nom)
        taille_ko = stats.st_size / 1024
        modif = datetime.fromtimestamp(stats.st_mtime)
        print(f"{nom:<30} {taille_ko:>8.1f} Ko   {modif:%d/%m/%Y %H:%M}")
# → ch01.md          12.4 Ko   24/08/2026 14:32   (par exemple)
```

Le test `isfile` écarte les dossiers (dont la « taille » n'aurait pas ce sens). Bonus de formatage : `<30` cale à gauche sur 30 caractères, `>8.1f` cale à droite — les f-strings font aussi les tableaux.

Solution 18.4 —

```
import os

nb, total = 0, 0
for dirpath, dirnames, filenames in os.walk(os.getcwd()):
    for filename in filenames:
        if filename.endswith('.py'):
            nb += 1
            total += os.stat(os.path.join(dirpath, filename)).st_size

print(f'{nb} fichiers Python, {total / 1024:.1f} Ko au total')
```

Point crucial : `os.stat(filename)` seul échouerait pour les fichiers des sous-dossiers — `filenames` ne contient que des **noms**, pas des chemins. D'où le `os.path.join(dirpath, filename)` systématique : c'est LE réflexe `os.walk`.

Solution 18.5 —

```
import os

MOIS = {'janvier': '01', 'fevrier': '02', 'mars': '03', 'avril': '04',
        'mai': '05', 'juin': '06', 'juillet': '07', 'aout': '08',
        'septembre': '09', 'octobre': '10', 'novembre': '11', 'decembre': '12'}
```

```

os.chdir('factures')
renommages = []

for nom in os.listdir():
    racine, ext = os.path.splitext(nom)          # 'facture-mars-2026', '.pdf'
    partie = racine.split('-')                    # ['facture', 'mars', '2026']
    if len(partie) == 3 and partie[1] in MOIS:
        doc, mois, annee = partie
        nouveau = f'{annee}-{MOIS[mois]}-{doc}{ext}'
        renommages.append((nom, nouveau))

for ancien, nouveau in renommages:
    print(f'{ancien} → {nouveau}')

if input('Confirmer ? (o/n) ') == 'o':
    for ancien, nouveau in renommages:
        os.rename(ancien, nouveau)
    print('Renommage effectué.')
# facture-mars-2026.pdf → 2026-03-facture.pdf
# facture-avril-2026.pdf → 2026-04-facture.pdf

```

Le garde-fou *dry run* est indispensable parce qu'un renommage de masse est **quasi irréversible** : pas de corbeille, et un bug de découpage (un fichier `facture-mars-2026 (1).pdf` inattendu...) écraserait ou mutilerait des dizaines de fichiers d'un coup. Règle générale pour tout script qui *modifie* le disque : montrer d'abord, agir ensuite. — Ce script est d'ailleurs le sujet exact d'une autre vidéo de Corey (« Automate Parsing and Renaming of Multiple Files »), c'est dire si le besoin est réel.

Fiche récap — Chapitre 18

Syntaxe	Effet
<code>os.getcwd()</code> / <code>os.chdir(p)</code>	répertoire courant / s'y déplacer (chemins relatifs !)
<code>os.listdir()</code>	contenu (fichiers + dossiers), noms seuls
<code>os.mkdir</code> / <code>os.makedirs</code>	un dossier / toute la chaîne intermédiaire
<code>os.rmdir</code> / <code>os.removedirs</code>	suppression (dossiers vides) ; préférer <code>rmdir</code> ciblé ; arbre entier → <code>shutil.rmtree</code>
<code>os.rename(a, b)</code>	renommer/déplacer
<code>os.stat(f)</code>	<code>.st_size</code> (octets), <code>.st_mtime</code> (timestamp → <code>datetime.fromtimestamp</code>)
<code>os.walk(p)</code>	descente récursive : triplets (<code>dirpath</code> , <code>dirnames</code> , <code>filenames</code>) ; joindre <code>dirpath</code> aux noms !
<code>os.environ.get('VAR')</code>	variables d'environnement (dict-like)
<code>os.path.join(a, b)</code>	assemble avec le bon séparateur — jamais de <code>+</code>

Syntaxe	Effet
<code>basename / dirname / split / splitext</code>	nom / dossier / les deux / (racine, <code>.ext</code>) — purement textuel
<code>exists / isdir / isfile</code>	tests d'existence et de nature
<code>r'C:\...'</code>	chaîne brute : antislashes Windows sans échappement
Moderne	<code>pathlib.Path</code> : <code>/</code> pour joindre, <code>.name</code> , <code>.suffix</code> , <code>.exists()</code> ...

PARTIE IV

Programmation orientée objet

CHAPITRE 19

POO 1 : classes et instances

Vidéo source : [Python OOP Tutorial 1: Classes and Instances](#) — Corey Schafer

Objectifs : comprendre ce qu'apporte la programmation orientée objet, créer une classe, initialiser des instances avec `__init__`, écrire des méthodes — et enfin percer le mystère de `self`.

19.1 Pourquoi des classes ?

La **programmation orientée objet** (POO, *OOP*) consiste à regrouper au même endroit des **données** et les **fonctions** qui les manipulent. Vocabulaire : les données d'une classe sont ses **attributs**, ses fonctions sont ses **méthodes** (le mot promis au chapitre 2 — une méthode est une fonction attachée à un objet).

Tu fais de la POO depuis le chapitre 2 sans le savoir : une chaîne est une instance de la classe `str`, dotée d'attributs et de méthodes (`upper`, `find`...); pareil pour `list`, `dict`, `datetime.date`... La nouveauté : **créer tes propres types**.

Le fil rouge de toute la série POO de Corey : représenter les **employés** d'une entreprise. Chaque employé a un prénom, un nom, un salaire, un email — et des comportements communs. Une classe est exactement ça : un **plan de fabrication** (*blueprint*) ; chaque employé concret construit d'après ce plan est une **instance**.

```
class Employee:
    pass
```

(`class` + nom en **CapWords** — convention : les classes prennent des majuscules, `Employee`, `BankAccount` — et le `pass` du chapitre 8 pour un corps provisoirement vide.)

```
emp_1 = Employee()
emp_2 = Employee()

print(emp_1)    # → <__main__.Employee object at 0x7f8a5c2d90>
print(emp_2)    # → <__main__.Employee object at 0x7f8a5c2e10>
```

Appeler la classe comme une fonction **fabrique une instance**. Les deux adresses mémoire diffèrent : `emp_1` et `emp_2` sont deux objets **distincts** (souvenir de `is` / `id()` du chapitre 6).

19.2 La mauvaise méthode : attributs à la main

On *peut* attacher des données à une instance après coup :

```
emp_1.first = 'Corey'
emp_1.last = 'Schafer'
emp_1.email = 'Corey.Schafer@company.com'
emp_1.pay = 50000
```

```
emp_2.first = 'Test'
emp_2.last = 'User'
emp_2.email = 'Test.User@company.com'
emp_2.pay = 60000

print(emp_1.email)    # → Corey.Schafer@company.com
print(emp_2.email)    # → Test.User@company.com
```

Ça fonctionne — et c'est exactement ce qu'il ne faut pas faire : huit lignes répétitives par paire d'employés, l'email reconstruit à la main (avec la faute de frappe qui guette), aucune garantie qu'une instance ait tous ses attributs. On viole le DRY du chapitre 8. La POO promet mieux : automatiser cette mise en place.

19.3 `__init__` : le constructeur

La méthode spéciale `__init__` (*initialize*) s'exécute **automatiquement à chaque création d'instance** :

```
class Employee:

    def __init__(self, first, last, pay):
        self.first = first
        self.last = last
        self.pay = pay
        self.email = first + '.' + last + '@company.com'
```

Lecture ligne à ligne :

- une méthode se définit comme une fonction (chapitre 8), *dans* la classe ;
- son premier paramètre est l'instance en cours de fabrication — nommée `self` par une convention universelle (le mot n'est pas réservé, mais ne t'avise pas d'en changer) ;
- `self.first = first` : « sur *cette* instance, crée l'attribut `first` avec la valeur du paramètre `first` ». C'est l'équivalent automatisé du `emp_1.first = 'Corey'` de tout à l'heure ;
- l'email est **dérivé** des autres données — une seule source de vérité, plus de faute de frappe possible.

La création devient :

```
emp_1 = Employee('Corey', 'Schafer', 50000)
emp_2 = Employee('Test', 'User', 60000)

print(emp_1.email)    # → Corey.Schafer@company.com
print(emp_2.email)    # → Test.User@company.com
```

Subtilité qui déroute tout le monde : on passe **trois** arguments alors que `__init__` en déclare quatre. C'est que **l'instance se passe toute seule** : `Employee('Corey', ...)` crée l'objet, le glisse dans `self`, et distribue le reste. Les attributs ainsi créés (`first`, `last`, `pay`, `email`) sont des **attributs d'instance** : chaque employé a les siens.

19.4 Les méthodes

Afficher le nom complet, sans méthode :

```
print(f'{emp_1.first} {emp_1.last}')    # → Corey Schafer
```

... à recopier partout où on en a besoin. La version POO — le comportement rejoint les données :

```
class Employee:

    def __init__(self, first, last, pay):
        self.first = first
        self.last = last
        self.pay = pay
        self.email = first + '.' + last + '@company.com'

    def fullname(self):
        return f'{self.first} {self.last}'
```

```
emp_1 = Employee('Corey', 'Schafer', 50000)
emp_2 = Employee('Test', 'User', 60000)

print(emp_1.fullname())    # → Corey Schafer
print(emp_2.fullname())    # → Test User
```

Chaque instance répond avec ses données : `self` est la cheville qui permet à un même code de servir tous les employés.

Piège n° 1 — oublier `self` : définis la méthode sans lui, dans la classe...

```
class Employee:
    # ... __init__ inchangé ...
    def fullname():
        return f'{self.first} {self.last}'

emp_1 = Employee('Corey', 'Schafer', 50000)
emp_1.fullname()
# TypeError: Employee.fullname() takes 0 positional arguments but 1 was given
```

« 1 was given » ?! On n'a rien passé... si : l'**instance**, automatiquement. Ce message cryptique — tu le rencontreras — signifie presque toujours « self oublié dans la définition ».

Piège n° 2 — oublier les parenthèses :

```
print(emp_1.fullname)
# → <bound method Employee.fullname of <__main__.Employee object at 0x...>>
```

Sans `()`, on affiche la méthode *elle-même* (souvenir du chapitre 8 : `f` vs `f()`). Avec, on l'exécute. Symptôme : un affichage `<bound method ...>` là où tu attendais une valeur.

19.5 Sous le capot : `self` démystifié

Le tour de magie final de la vidéo. Ces deux appels sont **rigoureusement équivalents** :

```
print(emp_1.fullname())          # → Corey Schafer   (forme courante)
print(Employee.fullname(emp_1))  # → Corey Schafer   (ce qui se passe vraiment)
```

Quand tu écris `emp_1.fullname()`, Python le transforme en `Employee.fullname(emp_1)` : il appelle la méthode **de la classe** en passant l'instance comme premier argument — qui atterrit dans... `self`. Voilà le mystère dissous : `self` n'est pas un mot magique, c'est le paramètre qui reçoit l'instance. Garde cette équivalence en tête ; elle rendra limpides les `classmethod` / `staticmethod` du chapitre 21.

Pour aller plus loin — Quand créer une classe ? Quand des données voyagent *ensemble* et ont des comportements propres : un compte bancaire (solde + dépôt/retrait), une obligation (nominal, coupon, échéance + prix, duration), un portefeuille (positions + valorisation). Si tu te surprends à trimballer des dictionnaires `{'first': ..., 'pay': ...}` de fonction en fonction, une classe attend de naître. À l'inverse, pour un simple groupe de valeurs sans comportement, un dict ou un tuple suffit — la POO est un outil, pas une religion. (Python moderne offre aussi les `dataclasses` qui écrivent `__init__` pour toi — à découvrir une fois cette série digérée.)

Exercices

Fil rouge des exercices de la partie IV : une classe `CompteBancaire`, construite chapitre après chapitre.

Exercice 19.1 ● — Écris la classe `CompteBancaire` : `__init__` reçoit `titulaire` (str) et `solde_initial` (float, défaut 0) ; attributs `titulaire` et `solde`. Crée deux comptes (dont un sans solde initial) et affiche leurs attributs.

Exercice 19.2 ● — Ajoute la méthode `resume(self)` qui **renvoie** `'Compte de Mathias — solde : 1500.00 €'` (f-string, 2 décimales). Pourquoi `return` plutôt que `print` dans la méthode (chapitre 8) ?

Exercice 19.3 ●● — Ajoute `deposer(self, montant)` et `retirer(self, montant)` qui mettent à jour `self.solde` ; `retirer` refuse (avec `raise ValueError(...)` — chapitre 13) tout retrait supérieur au solde. Enchaîne : dépôt de 500, retrait de 200, tentative de retrait de 10 000 (protégée par try/except), puis `resume()`.

Exercice 19.4 ●● — Sans exécuter : que produisent ces lignes, et pourquoi ?

```
c = CompteBancaire('Test', 100)
print(c.resume)
print(CompteBancaire.resume(c))
c2 = c
c2.deposer(50)
print(c.solde)
```

Exercice 19.5 ●●● — Nouvelle classe `Obligation` : attributs `nominal`, `taux_coupon`, `maturite` (années), et une méthode `prix(self, taux_actuariel)` qui renvoie $\sum_{t=1}^n \frac{c}{(1+r)^t} + \frac{N}{(1+r)^n}$ avec $c = N \times \text{taux_coupon}$ (ta fonction de l'exercice 8.5, devenue méthode). Vérifie : `Obligation(1000, 0.10, 3).prix(0.05) ≈ 1136.16` (comme au chapitre 8), et le prix au pair : `Obligation(1000, 0.05, 10).prix(0.05) ≈ 1000`.

Solutions détaillées

Solution 19.1 —

```
class CompteBancaire:

    def __init__(self, titulaire, solde_initial=0):
        self.titulaire = titulaire
        self.solde = solde_initial

c1 = CompteBancaire('Mathias', 1500)
c2 = CompteBancaire('Camille')

print(c1.titulaire, c1.solde)    # → Mathias 1500
print(c2.titulaire, c2.solde)    # → Camille 0
```

Les défauts de paramètres (chapitre 8) s'appliquent tels quels à `__init__` — noter que l'attribut (`solde`) n'est pas obligé de porter le nom du paramètre (`solde_initial`).

Solution 19.2 —

```
def resume(self):
    return f'Compte de {self.titulaire} — solde : {self.solde:.2f} €'

print(c1.resume())    # → Compte de Mathias — solde : 1500.00 €
```

`return` laisse l'appelant **libre** de l'usage : afficher, écrire dans un fichier, insérer dans un rapport, tester. Un `print` interne fige la méthode dans un seul usage (et renverrait `None` — piège 8.3).

Solution 19.3 —

```

class CompteBancaire:

    def __init__(self, titulaire, solde_initial=0):
        self.titulaire = titulaire
        self.solde = solde_initial

    def resume(self):
        return f'Compte de {self.titulaire} – solde : {self.solde:.2f} €'

    def deposer(self, montant):
        self.solde += montant

    def retirer(self, montant):
        if montant > self.solde:
            raise ValueError(f'Retrait de {montant} € impossible : solde {self.solde} €')
        self.solde -= montant

c = CompteBancaire('Mathias', 1500)
c.deposer(500)
c.retirer(200)

try:
    c.retirer(10000)
except ValueError as e:
    print('Refusé : ', e)      # → Refusé : Retrait de 10000 € impossible : solde 1800 €

print(c.resume())           # → Compte de Mathias – solde : 1800.00 €

```

La classe **protège son invariant** ($\text{solde} \geq 0$) : c'est l'un des grands bénéfices de la POO — les règles vivent avec les données, impossible de les contourner par étourderie.

Solution 19.4 —

```

print(c.resume)          → <bound method CompteBancaire.resume of ...>   (parenthèses
oubliées – piège n° 2)
print(CompteBancaire.resume(c)) → Compte de Test – solde : 100.00 €      (l'équivalence
de la section 19.5)
c2 = c ; c2.deposer(50) ; print(c.solde) → 150

```

Le dernier point recycle le chapitre 4 : `c2 = c` n'est **pas** une copie — deux noms, un seul objet ; le dépôt via `c2` se voit sur `c`. Les instances de classes sont des objets muables comme les listes.

Solution 19.5 —

```

class Obligation:

    def __init__(self, nominal, taux_coupon, maturite):
        self.nominal = nominal
        self.taux_coupon = taux_coupon
        self.maturite = maturite

```

```

def prix(self, taux_actuariel):
    """Prix = somme des flux actualisés au taux donné."""
    coupon = self.nominal * self.taux_coupon
    total = 0
    for t in range(1, self.maturite + 1):
        total += coupon / (1 + taux_actuariel) ** t
    total += self.nominal / (1 + taux_actuariel) ** self.maturite
    return total

oat = Obligation(1000, 0.10, 3)
print(f'{oat.prix(0.05):.2f}')          # → 1136.16

pair = Obligation(1000, 0.05, 10)
print(f'{pair.prix(0.05):.2f}')        # → 1000.00

```

La fonction de l'exercice 8.5 gagne un domicile : les caractéristiques du titre vivent dans l'objet, seul le taux du marché reste un paramètre — la signature raconte la finance. Et le test « coupon = taux actuariel $\Rightarrow$ prix = pair » est un excellent invariant de vérification (toute erreur d'indice le casse immédiatement).

Fiche récap — Chapitre 19

Syntaxe	Effet
<code>class Employee:</code>	définit une classe (noms en <code>CapWords</code>)
<code>emp = Employee(...)</code>	instancie — fabrique un objet d'après le plan
<code>def __init__(self, ...):</code>	constructeur, exécuté à chaque création
<code>self</code>	l'instance elle-même (1 paramètre de toute méthode — convention absolue)
<code>self.attr = valeur</code>	crée un attribut d'instance
<code>def methode(self):</code>	méthode ; accède aux données via <code>self.attr</code>
<code>emp.methode()</code>	$\Leftrightarrow$ <code>Employee.methode(emp)</code> — l'instance se passe toute seule
<code>TypeError ... but 1 was given</code>	<code>self</code> oublié dans la définition
<code><bound method ...></code> affiché	parenthèses d'appel oubliées
Quand une classe ?	données + comportements qui voyagent ensemble ; invariants à protéger

CHAPITRE 20

POO 2 : variables de classe

Vidéo source : [Python OOP Tutorial 2: Class Variables](#) — Corey Schafer

Objectifs : distinguer variables d'instance et variables de classe, comprendre l'ordre de recherche des attributs (et le masquage), et choisir en connaissance de cause entre `self.var` et `Classe.var`.

20.1 Le besoin : une donnée partagée

Les attributs du chapitre 19 (`first`, `pay`...) sont **propres à chaque instance**. Mais certaines données concernent **la classe entière** — l'exemple de la vidéo : le taux d'augmentation annuel, identique pour tous les employés.

Première tentative, en dur dans une méthode :

```
class Employee:

    def __init__(self, first, last, pay):
        self.first = first
        self.last = last
        self.pay = pay

    def apply_raise(self):
        self.pay = int(self.pay * 1.04)

emp_1 = Employee('Corey', 'Schafer', 50000)

print(emp_1.pay)      # → 50000
emp_1.apply_raise()
print(emp_1.pay)      # → 52000
```

Ça marche, mais Corey liste les défauts : le `1.04` est **enterré** dans la méthode (pour connaître le taux, il faut lire le code) ; impossible d'y accéder (`emp_1.raise_amount` n'existe pas) ; et s'il apparaît à plusieurs endroits, chaque changement de taux devient une chasse au trésor. Le remède : une **variable de classe**.

20.2 La variable de classe

Elle se déclare **dans la classe, hors de toute méthode** :

```
class Employee:

    raise_amount = 1.04          # variable de CLASSE

    def __init__(self, first, last, pay):
        self.first = first
        self.last = last
```

```

        self.pay = pay

    def apply_raise(self):
        self.pay = int(self.pay * self.raise_amount)

```

Et — première surprise — elle est accessible **par la classe ET par les instances** :

```

emp_1 = Employee('Corey', 'Schafer', 50000)
emp_2 = Employee('Test', 'User', 60000)

print(Employee.raise_amount)    # → 1.04
print(emp_1.raise_amount)       # → 1.04
print(emp_2.raise_amount)       # → 1.04

```

Comment `emp_1.raise_amount` peut-il marcher alors qu'aucun `self.raise_amount = ...` n'existe dans `__init__` ?

20.3 L'ordre de recherche des attributs

Règle fondamentale : quand on demande `emp_1.raise_amount`, Python cherche **d'abord dans l'instance**, puis, faute de trouver, **remonte à la classe** (puis aux classes parentes — chapitre 22). L'espace de noms de chacun se consulte avec `__dict__` :

```

print(emp_1.__dict__)
# → {'first': 'Corey', 'last': 'Schafer', 'pay': 50000}    ← pas de raise_amount !

print(Employee.__dict__)
# → {..., 'raise_amount': 1.04, '__init__': <function ...>, 'apply_raise':
<function ...>, ...}

```

`raise_amount` ne vit **que** dans la classe : les instances ne font que le *voir par transparence*. Modifier la classe se répercute donc partout :

```

Employee.raise_amount = 1.05

print(Employee.raise_amount)    # → 1.05
print(emp_1.raise_amount)       # → 1.05
print(emp_2.raise_amount)       # → 1.05

```

20.4 Le masquage : l'expérience clé de la vidéo

Que se passe-t-il si on assigne via **une instance** ? (On remet d'abord la classe à 1.04 pour repartir de la situation initiale.)

```

Employee.raise_amount = 1.04    # retour à la situation de départ

```

```
emp_1.raise_amount = 1.05

print(Employee.raise_amount)  # → 1.04    la classe n'a pas bougé
print(emp_1.raise_amount)    # → 1.05    l'instance a SA valeur
print(emp_2.raise_amount)    # → 1.04    l'autre instance suit la classe

print(emp_1.__dict__)
# → {'first': 'Corey', ..., 'raise_amount': 1.05} ← un attribut d'INSTANCE est né
```

L'assignation `emp_1.raise_amount = 1.05` n'a **pas** modifié la variable de classe : elle a créé un attribut d'instance du même nom, qui — ordre de recherche oblige — **masque** (*shadows*) désormais la version de la classe, pour cette instance seulement.

Piège — Résumons la dissymétrie, source de bugs subtils : **lire** `emp_1.raise_amount` remonte à la classe si besoin ; **écrire** `emp_1.raise_amount = ...` crée toujours un attribut d'instance. Lecture transparente, écriture locale.

Et dans `apply_raise`, alors : `self.raise_amount` ou `Employee.raise_amount` ? Corey a choisi `self.raise_amount`, et c'est réfléchi : cela laisse la porte ouverte à un taux personnalisé par instance (le masquage devient une *fonctionnalité* : donner 5 % à un seul employé), et cela permettra aux **sous-classes** de redéfinir le taux (chapitre 22 — les `Developer` à 10 %). `Employee.raise_amount` figerait tout le monde au taux de la classe mère.

20.5 Quand la classe s'impose : le compteur d'instances

Second exemple de la vidéo, où le choix inverse s'impose — compter les employés créés :

```
class Employee:

    num_of_emps = 0
    raise_amount = 1.04

    def __init__(self, first, last, pay):
        self.first = first
        self.last = last
        self.pay = pay

        Employee.num_of_emps += 1    # via la CLASSE, surtout pas self

print(Employee.num_of_emps)    # → 0

emp_1 = Employee('Corey', 'Schafer', 50000)
emp_2 = Employee('Test', 'User', 60000)

print(Employee.num_of_emps)    # → 2
```

`__init__` s'exécutant à chaque création, le compteur s'incrémente tout seul. Et ici, `Employee.num_of_emps` est le bon choix : un décompte global n'a aucune raison d'être personnalisé par instance — et un `self.num_of_emps +=`

1 serait même un bug caché : il se déplie en `self.num_of_emps = self.num_of_emps + 1`, dont l'assignation... créerait un attribut d'instance masquant le compteur (piège 20.4 en embuscade) !

La règle de décision, à retenir :

- `self.variable_de_classe` : quand un *override* par instance ou par sous-classe est souhaitable (paramètres, réglages, constantes ajustables) ;
- `Classe.variable_de_classe` : quand la donnée est un état global de la classe (compteurs, registres) qui ne doit jamais être masqué.

Pour aller plus loin — Autre piège du même tonneau : une variable de classe **mutable**. `class Portefeuille:`
`positions = []` partagerait LA MÊME liste entre toutes les instances (chacune `append`-ant dans la liste commune !). Les variables de classe se réservent aux immuables (nombres, chaînes, tuples) ou aux vraies données volontairement partagées ; tout ce qui est propre à l'instance naît dans `__init__`. C'est le cousin du piège des défauts muables qu'on recroisera au chapitre 22.

Exercices

Exercice 20.1 ● — Ajoute à `CompteBancaire` (chapitre 19) une variable de classe `banque = 'PyBanque'` et une variable `taux_interet = 0.02`. Vérifie qu'elles se lisent depuis la classe et depuis deux instances, puis affiche le `__dict__` d'une instance : pourquoi n'y figurent-elles pas ?

Exercice 20.2 ● — Ajoute la méthode `appliquer_interets(self)` qui fait `self.solde *= (1 + self.taux_interet)`. Justifie le choix de `self.taux_interet` plutôt que `CompteBancaire.taux_interet` (imagine un futur `CompteEpargne` ...).

Exercice 20.3 ●● — Prédise les six affichages, puis vérifie :

```
a = CompteBancaire('A', 100)
b = CompteBancaire('B', 100)
CompteBancaire.taux_interet = 0.03
print(a.taux_interet, b.taux_interet)
b.taux_interet = 0.10
print(a.taux_interet, b.taux_interet, CompteBancaire.taux_interet)
a.appliquer_interets(); b.appliquer_interets()
print(round(a.solde, 2), round(b.solde, 2))
```

Exercice 20.4 ●● — Ajoute le compteur `nb_comptes`, incrémenté dans `__init__`, et la variable de classe `total_depense = 0`, alimentée par `deposer` (chaque dépôt s'y ajoute). Pour chacune des deux, choisis et justifie `self.` ou `CompteBancaire.`. Crée trois comptes, fais deux dépôts, affiche les deux compteurs.

Exercice 20.5 ●●● — Bug hunt : ce code veut donner un historique à chaque compte, mais quelque chose cloche. Diagnostique précisément (avec les `__dict__` !) et corrige.


```

class CompteBuggy:
    historique = []

    def __init__(self, titulaire):
        self.titulaire = titulaire

    def deposer(self, montant):
        self.historique.append(montant)

x = CompteBuggy('X')
y = CompteBuggy('Y')
x.deposer(100)
y.deposer(9999)
print(x.historique)

```

Solutions détaillées

Solution 20.1 —

```

class CompteBancaire:

    banque = 'PyBanque'
    taux_interet = 0.02

    def __init__(self, titulaire, solde_initial=0):
        self.titulaire = titulaire
        self.solde = solde_initial

c1 = CompteBancaire('Mathias', 1500)
c2 = CompteBancaire('Camille')

print(CompteBancaire.banque, c1.banque, c2.banque)    # → PyBanque PyBanque PyBanque
print(c1.taux_interet)                                # → 0.02
print(c1.__dict__)    # → {'titulaire': 'Mathias', 'solde': 1500}

```

Le `__dict__` de l'instance ne contient que ce que `__init__` y a mis : `banque` et `taux_interet` vivent dans `CompteBancaire.__dict__`, et les instances les trouvent par remontée.

Solution 20.2 —

```

def appliquer_interets(self):
    self.solde *= (1 + self.taux_interet)

```

Avec `self.taux_interet`, un futur `CompteEpargne(CompteBancaire)` n'aura qu'à déclarer `taux_interet = 0.035` pour que la même méthode, héritée sans modification, serve son taux — et un compte négocié

individuellement peut recevoir son propre taux par masquage volontaire. `CompteBancaire.taux_interet` verrouillerait tout ce monde sur 2 %.

Solution 20.3 —

```
0.03 0.03          ← la modif de CLASSE se voit partout (aucun masque encore)
0.03 0.1 0.03      ← b s'est créé un attribut d'instance ; a et la classe restent à 0.03
103.0 110.0        ← a : 100 × 1.03 ; b : 100 × 1.10 — self.taux_interet sert le masque de
b
```

Toute la mécanique du chapitre en trois lignes : modification par la classe (globale), masquage par l'instance (local), et `self` dans la méthode qui honore le masque.

Solution 20.4 —

```
class CompteBancaire:

    banque = 'PyBanque'
    taux_interet = 0.02
    nb_comptes = 0
    total_depense = 0

    def __init__(self, titulaire, solde_initial=0):
        self.titulaire = titulaire
        self.solde = solde_initial
        CompteBancaire.nb_comptes += 1          # état global → via la classe

    def deposer(self, montant):
        self.solde += montant
        CompteBancaire.total_depense += montant # idem

a = CompteBancaire('A'); b = CompteBancaire('B'); c = CompteBancaire('C', 50)
a.deposer(100); b.deposer(250)

print(CompteBancaire.nb_comptes)    # → 3
print(CompteBancaire.total_depense)  # → 350
```

Les deux sont des agrégats **de la classe** — aucun sens à les personnaliser, et la forme `self.compteur += 1` créerait le masque-bug décrit en 20.5. (Le `taux_interet`, lui, reste en `self` : c'est toute la nuance.)

Solution 20.5 — Sortie du code buggé : `[100, 9999]` — l'historique de `x` contient le dépôt de `y` ! Diagnostic : `historique = []` est une variable de classe **mutable** ; `self.historique.append(...)` lit l'attribut (remontée vers la classe : pas de masque créé, car on ne fait pas d'assignation) puis mute LA liste commune. Preuve : `x.__dict__` et `y.__dict__` ne contiennent pas d'`historique`, et `x.historique is y.historique` → `True`. Correction — l'historique est un état **d'instance**, il naît dans `__init__` :

```
class Compte:
    def __init__(self, titulaire):
        self.titulaire = titulaire
```

```

self.historique = [] # une liste NEUVE par instance

def depoter(self, montant):
    self.historique.append(montant)

```

Règle mémorisable : *variable de classe muable* = *état partagé, voulu ou pas*. Dans le doute, les données de l'instance naissent dans `__init__`.

Fiche récap — Chapitre 20

Syntaxe	Effet
<code>class C: puis var = valeur</code> (hors méthode)	variable de classe , partagée
<code>C.var</code> et <code>inst.var</code>	lisibles toutes les deux (remontée instance → classe → parents)
<code>inst.__dict__</code> / <code>C.__dict__</code>	les espaces de noms — l'outil de diagnostic
<code>C.var = x</code>	modifie pour tout le monde
<code>inst.var = x</code>	crée un attribut d'instance qui masque la classe (pour cette instance)
Lecture vs écriture	lire remonte ; écrire crée local — la dissymétrie à retenir
<code>self.var</code> dans une méthode	honore masques et overrides de sous-classes (paramètres ajustables)
<code>C.var</code> dans une méthode	état global inmasquable (compteurs : <code>C.count += 1</code> dans <code>__init__</code>)
muables	variable de classe liste/dict = partagée entre toutes les instances

CHAPITRE 21

POO 3 : classmethods et staticmethods

Vidéo source : [Python OOP Tutorial 3: classmethods and staticmethods](#) — Corey Schafer

Objectifs : distinguer les trois espèces de méthodes — d'instance (`self`), de classe (`cls`), statiques (ni l'un ni l'autre) —, écrire des constructeurs alternatifs, et savoir choisir la bonne espèce.

21.1 Rappel : la méthode d'instance

Les méthodes vues jusqu'ici reçoivent l'**instance** en premier argument (`self`) — ce sont les méthodes *régulières*, qui travaillent sur les données d'un objet particulier. Question de la vidéo : et si une méthode devait travailler sur **la classe** plutôt que sur une instance ?

21.2 `@classmethod` : recevoir la classe

On transforme une méthode avec le **décorateur** `@classmethod` ; son premier paramètre reçoit alors **la classe**, nommée `cls` par convention (impossible d'utiliser `class`, mot réservé) :

```
class Employee:

    raise_amount = 1.04

    def __init__(self, first, last, pay):
        self.first = first
        self.last = last
        self.pay = pay

    def fullname(self):
        return f'{self.first} {self.last}'

    @classmethod
    def set_raise_amt(cls, amount):
        cls.raise_amount = amount
```

(Un *décorateur* — la ligne `@...` au-dessus du `def` — modifie le comportement de la fonction qu'il coiffe ; accepte-le comme une notation pour l'instant, Corey a une vidéo dédiée au mécanisme.) Usage :

```
emp_1 = Employee('Corey', 'Schafer', 50000)
emp_2 = Employee('Test', 'User', 60000)

Employee.set_raise_amt(1.05)

print(Employee.raise_amount) # → 1.05
print(emp_1.raise_amount)   # → 1.05
print(emp_2.raise_amount)   # → 1.05
```

`cls.raise_amount = amount` $\Leftrightarrow$ `Employee.raise_amount = amount` : on a modifié la variable **de classe**, donc tout le monde suit (chapitre 20). L'intérêt face à l'assignation directe : un point d'entrée officiel, documentable, où l'on pourra ajouter validation ou journalisation.

Piège — On *peut* appeler un classmethod depuis une instance (`emp_1.set_raise_amt(1.05)`)... mais c'est trompeur : `cls` reçoit quand même **la classe**, et la modification touche tout le monde — pas seulement `emp_1`. Ça se fait pas (dixit Corey) : appelle les classmethods via la classe, le code dira ce qu'il fait.

21.3 Le grand cas d'usage : les constructeurs alternatifs

Scénario réaliste de la vidéo : les données d'employés arrivent en **chaînes** au format `'John-Doe-70000'`. Sans outillage, chaque utilisateur de la classe répète le découpage :

```
emp_str_1 = 'John-Doe-70000'

first, last, pay = emp_str_1.split('-')      # déballage du split (chapitres 4 et 8)
new_emp = Employee(first, last, pay)
```

Si ce besoin est récurrent, la classe doit l'offrir elle-même : un **constructeur alternatif** — un classmethod qui fabrique et **renvoie** une instance :

```
@classmethod
def from_string(cls, emp_str):
    first, last, pay = emp_str.split('-')
    return cls(first, last, int(pay))
```

```
new_emp = Employee.from_string('John-Doe-70000')

print(new_emp.fullname())    # → John Doe
print(new_emp.pay)          # → 70000
```

La ligne magique est `return cls(first, last, int(pay))` : comme `cls` est la classe, `cls(...)` $\Leftrightarrow$ `Employee(...)` — le constructeur normal, appelé depuis l'intérieur. (Pourquoi `cls` et pas `Employee` en dur ? Pour que les sous-classes du chapitre 22 héritent d'un `from_string` qui construise des instances de *leur* type.) Convention de nommage : ces constructeurs s'appellent `from_...`.

La preuve que le motif est central — la `stdlib` en regorge, exemples de la vidéo à l'appui, que tu connais depuis le chapitre 17 :

```
import datetime

d1 = datetime.date(2026, 8, 24)          # constructeur normal
d2 = datetime.date.today()               # constructeur alternatif !
```

```
d3 = datetime.date.fromtimestamp(1787245123)    # un autre !
d4 = datetime.date.fromisoformat('2026-08-24') # encore un !
```

`today`, `fromtimestamp`, `fromisoformat`... sont tous des classmethods qui offrent d'autres portes d'entrée vers le même objet.

21.4 @staticmethod : ni instance ni classe

Troisième espèce : la méthode **statique** ne reçoit rien automatiquement — ni `self`, ni `cls`. C'est une fonction ordinaire, simplement rangée dans la classe parce qu'elle a un lien logique avec elle :

```
@staticmethod
def is_workday(day):
    if day.weekday() == 5 or day.weekday() == 6:
        return False
    return True
```

```
import datetime

my_date = datetime.date(2026, 8, 23)           # un dimanche
print(Employee.is_workday(my_date))           # → False

my_date = datetime.date(2026, 8, 24)           # un lundi
print(Employee.is_workday(my_date))           # → True
```

Savoir si une date est un jour ouvré concerne la gestion des employés — d'où sa place dans `Employee` — mais le calcul n'utilise ni un employé particulier ni la classe : parfait candidat statique. (Le `weekday()` — lundi 0, dimanche 6 — sort du chapitre 17 ; samedi = 5, dimanche = 6.)

La règle de choix, donnée telle quelle par Corey : si une méthode ne mentionne ni `self` ni `cls` dans son corps, elle devrait probablement être statique. Inversement :

Espèce	Décorateur	Reçoit	Sert à
méthode d'instance	—	<code>self</code>	travailler sur les données d'UN objet
méthode de classe	<code>@classmethod</code>	<code>cls</code>	travailler sur la classe ; constructeurs alternatifs <code>from...</code>
méthode statique	<code>@staticmethod</code>	rien	fonction utilitaire liée logiquement à la classe

Exercices

Exercice 21.1 ● — Ajoute à `CompteBancaire` le classmethod `changer_taux(cls, nouveau_taux)` qui met à jour `taux_interet` — en refusant (`ValueError`) tout taux négatif ou supérieur à 20 %. Démontre l'effet sur deux comptes existants, puis la protection.

Exercice 21.2 ● — Ajoute le constructeur alternatif `depuis_chaine(cls, texte)` pour le format `'Mathias:1500'`. Attention au type du solde ! Vérifie avec `CompteBancaire.depuis_chaine('Camille:250')`.

Exercice 21.3 ●● — Ajoute le staticmethod `valider_iban(iban)` (version jouet) : renvoie `True` si la chaîne commence par `'FR'` et compte exactement 27 caractères. Pourquoi statique plutôt que méthode d'instance — applique la règle de Corey. Teste `'FR' + '7' * 25` (True) et `'DE89370400440532013000'` (False).

Exercice 21.4 ●● — Ajoute un second constructeur `commun(cls, *titulaires)` qui crée un compte joint : titulaire = les noms joints par `' & '` (chapitre 4), solde 0. `CompteBancaire.commun('Mathias', 'Camille').titulaire` → `'Mathias & Camille'`. Quel chapitre t'a donné `*titulaires` ?

Exercice 21.5 ●●● — Question de design, sans code : pour chacune, instance / classe / statique ? Justifie en une ligne. a) `solde_en(self?, devise)` — convertir le solde du compte en USD ; b) `depuis_csv(...)` — créer une liste de comptes depuis un fichier CSV (chapitre 15) ; c) `est_jour_ouvre(...)` — dire si une date permet un virement ; d) `fermer(...)` — clôturer le compte ; e) `nb_comptes_ouverts(...)` — consulter le compteur du chapitre 20.

Solutions détaillées

Solution 21.1 —

```
class CompteBancaire:

    taux_interet = 0.02

    def __init__(self, titulaire, solde_initial=0):
        self.titulaire = titulaire
        self.solde = solde_initial

    @classmethod
    def changer_taux(cls, nouveau_taux):
        if not 0 <= nouveau_taux <= 0.20:
            raise ValueError(f'Taux invalide : {nouveau_taux}')
        cls.taux_interet = nouveau_taux

a = CompteBancaire('A'); b = CompteBancaire('B')
CompteBancaire.changer_taux(0.03)
print(a.taux_interet, b.taux_interet)    # → 0.03 0.03

try:
    CompteBancaire.changer_taux(0.5)
except ValueError as e:
    print(e)                            # → Taux invalide : 0.5
```

Voilà l'avantage du classmethod sur l'assignation brute `CompteBancaire.taux_interet = 0.5` : la validation vit dans le point d'entrée (comparaison chaînée du chapitre 6, `raise` du chapitre 13).

Solution 21.2 —

```
@classmethod
def depuis_chaine(cls, texte):
    titulaire, solde = texte.split(':')
    return cls(titulaire, float(solde))

c = CompteBancaire.depuis_chaine('Camille:250')
print(c.titulaire, c.solde)    # → Camille 250.0
```

Sans le `float(...)`, le solde resterait la chaîne `'250'` — et le premier `deposer` planterait ou concatènerait (piège des types, chapitres 3/15). Le constructeur alternatif est justement l'endroit où *centraliser* ce nettoyage.

Solution 21.3 —

```
@staticmethod
def valider_iban(iban):
    return iban.startswith('FR') and len(iban) == 27

print(CompteBancaire.valider_iban('FR' + '7' * 25))    # → True
print(CompteBancaire.valider_iban('DE89370400440532013000')) # → False
```

Le corps n'utilise ni `self` ni `cls` — règle de Corey : statique. Valider un IBAN *avant* de créer un compte n'a besoin d'aucun compte : la méthode d'instance imposerait d'avoir déjà l'objet qu'on cherche à valider, absurde. (Et le vrai contrôle IBAN — mod 97 — se brancherait ici sans changer l'interface.)

Solution 21.4 —

```
@classmethod
def commun(cls, *titulaires):
    return cls(' & '.join(titulaires), 0)

joint = CompteBancaire.commun('Mathias', 'Camille')
print(joint.titulaire)    # → Mathias & Camille
```

`*titulaires` (les `*args` du chapitre 8) accepte deux, trois, N cotitulaires ; `' & '.join(...)` vient du chapitre 4. Deux constructeurs alternatifs cohabitent sans problème — c'est toute l'élégance du motif : autant de portes d'entrée que de formats d'arrivée.

Solution 21.5 — a) **instance** : convertir LE solde de CE compte — `self.solde` est au cœur du calcul. b) **classe** : c'est un constructeur alternatif de masse — `from / depuis_csv` renvoie des `cls(...)` ; en faire un statique qui appelle `CompteBancaire(...)` en dur casserait l'héritage. c) **statique** : pur calcul calendaire, ni `self` ni `cls` (c'est `is_workday`). d) **instance** : on clôt UN compte précis. e) **classe** : consulter un état de la classe (`cls.nb_comptes`) — un statique marcherait mais cacherait la dépendance à la classe ; `cls` la rend explicite.

Fiche récap — Chapitre 21

Syntaxe	Effet
méthode d'instance	reçoit <code>self</code> — données d'un objet
<code>@classmethod + def</code> <code>m(cls, ...)</code>	reçoit la classe ; <code>cls.var = ...</code> ⇔ <code>Classe.var = ...</code>
Constructeur alternatif	classmethod <code>from_...</code> qui <code>return cls(...)</code> — autre porte d'entrée (cf. <code>date.today()</code> , <code>fromtimestamp</code> , <code>fromisoformat</code>)
<code>@staticmethod + def</code> <code>m(...)</code>	ne reçoit rien — fonction utilitaire rangée dans la classe
Règle de choix	corps sans <code>self</code> ni <code>cls</code> ⇒ statique ; touche la classe / construit ⇒ classe ; sinon instance
Appel	classmethods et staticmethods s'appellent via la classe (<code>Employee.from_string(...)</code>)
	un classmethod appelé depuis une instance modifie quand même la classe entière

CHAPITRE 22

POO 4 : l'héritage — créer des sous-classes

Vidéo source : [Python OOP Tutorial 4: Inheritance - Creating Subclasses](#) — Corey Schafer

Objectifs : hériter d'une classe pour la spécialiser sans la dupliquer ni la modifier, lire le MRO, surcharger attributs et `__init__` avec `super()`, et vérifier les liens de parenté avec `isinstance` / `issubclass`.

22.1 L'idée

L'**héritage** (*inheritance*) permet de créer une classe qui **récupère** tous les attributs et méthodes d'une classe **mère** (*parent/base class*), puis y ajoute ou y modifie ce qu'elle veut — **sans toucher à la mère**. Dans l'entreprise de Corey : les développeurs et les managers *sont* des employés (nom, paie, email, `fullname` ...) avec leurs spécificités. Plutôt que copier-coller la classe `Employee` deux fois : on hérite.

Point de départ (la classe des chapitres 19-20) :

```
class Employee:

    raise_amt = 1.04

    def __init__(self, first, last, pay):
        self.first = first
        self.last = last
        self.pay = pay
        self.email = first + '.' + last + '@company.com'

    def fullname(self):
        return f'{self.first} {self.last}'

    def apply_raise(self):
        self.pay = int(self.pay * self.raise_amt)
```

La sous-classe s'écrit en nommant sa mère entre parenthèses :

```
class Developer(Employee):
    pass
```

Et c'est déjà fonctionnel :

```
dev_1 = Developer('Corey', 'Schafer', 50000)

print(dev_1.email)      # → Corey.Schafer@company.com
print(dev_1.fullname()) # → Corey Schafer
```

`Developer` n'a **rien** défini — d'où sortent `__init__` et `fullname` ?

22.2 Le MRO : la chaîne de résolution

Quand Python ne trouve pas un attribut dans `Developer`, il remonte la chaîne d'héritage — c'est le prolongement direct de la remontée instance → classe du chapitre 20. Cette chaîne s'appelle le **MRO** (*method resolution order*), et `help()` l'affiche :

```
help(Developer)
# → Help on class Developer in module __main__:
#
# class Developer(Employee)
# | Method resolution order:
# | Developer
# | Employee
# | builtins.object
# |
# | Methods inherited from Employee:
# | __init__(self, first, last, pay)
# | apply_raise(self)
# | fullname(self)
# ...
```

Ordre de recherche : `Developer` → `Employee` → `object`. Deux leçons : nos appels ont abouti dans `Employee` (rien dans `Developer`) ; et **toute** classe Python descend d'`object`, l'ancêtre universel. `help()` sur une classe est un outil de compréhension sous-coté — la vidéo insiste, à juste titre.

22.3 Surcharger un attribut de classe

Les développeurs obtiennent 10 % d'augmentation :

```
class Developer(Employee):
    raise_amt = 1.10
```

```
emp_1 = Employee('Test', 'User', 50000)
dev_1 = Developer('Corey', 'Schafer', 50000)

emp_1.apply_raise()
dev_1.apply_raise()

print(emp_1.pay)    # → 52000    (1.04 — la mère n'a pas bougé)
print(dev_1.pay)    # → 55000    (1.10 — la sous-classe a la priorité)
```

Le MRO explique tout : pour `dev_1`, `self.raise_amt` se résout dans `Developer` (1.10) ; pour `emp_1`, dans `Employee` (1.04). Et voilà le *payoff* du choix `self.raise_amt` fait au chapitre 20 : la méthode `apply_raise`, écrite **une seule fois** dans la mère, sert tous les taux. Modifier la sous-classe n'affecte jamais la mère.

22.4 Étendre `__init__` : `super()`

Les développeurs ont un attribut de plus : leur langage principal. Il faut un `__init__` spécifique... sans recopier celui de la mère :

```
class Developer(Employee):
    raise_amt = 1.10

    def __init__(self, first, last, pay, prog_lang):
        super().__init__(first, last, pay)
        self.prog_lang = prog_lang
```

`super().__init__(...)` appelle le `__init__` de la classe mère : elle installe `first`, `last`, `pay`, `email` comme d'habitude — puis on ajoute la nouveauté. Chacun sa responsabilité, zéro duplication (DRY, toujours lui).

```
dev_1 = Developer('Corey', 'Schafer', 50000, 'Python')
dev_2 = Developer('Test', 'User', 60000, 'Java')

print(dev_1.email)      # → Corey.Schafer@company.com
print(dev_1.prog_lang)  # → Python
```

(Variante que montre la vidéo : `Employee.__init__(self, first, last, pay)` — même effet ici, mais `super()` est la forme à retenir : pas de nom de mère en dur, et un comportement correct dans les héritages complexes.)

22.5 Deuxième sous-classe : `Manager`

Un manager supervise une liste d'employés — attribut supplémentaire et méthodes supplémentaires :

```
class Manager(Employee):

    def __init__(self, first, last, pay, employees=None):
        super().__init__(first, last, pay)
        if employees is None:
            self.employees = []
        else:
            self.employees = employees

    def add_emp(self, emp):
        if emp not in self.employees:
            self.employees.append(emp)

    def remove_emp(self, emp):
        if emp in self.employees:
            self.employees.remove(emp)

    def print_emps(self):
```

```
for emp in self.employees:
    print('-->', emp.fullname())
```

```
mgr_1 = Manager('Sue', 'Smith', 90000, [dev_1])

print(mgr_1.email)    # → Sue.Smith@company.com

mgr_1.add_emp(dev_2)
mgr_1.print_emps()
# → --> Corey Schafer
# → --> Test User

mgr_1.remove_emp(dev_1)
mgr_1.print_emps()
# → --> Test User
```

Piège capital (et universel en Python) — Pourquoi `employees=None` puis un test, au lieu du direct `employees=[]` ? Parce qu'une valeur par défaut n'est évaluée qu'UNE fois, à la définition de la fonction : avec `=[]`, tous les managers créés sans liste partageraient LA MÊME liste (le bug du chapitre 20, version paramètres — ajoute un employé à l'un, tous l'ont). L'idiome consacré : défaut `None`, création du muable **dans** le corps. À appliquer à toute fonction, pas qu'aux classes.

Remarque au passage : `mgr_1.print_emps()` appelle `emp.fullname()` sur des `Developer` — les objets circulent entre classes sans friction, chacun sachant répondre de ses méthodes. C'est la POO qui commence à payer.

22.6 `isinstance` et `issubclass`

Deux fonctions intégrées pour interroger les parentés :

```
print(isinstance(mgr_1, Manager))    # → True
print(isinstance(mgr_1, Employee))    # → True      un Manager EST un Employee
print(isinstance(mgr_1, Developer))    # → False     mais pas un Developer

print(issubclass(Developer, Employee)) # → True
print(issubclass(Manager, Employee))    # → True
print(issubclass(Manager, Developer))    # → False   classes sœurs, pas mère-fille
```

`isinstance(objet, Classe)` : « cet objet est-il une instance de cette classe *ou d'une de ses sous-classes* ? » ;

`issubclass(A, B)` : « A descend-elle de B ? ». Utile pour des traitements conditionnels par type — avec modération : trop d'`isinstance` signale souvent qu'une méthode surchargée ferait mieux le travail.

Pour aller plus loin — L'héritage brille dans les bibliothèques réelles : Corey cite le framework web dont les exceptions HTTP (404 NotFound , 500 ServerError ...) sont des dizaines de sous-classes ne redéfinissant que deux attributs (code, message) au-dessus d'une mère commune. Tu peux faire pareil dès maintenant avec les exceptions du chapitre 13 : `class SoldeInsuffisant(Exception):` passe te donne une exception maison, attrapable spécifiquement (`except SoldeInsuffisant:`) — l'héritage au service de la gestion d'erreurs. Garde aussi en tête la limite : n'hérite que si la relation « **est un** » est vraie (un Developer est un Employee) ; pour « possède un », la **composition** — un attribut contenant l'autre objet, comme `Manager.employees` — est le bon outil.

Exercices

Exercice 22.1 ● — Crée `CompteEpargne(CompteBancaire)` : d'abord avec `pass`, vérifie qu'un compte épargne sait déjà tout faire (dépôt, résumé). Puis surcharge `taux_interet = 0.035` et montre qu' `appliquer_interets` (héritée, non recopiée !) sert 2 % à un compte courant et 3,5 % à l'épargne. Quel choix du chapitre 20 rend cela possible ?

Exercice 22.2 ● — Affiche `help(CompteEpargne)` et relève : le MRO complet, et la classe d'où chaque méthode est héritée.

Exercice 22.3 ●● — Crée `CompteTitres(CompteBancaire)` : `__init__` supplémentaire `positions` (dict ticker → quantité, défaut... attention au piège !), avec `super()`. Méthodes `acheter(ticker, quantite)` (met à jour le dict — motif `get` du chapitre 5) et `vendre(ticker, quantite)` (refuse par `ValueError` de vendre plus que détenu ; retire la clé si la position tombe à 0). Scénario de test complet.

Exercice 22.4 ●● — Sur tes trois classes : prédis les six résultats — `isinstance(ep, CompteBancaire)`, `isinstance(ep, CompteTitres)`, `isinstance(cb, CompteEpargne)`, `issubclass(CompteTitres, CompteBancaire)`, `issubclass(CompteEpargne, CompteTitres)`, `issubclass(CompteBancaire, object)` — puis vérifie.

Exercice 22.5 ●●● — Le retrait plafonné : surcharge la méthode `retirer` dans `CompteEpargne` pour imposer un plafond de 1 000 € par retrait (variable de classe `plafond_retrait = 1000`), tout en réutilisant la vérification de solde de la mère via `super().retirer(montant)` — n'écris PAS deux fois le test de solde. Démontre : retrait de 800 (ok), de 1 200 (refus plafond), et de 900 sur un solde de 500 (refus solde — venu de la mère). En quoi ce montage illustre-t-il « étendre sans dupliquer » ?

Solutions détaillées

Solution 22.1 —

```
class CompteEpargne(CompteBancaire):
    taux_interet = 0.035
```

```

CompteBancaire.taux_interet = 0.02      # on repart du taux standard (les ex. 20-21
l'avaient modifié)

cb = CompteBancaire('Mathias', 1000)
ep = CompteEpargne('Mathias', 1000)

cb.appliquer_interets()
ep.appliquer_interets()

print(cb.solde)      # → 1020.0
print(ep.solde)      # → 1035.0

```

Le choix payant : `appliquer_interets` lit `self.taux_interet` (chapitre 20, section 20.4) — la résolution passe par le MRO de *l'instance*, donc `CompteEpargne.taux_interet` gagne pour `ep`. Avec

`CompteBancaire.taux_interet` en dur dans la méthode, les deux comptes auraient pris 2 %.

Solution 22.2 — `help(CompteEpargne)` affiche `Method resolution order: CompteEpargne → CompteBancaire → builtins.object`, la variable `taux_interet = 0.035` définie dans `CompteEpargne`, et toutes les méthodes (`__init__`, `deposer`, `retirer`, `resume`, `appliquer_interets`...) listées sous « Methods inherited from `CompteBancaire` ».

Solution 22.3 —

```

class CompteTitres(CompteBancaire):

    def __init__(self, titulaire, solde_initial=0, positions=None):
        super().__init__(titulaire, solde_initial)
        if positions is None:
            self.positions = {}
        else:
            self.positions = positions

    def acheter(self, ticker, quantite):
        self.positions[ticker] = self.positions.get(ticker, 0) + quantite

    def vendre(self, ticker, quantite):
        detenu = self.positions.get(ticker, 0)
        if quantite > detenu:
            raise ValueError(f'Vente impossible : {quantite} > {detenu} {ticker}')
        self.positions[ticker] = detenu - quantite
        if self.positions[ticker] == 0:
            del self.positions[ticker]

ct = CompteTitres('Mathias', 5000)
ct.acheter('AAPL', 10)
ct.acheter('MSFT', 4)
ct.acheter('AAPL', 5)
print(ct.positions)      # → {'AAPL': 15, 'MSFT': 4}

ct.vendre('MSFT', 4)
print(ct.positions)      # → {'AAPL': 15}

```

```
try:
    ct.vendre('AAPL', 99)
except ValueError as e:
    print(e)                # → Vente impossible : 99 > 15 AAPL
```

Le défaut `positions=None` évite le dict partagé entre tous les comptes (piège 22.5 de la vidéo) ; `acheter` recycle le motif d'agrégation du chapitre 5 ; et `super().__init__` garde titulaire/solde sous la responsabilité de la mère.

Solution 22.4 —

```
print(isinstance(ep, CompteBancaire))    # → True      (un épargne EST un compte)
print(isinstance(ep, CompteTitres))      # → False     (classes sœurs)
print(isinstance(cb, CompteEpargne))     # → False     (la mère n'est pas sa fille !)
print(issubclass(CompteTitres, CompteBancaire)) # → True
print(issubclass(CompteEpargne, CompteTitres)) # → False
print(issubclass(CompteBancaire, object))  # → True     (tout descend d'object)
```

Le troisième est le plus instructif : l'héritage est **orienté** — toute instance de la fille est de type mère, jamais l'inverse.

Solution 22.5 —

```
class CompteEpargne(CompteBancaire):
    taux_interet = 0.035
    plafond_retrait = 1000

    def retirer(self, montant):
        if montant > self.plafond_retrait:
            raise ValueError(f'Plafond épargne dépassé : {montant} > {self.plafond_retrait} €')
        super().retirer(montant)

ep = CompteEpargne('Mathias', 2000)
ep.retirer(800)
print(ep.solde)                # → 1200

try:
    ep.retirer(1200)
except ValueError as e:
    print(e)                    # → Plafond épargne dépassé : 1200 > 1000 €

petit = CompteEpargne('Camille', 500)
try:
    petit.retirer(900)
except ValueError as e:
    print(e)                    # → Retrait de 900 € impossible : solde 500 €
```

La surcharge **ajoute** sa règle (le plafond) puis **délègue** le reste à `super().retirer` : la vérification de solde n'existe qu'à un endroit — la corriger ou l'enrichir demain profitera aux deux classes. C'est le patron de conception le plus

rentable de l'héritage : *spécialiser en tête, déléguer le tronc commun*. (Note le troisième message : il vient bien du `retirer` de la mère.)

Fiche récap — Chapitre 22

Syntaxe	Effet
<code>class Fille(Mere):</code>	hérite de tout ; <code>pass</code> suffit pour démarrer
MRO	ordre de résolution : Fille → Mère → ... → <code>object</code> ; visible via <code>help(Fille)</code>
Surcharge d'attribut	<code>raise_amt = 1.10</code> dans la fille — la mère intacte (grâce à <code>self.</code> dans les méthodes)
<code>super().__init__(...)</code>	délègue l'initialisation commune à la mère, puis on ajoute le spécifique
Surcharge de méthode	redéfinir + <code>super().methode(...)</code> pour réutiliser le tronc commun
<code>employees=None</code> + test	jamais de défaut muable (<code>=[]</code>) : évalué une fois, partagé entre appels
<code>isinstance(obj, C)</code>	instance de C ou d'une sous-classe ?
<code>issubclass(A, B)</code>	A descend-elle de B ? (orienté !)
Quand hériter	relation « est un » ; sinon composition (« possède un »)
Bonus	<code>class MonErreur(Exception): pass</code> — exceptions maison attrapables spécifiquement

CHAPITRE 23

POO 5 : les méthodes spéciales (dunder)

Vidéo source : [Python OOP Tutorial 5: Special \(Magic/Dunder\) Methods](#) — Corey Schafer

Objectifs : donner à tes objets un affichage digne de ce nom avec `__repr__` et `__str__`, comprendre la surcharge d'opérateurs (`__add__`, `__len__` ...), et savoir quand y recourir.

23.1 L'idée : brancher tes objets sur la syntaxe de Python

Les **méthodes spéciales** — encadrées de doubles underscores, d'où leur surnom **dunder** (*double underscore*) — sont les prises électriques par lesquelles Python branche sa syntaxe sur tes objets. Tu en connais déjà une : `__init__`, appelée par la syntaxe `Employee(...)`.

L'indice qui montre qu'elles existent partout, ouvert par la vidéo :

```
print(1 + 2)          # → 3
print('a' + 'b')      # → ab
```

Le **même** opérateur `+` additionne ou concatène **selon le type** : chaque type définit sa réaction à `+` via sa méthode `__add__`. C'est la **surcharge d'opérateurs** (*operator overloading*), et elle t'est ouverte.

Le problème concret qui motive le chapitre :

```
emp_1 = Employee('Corey', 'Schafer', 50000)
print(emp_1)      # → <__main__.Employee object at 0x7f8a5c2d90>
```

Illisible. On peut mieux.

23.2 `__repr__` : la représentation pour développeurs

`__repr__` doit renvoyer une représentation **non ambiguë**, destinée au débogage. La règle d'or (recommandation officielle, reprise par Corey) : viser une chaîne qui permettrait de **recréer l'objet** — du code Python valide :

```
class Employee:

    def __init__(self, first, last, pay):
        self.first = first
        self.last = last
        self.pay = pay
        self.email = first + '.' + last + '@company.com'

    def fullname(self):
        return f'{self.first} {self.last}'
```

```
def __repr__(self):
    return f"Employee('{self.first}', '{self.last}', {self.pay})"
```

```
emp_1 = Employee('Corey', 'Schafer', 50000)
print(emp_1)    # → Employee('Corey', 'Schafer', 50000)
```

Copiable-collable dans un interpréteur pour reconstruire un employé identique : le debug dit tout.

23.3 `__str__` : la représentation pour humains

`__str__` vise l'utilisateur final — libre et lisible :

```
def __str__(self):
    return f'{self.fullname()} - {self.email}'
```

```
print(emp_1)    # → Corey Schafer - Corey.Schafer@company.com
```

Qui est appelé quand ? `print` (et `str(...)`) utilisent `__str__` s'il existe, sinon se rabattent sur `__repr__`. D'où la règle pratique : définis **toujours** `__repr__` au minimum (le filet de sécurité) ; ajoute `__str__` quand l'objet a un affichage « grand public ». Les deux restent accessibles explicitement :

```
print(repr(emp_1))    # → Employee('Corey', 'Schafer', 50000)
print(str(emp_1))     # → Corey Schafer - Corey.Schafer@company.com
```

(Ces appels ne font que déclencher `emp_1.__repr__()` et `emp_1.__str__()` — la vidéo le montre en les appelant directement.) Un exemple `stdlib` éclairant, avec les dates du chapitre 17 : `repr(d)` → `datetime.date(2026, 8, 24)` (recréable), `str(d)` → `2026-08-24` (lisible). Deux publics, deux formats.

23.4 Surcharger l'arithmétique : `__add__`

Retour au mystère du `+`. La vidéo révèle l'envers du décor :

```
print(int.__add__(1, 2))    # → 3        ce que fait vraiment 1 + 2
print(str.__add__('a', 'b')) # → ab      ce que fait vraiment 'a' + 'b'
```

Donnons un sens à `emp_1 + emp_2` — disons, la somme des salaires (exemple volontairement artificiel, Corey le concède) :

```
def __add__(self, other):
    return self.pay + other.pay
```

```
emp_1 = Employee('Corey', 'Schafer', 50000)
emp_2 = Employee('Test', 'User', 60000)
```

```
print(emp_1 + emp_2)    # → 110000
```

`self` est l'opérande de gauche, `other` celui de droite. Sans cette méthode, `emp_1 + emp_2` lèverait `TypeError: unsupported operand type(s) for +: 'Employee' and 'Employee'` — le comportement par défaut, et le bon si l'addition n'a pas de sens pour ton type.

Pour aller plus loin — `NotImplemented` — Que faire si `other` n'est pas du bon type ? La convention `stdlib` (Corey ouvre le code source de `timedelta.__add__` pour le montrer) : renvoyer la sentinelle `NotImplemented`.

```
def __add__(self, other):
    if not isinstance(other, Employee):
        return NotImplemented
    return self.pay + other.pay
```

Python essaie alors la méthode symétrique de l'autre opérande (`other.__radd__`), et ne lève `TypeError` que si personne ne sait faire. Ta classe devient un citoyen poli du système de types.

Il existe un dunder par opérateur : `__sub__` (-), `__mul__` (*), `__truediv__` (/), les comparaisons `__eq__` (==), `__lt__` (<)... La liste complète vit dans la doc (« data model ») — inutile de la mémoriser, il faut savoir qu'elle existe.

23.5 `__len__` et le principe général

```
print(len('test'))      # → 4
print('test'.__len__()) # → 4    len() ne fait qu'appeler __len__
```

Pour nos employés :

```
def __len__(self):
    return len(self.fullname())
```

```
print(len(emp_1))    # → 13    ('Corey Schafer')
```

Le principe général, maintenant limpide : **chaque syntaxe de Python** (`print`, `+`, `len`, `[]`, `in`, `==`, boucles `for`...) **délègue à une méthode dunder**. Implémenter la méthode, c'est rendre la syntaxe disponible pour ton type. Toute la « magie » des chapitres 2 à 7 — `len('abc')`, `'x' in liste`, `d[cle]` — n'était que des dunder des types intégrés.

Piège / bon goût — La surcharge est un pouvoir à manier sobrement : ne surcharge que si le sens est **évident** pour le lecteur (`+` qui concatène deux portefeuilles : oui ; `+` qui envoie un email : non). Dans le doute, une méthode nommée (`fusionner`, `total_paie`) est plus claire qu'un opérateur détourné. La lisibilité, encore et toujours, est le juge.

Exercices

Exercice 23.1 ● — Donne à `CompteBancaire` un `__repr__` fidèle à la règle d'or (`CompteBancaire('Mathias', 1500)`) et un `__str__` lisible (`Compte de Mathias - 1500.00 €` — recycle ton `resume()` du chapitre 19, ou remplace-le). Montre `print(c)`, `repr(c)`, `str(c)`.

Exercice 23.2 ● — Avant d'exécuter : que fait `print(c)` si la classe ne définit QUE `__repr__` ? Que `__str__` ? Ni l'un ni l'autre ? Vérifie les trois configurations et tire la règle.

Exercice 23.3 ●● — Implémente `__len__` sur `CompteTitres` (chapitre 22) : le nombre de lignes du portefeuille (`len(self.positions)`). Puis `__contains__(self, ticker)` pour que `'AAPL' in ct` fonctionne. (Indice : `in` délègue à `__contains__` ; renvoie un booléen — le dict sait déjà faire, délègue-lui.)

Exercice 23.4 ●● — Implémente `__eq__` sur `CompteBancaire` : deux comptes sont égaux si même titulaire ET même solde. Vérifie qu'avant l'implémentation, `CompteBancaire('A', 100) == CompteBancaire('A', 100)` vaut `False` (pourquoi ? — chapitre 6 !) et `True` après.

Exercice 23.5 ●●● — Une classe `Portefeuille` : `__init__(self, positions=None)` (dict ticker → valeur en €), `__repr__`, `__add__` qui fusionne deux portefeuilles en **additionnant** les positions communes (rends un nouveau `Portefeuille`, sans modifier les opérandes — motif du chapitre 5), et `__len__`. Vérifie : `Portefeuille({'AAPL': 1000, 'MSFT': 500}) + Portefeuille({'AAPL': 200, 'NVDA': 300})` → positions `{'AAPL': 1200, 'MSFT': 500, 'NVDA': 300}`, et que les deux portefeuilles d'origine sont intacts. Pourquoi ce `+` -là est-il du *bon* usage de la surcharge ?

Solutions détaillées

Solution 23.1 —

```
class CompteBancaire:

    def __init__(self, titulaire, solde_initial=0):
        self.titulaire = titulaire
        self.solde = solde_initial

    def __repr__(self):
        return f"CompteBancaire('{self.titulaire}', {self.solde})"

    def __str__(self):
        return f"Compte de {self.titulaire} - {self.solde:.2f} €"

c = CompteBancaire('Mathias', 1500)
print(c)          # → Compte de Mathias - 1500.00 €
print(repr(c))    # → CompteBancaire('Mathias', 1500)
print(str(c))     # → Compte de Mathias - 1500.00 €
```

Solution 23.2 — Que `__repr__` : `print(c)` affiche la forme `repr` (le fallback joue). Que `__str__` : `print(c)` affiche la forme `str`... mais `repr(c)` retombe sur l'illisible `<__main__... at 0x...>` — debug appauvri. Ni l'un ni l'autre : `<__main__... at 0x...>` partout (le `__repr__` par défaut hérité d'`object` — chapitre 22 !). Règle confirmée : `__repr__` d'abord, toujours ; `__str__` en bonus.

Solution 23.3 —

```
class CompteTitres(CompteBancaire):
    # ... (__init__, acheter, vendre du chapitre 22)

    def __len__(self):
        return len(self.positions)

    def __contains__(self, ticker):
        return ticker in self.positions

ct = CompteTitres('Mathias', 5000)
ct.acheter('AAPL', 10)
ct.acheter('MSFT', 4)

print(len(ct))          # → 2
print('AAPL' in ct)     # → True
print('NVDA' in ct)     # → False
```

Double délégation élégante : notre `__len__` délègue au `len` du dict, notre `__contains__` au `in` du dict — on ne recode rien, on *route* la syntaxe vers la bonne donnée.

Solution 23.4 — Avant : `False`. Sans `__eq__`, l'`==` hérité d'`object` compare l'*identité* (`is`, chapitre 6) — deux objets distincts, même contenu, ne sont pas « égaux ». Après :

```
class CompteBancaire:

    def __init__(self, titulaire, solde_initial=0):
        self.titulaire = titulaire
        self.solde = solde_initial

    def __eq__(self, other):
        if not isinstance(other, CompteBancaire):
            return NotImplemented
        return self.titulaire == other.titulaire and self.solde == other.solde

print(CompteBancaire('A', 100) == CompteBancaire('A', 100))  # → True
print(CompteBancaire('A', 100) == CompteBancaire('A', 999))  # → False
```

C'est exactement ce que font `str`, `list`, `date`... : leurs `__eq__` comparent le contenu — voilà pourquoi `[1,2] == [1,2]` était `True` au chapitre 6.

Solution 23.5 —

```

class Portefeuille:

    def __init__(self, positions=None):
        self.positions = {} if positions is None else dict(positions)

    def __repr__(self):
        return f'Portefeuille({self.positions})'

    def __len__(self):
        return len(self.positions)

    def __add__(self, other):
        if not isinstance(other, Portefeuille):
            return NotImplemented
        fusion = dict(self.positions) # copie – ne pas muter self !
        for ticker, valeur in other.positions.items():
            fusion[ticker] = fusion.get(ticker, 0) + valeur
        return Portefeuille(fusion)

p1 = Portefeuille({'AAPL': 1000, 'MSFT': 500})
p2 = Portefeuille({'AAPL': 200, 'NVDA': 300})
total = p1 + p2

print(total) # → Portefeuille({'AAPL': 1200, 'MSFT': 500, 'NVDA': 300})
print(p1)    # → Portefeuille({'AAPL': 1000, 'MSFT': 500}) (intact)
print(len(total)) # → 3

```

Bon usage, car l'analogie est parfaite : « additionner » deux portefeuilles a UN sens naturel et attendu (l'agrégation des positions) — le lecteur devine le comportement sans ouvrir le code, critère ultime de la section 23.5. Et le `+` respecte le contrat des types intégrés : il **renvoie du neuf** sans modifier les opérandes (comme `[1] + [2]` ou `'a' + 'b'`) — d'où la copie initiale et le `Portefeuille(fusion)` final.

Fiche récap — Chapitre 23

Dunder	Déclenché par	À retenir
<code>__init__</code>	<code>Classe(...)</code>	déjà vu — le constructeur
<code>__repr__</code>	<code>repr(x)</code> , debug, fallback de print	toujours le définir ; viser une chaîne recréant l'objet
<code>__str__</code>	<code>print(x)</code> , <code>str(x)</code>	lisible pour l'utilisateur ; optionnel (fallback → repr)
<code>__add__(self, other)</code>	<code>x + y</code>	surcharge d'opérateur ; renvoyer du neuf ; <code>NotImplemented</code> si type inconnu
<code>__eq__</code>	<code>x == y</code>	défaut hérité d'object = identité (<code>is</code>) ; définir pour comparer le contenu

Dunder	Déclenché par	À retenir
<code>__len__</code>	<code>len(x)</code>	déléguer à la donnée interne quand c'est naturel
<code>__contains__</code>	<code>x in y</code>	idem — router la syntaxe
Principe	toute syntaxe Python délègue à un dunder	doc « data model » pour la liste complète
Sobriété	surcharger seulement si le sens est évident	sinon, méthode nommée

CHAPITRE 24

POO 6 : properties — getters, setters, deleters

Vidéo source : [Python OOP Tutorial 6: Property Decorators - Getters, Setters, and Deleters](#) — Corey Schafer

Objectifs : transformer une méthode en attribut calculé avec `@property`, contrôler l'assignation avec un setter, et comprendre pourquoi ce mécanisme est la réponse Python au dilemme getters/setters des autres langages.

24.1 Le problème : l'attribut périmé

Version simplifiée de la classe, comme dans la vidéo :

```
class Employee:

    def __init__(self, first, last):
        self.first = first
        self.last = last
        self.email = first + '.' + last + '@email.com'

    def fullname(self):
        return f'{self.first} {self.last}'

emp_1 = Employee('John', 'Smith')
```

Changeons le prénom :

```
emp_1.first = 'Jim'

print(emp_1.first)          # → Jim
print(emp_1.fullname())     # → Jim Smith          à jour
print(emp_1.email)         # → John.Smith@email.com  PÉRIMÉ !
```

Asymétrie révélatrice : `fullname()` est **recalculé à chaque appel** à partir de l'état courant — il ne peut pas se périmiser. `email`, lui, a été **figé une fois pour toutes** dans `__init__` : la donnée dérivée s'est désynchronisée de sa source.

Première idée : transformer `email` en méthode `email()`, comme `fullname`. Ça corrige la fraîcheur... mais **casse tout le code existant** : tous les utilisateurs de la classe écrivent `emp_1.email` (attribut) et devraient réécrire `emp_1.email()` (appel). Pour une classe diffusée, rupture d'interface inacceptable — le dilemme est posé.

24.2 `@property` : une méthode déguisée en attribut

Le décorateur `@property` (cousin des `@classmethod` / `@staticmethod` du chapitre 21) résout le dilemme : on définit une **méthode**, on y accède comme un **attribut** :

```

class Employee:

    def __init__(self, first, last):
        self.first = first
        self.last = last

    @property
    def email(self):
        return f'{self.first}.{self.last}@email.com'

    @property
    def fullname(self):
        return f'{self.first} {self.last}'

emp_1 = Employee('John', 'Smith')
emp_1.first = 'Jim'

print(emp_1.first)      # → Jim
print(emp_1.email)      # → Jim.Smith@email.com      recalculé — SANS parenthèses !
print(emp_1.fullname)   # → Jim Smith                idem

```

Trois observations :

- plus de `self.email = ...` dans `__init__` : l'email n'est plus un état **stocké** mais une valeur **dérivée**, calculée à la demande — il ne peut plus se périmier *par construction* ;
- l'appelant écrit `emp_1.email`, sans parenthèses : l'**interface n'a pas bougé**, aucun code client à modifier ;
- `fullname` reçoit le même traitement — cohérence de l'interface (plus de mélange `email` / `fullname()`).

C'est la grande leçon design du chapitre : en Python, on **commence simple** (attributs publics nus) et, le jour où une logique devient nécessaire, on glisse une property **sans rien casser**. Dans d'autres langages (Java...), l'impossibilité de faire ça pousse à écrire préventivement des getters/setters partout ; en Python, c'est inutile — le filet de sécurité existe.

24.3 Le setter : contrôler l'assignation

Nouveau besoin, symétrique : pouvoir écrire `emp_1.fullname = 'Corey Schafer'` et que prénom/nom se mettent à jour. Sans rien faire :

```

# emp_1.fullname = 'Corey Schafer'
# AttributeError: can't set attribute

```

(Une property est en lecture seule par défaut — déjà une protection utile.) Le **setter** s'ajoute avec un décorateur au nom composé — `@<nom_de_la_property>.setter` — et une méthode du **même nom** :

```

@fullname.setter
def fullname(self, name):
    first, last = name.split(' ')

```

```
self.first = first
self.last = last
```

```
emp_1 = Employee('John', 'Smith')
emp_1.fullname = 'Corey Schafer'

print(emp_1.first)      # → Corey
print(emp_1.last)       # → Schafer
print(emp_1.email)      # → Corey.Schafer@email.com
```

L'assignation `emp_1.fullname = ...` déclenche la méthode setter, qui **répartit** la valeur sur les attributs sources — et l'email dérivé suit, évidemment. (Le déballage `first, last = name.split(' ')` : chapitres 4 et 8.)

Piège de syntaxe — Le setter exige : 1) que la `@property` du même nom existe **au-dessus** dans la classe ; 2) le décorateur `@fullname.setter` (pas `@setter`, pas `@property.setter`) ; 3) une méthode nommée **exactement** `fullname`. Trois occasions de typo, un seul message d'erreur peu bavard — vérifie ces trois points en cas de souci.

24.4 Le deleter

Dernier morceau, plus rare : intercepter `del emp_1.fullname` :

```
@fullname.deleter
def fullname(self):
    print('Delete Name!')
    self.first = None
    self.last = None
```

```
del emp_1.fullname
# → Delete Name!
print(emp_1.first)  # → None
```

Utile pour du nettoyage à la suppression — sache qu'il existe, tu l'utiliseras rarement.

24.5 Bilan : quand utiliser une property ?

- **Attribut dérivé** d'autres attributs (email, fullname, aire d'un cercle, valeur d'un portefeuille) : property en lecture — jamais de donnée dupliquée qui se périmé.
- **Validation à l'assignation** (interdire un solde négatif, une date incohérente) : property + setter qui vérifie avant d'écrire.
- **Compatibilité** : transformer un attribut public existant en valeur calculée, sans casser les appelants — LA raison d'être historique.

Et le contre-usage : une property qui déclenche un calcul **coûteux** ou une action inattendue (requête réseau !) trahit le lecteur — la syntaxe attribut promet quelque chose d'instantané et d'anodin. Dans ce cas, une méthode explicite (`calculer_...()`) est plus honnête.

Pour aller plus loin — Les properties concluent la série POO de Corey, et tu as désormais le socle complet : classes/instances, variables de classe, les trois espèces de méthodes, l'héritage, les dunders, les properties. La suite logique de son catalogue, quand tu t'y sentiras prêt : *dataclasses* (qui automatisent `__init__` / `__repr__` / `__eq__`), *itérateurs et générateurs*, *décorateurs* (le mécanisme derrière tous ces `@`), et *context managers* (écrire son propre `with` — chapitre 12 côté coulisses).

Exercices

Exercice 24.1 ● — Reprends `CompteBancaire` et transforme le `resume()` du chapitre 19 en property `resume`. Vérifie l'accès sans parenthèses, puis que l'assignation `c.resume = 'x'` échoue — quel est le message, et en quoi est-ce une protection ?

Exercice 24.2 ● — Ajoute à `CompteTitres` (chapitres 22-23) une property `nb_lignes` (nombre de positions) et une property `plus_grosse_ligne` qui renvoie le ticker de la plus grande quantité (`max(d, key=d.get)` — ou `None` si le portefeuille est vide : truthiness !).

Exercice 24.3 ●● — Protège le solde : renomme l'attribut stocké en `_solde` (underscore initial : convention « interne, ne pas toucher »), expose une property `solde` en lecture, et un setter qui lève `ValueError` sur toute assignation négative. Vérifie que `deposer` / `retirer` (qui font `self.solde += ...`) passent DÉSORMAIS par le setter — et donc que même un bug interne ne peut plus rendre le solde négatif.

Exercice 24.4 ●● — Une classe `Rectangle(largeur, hauteur)` avec les properties `aire` et `perimetre`, et une classe `Cercle(rayon)` avec `aire` et `circonference` (`math.pi`). Pourquoi properties plutôt qu'attributs calculés dans `__init__` ? Démontre le problème de l'alternative en modifiant `rayon` après création.

Exercice 24.5 ●●● — Le grand final, tout le cours en une classe : `Obligation` (chapitre 19, ex. 19.5) revue — attributs `nominal`, `taux_coupon`, `maturite` ; property `coupon` (montant annuel) ; méthode `prix(taux)` ; property `annees_restantes` calculée depuis une date d'émission `datetime.date` (chapitre 17) ; `__repr__` recréable (chapitre 23) ; constructeur alternatif `depuis_chaine('1000;0.05;10')` (chapitre 21) ; et un setter sur `taux_coupon` qui refuse les taux hors `[0, 1]` (chapitre 13). Écris le scénario de démonstration qui exerce chaque pièce.

Solutions détaillées

Solution 24.1 —

```
class CompteBancaire:
```

```

def __init__(self, titulaire, solde_initial=0):
    self.titulaire = titulaire
    self.solde = solde_initial

@property
def resume(self):
    return f'Compte de {self.titulaire} — solde : {self.solde:.2f} €'

c = CompteBancaire('Mathias', 1500)
print(c.resume)      # → Compte de Mathias — solde : 1500.00 €

# c.resume = 'x'      # AttributeError: property 'resume' of 'CompteBancaire' object has no
#                      # setter

```

Sans setter, la property est en **lecture seule** : impossible d'écraser par mégarde une valeur qui n'est pas un état mais un calcul. L'erreur immédiate vaut mieux qu'une donnée fantôme.

Solution 24.2 —

```

class CompteTitres(CompteBancaire):
    # ... __init__, acheter, vendre (ch. 22)

    @property
    def nb_lignes(self):
        return len(self.positions)

    @property
    def plus_grosse_ligne(self):
        if not self.positions:
            return None
        return max(self.positions, key=self.positions.get)

ct = CompteTitres('Mathias')
print(ct.plus_grosse_ligne)  # → None
ct.acheter('AAPL', 10); ct.acheter('MSFT', 25)
print(ct.nb_lignes, ct.plus_grosse_ligne)  # → 2 MSFT

```

`max(d, key=d.get)` : itérer un dict donne ses clés (chapitre 5), et `key=` (aperçu au chapitre 4) les départage par leur valeur associée — deux chapitres qui se serrent la main.

Solution 24.3 —

```

class CompteBancaire:

    def __init__(self, titulaire, solde_initial=0):
        self.titulaire = titulaire
        self.solde = solde_initial          # passe déjà par le setter !

    @property
    def solde(self):

```

```

        return self._solde

    @solde.setter
    def solde(self, valeur):
        if valeur < 0:
            raise ValueError(f'Solde négatif interdit : {valeur}')
        self._solde = valeur

    def deposer(self, montant):
        self.solde += montant

    def retirer(self, montant):
        self.solde -= montant          # plus besoin de test explicite !

c = CompteBancaire('Mathias', 100)
c.deposer(50)
print(c.solde)          # → 150

try:
    c.retirer(500)
except ValueError as e:
    print(e)             # → Solde négatif interdit : -350
print(c.solde)          # → 150    (l'assignation a été refusée AVANT d'écrire)

```

Le point remarquable : `self.solde += montant` se déplie en `self.solde = self.solde + montant` → lecture par le getter, **écriture par le setter** → validation automatique, partout, y compris dans `__init__` (un `CompteBancaire('X', -50)` est refusé d'office). L'invariant « solde ≥ 0 » est désormais inviolable par construction — la version aboutie de la protection artisanale du chapitre 19.

Solution 24.4 —

```

import math

class Rectangle:
    def __init__(self, largeur, hauteur):
        self.largeur = largeur
        self.hauteur = hauteur

    @property
    def aire(self):
        return self.largeur * self.hauteur

    @property
    def perimetre(self):
        return 2 * (self.largeur + self.hauteur)

class Cercle:
    def __init__(self, rayon):
        self.rayon = rayon

    @property

```

```

def aire(self):
    return math.pi * self.rayon ** 2

@property
def circonference(self):
    return 2 * math.pi * self.rayon

c = Cercle(1)
print(round(c.aire, 4))          # → 3.1416
c.rayon = 2
print(round(c.aire, 4))          # → 12.5664      suit le rayon

```

L'alternative (`self.aire = math.pi * rayon ** 2` dans `__init__`) recréerait *exactement* le bug de l'email : après `c.rayon = 2`, l'aire stockée dirait encore 3.1416. Morale générale : **ne stocke jamais ce qui se déduit** — calcule-le. (Même philosophie qu'en base de données ou qu'en maths : pas de redondance non contrôlée.)

Solution 24.5 —

```

import datetime

class Obligation:

    def __init__(self, nominal, taux_coupon, maturite, emission=None):
        self.nominal = nominal
        self.taux_coupon = taux_coupon          # via le setter → validé
        self.maturite = maturite
        self.emission = emission if emission is not None else datetime.date.today()

    def __repr__(self):
        return f'Obligation({self.nominal}, {self._taux_coupon}, {self.maturite})'

    @property
    def taux_coupon(self):
        return self._taux_coupon

    @taux_coupon.setter
    def taux_coupon(self, valeur):
        if not 0 <= valeur <= 1:
            raise ValueError(f'Taux de coupon invalide : {valeur}')
        self._taux_coupon = valeur

    @property
    def coupon(self):
        return self.nominal * self.taux_coupon

    @property
    def annees_restantes(self):
        echeance = self.emission + datetime.timedelta(days=365 * self.maturite)
        return max(0, (echeance - datetime.date.today()).days) / 365

    @classmethod
    def depuis_chaine(cls, texte):

```

```

    nominal, taux, maturite = texte.split(';')
    return cls(float(nominal), float(taux), int(maturite))

def prix(self, taux_actuariel):
    total = sum(self.coupon / (1 + taux_actuariel) ** t
                for t in range(1, self.maturite + 1))
    return total + self.nominal / (1 + taux_actuariel) ** self.maturite

# — démonstration —
o = Obligation.depuis_chaine('1000;0.05;10')      # constructeur alternatif (ch. 21)
print(o)                                           # → Obligation(1000.0, 0.05, 10) (ch.
23)
print(o.coupon)                                   # → 50.0
(property)
print(f'{o.prix(0.05):.2f}')                       # → 1000.00 (au pair — ch. 19)
print(f'{o.prix(0.03):.2f}')                       # → 1170.60 (taux ↓ ⇒ prix ↑)
print(round(o.annees_restantes, 2))               # → 10.0 environ (ch. 17)

try:
    o.taux_coupon = 1.5                            # setter → validation (ch. 13/24)
except ValueError as e:
    print(e)                                         # → Taux de coupon invalide : 1.5

```

Inventaire des chapitres mobilisés : f-strings (2), arithmétique et validation d'intervalle (3, 6), `split`/déballage (4, 8), défaut `None` pour l'objet muable/dynamique (22), `datetime` et `timedelta` (17), compréhension/génératrice dans `prix` (14), `raise` (13), classmethod (21), property + setter (24), `__repr__` (23), et la relation inverse taux-prix en prime. Si ce code te paraît *naturel* à lire — c'est que le cours a fait son travail.

Fiche récap — Chapitre 24

Syntaxe	Effet
<code>@property + def attr(self):</code>	méthode accessible sans parenthèses — attribut calculé, jamais périmé
Sans setter	lecture seule : l'assignation lève <code>AttributeError</code>
<code>@attr.setter + def attr(self, valeur):</code>	intercepte <code>obj.attr = ...</code> — validation, répartition ; même nom, property définie au-dessus
<code>@attr.deleter</code>	intercepte <code>del obj.attr</code> (rare)
<code>self._attr</code>	convention « interne » : le stockage derrière la property
<code>obj.attr += x</code>	lecture getter + écriture setter → validation automatique partout
Philosophie	attributs simples d'abord ; property le jour venu, sans casser l'interface
Ne stocke pas le dérivable	calcule-le (sinon : données périmées, le bug de l'email)
Contre-usage	calcul coûteux/action cachée derrière une syntaxe d'attribut